

Universidad Nacional Mayor de San Marcos

Universidad del Perú. Decana de América

Dirección General de Estudios de Posgrado

Facultad de Ingeniería de Sistemas e Informática

Unidad de Posgrado

Herramienta para el modelado y generación de código de Arquitecturas de Software basadas en Microservicios

TESIS

Para optar el Grado Académico de Magíster en Ingeniería de Sistemas e Informática con mención en Ingeniaría de Software

AUTOR

Waldo Jefferson TREBEJO LOAYZA

ASESOR

Mg. Fany Yexenia SOBERO RODRÍGUEZ

Lima, Perú

2023



Reconocimiento - No Comercial - Compartir Igual - Sin restricciones adicionales

https://creativecommons.org/licenses/by-nc-sa/4.0/

Usted puede distribuir, remezclar, retocar, y crear a partir del documento original de modo no comercial, siempre y cuando se dé crédito al autor del documento y se licencien las nuevas creaciones bajo las mismas condiciones. No se permite aplicar términos legales o medidas tecnológicas que restrinjan legalmente a otros a hacer cualquier cosa que permita esta licencia.

Referencia bibliográfica

Trebejo, W. (2023). *Herramienta para el modelado y generación de código de Arquitecturas de Software basadas en Microservicios*. [Tesis de maestría, Universidad Nacional Mayor de San Marcos, Facultad de Ingeniería de Sistemas e Informática, Unidad de Posgrado]. Repositorio institucional Cybertesis UNMSM.

Metadatos complementarios

Datos de autor	
Nombres y apellidos	Waldo Jefferson Trebejo Loayza
Tipo de documento de identidad	DNI
Número de documento de identidad	45975287
URL de ORCID	https://orcid.org/0009-0008-5011-6315
Datos de asesor	
Nombres y apellidos	Fany Yexenia Sobero Rodríguez
Tipo de documento de identidad	DNI
Número de documento de identidad	20120467
URL de ORCID	https://orcid.org/0000-0002-0323-6110
Datos del jurado	
Presidente del jurado	
Nombres y apellidos	Cayo Víctor León Fernández
Tipo de documento	DNI
Número de documento de identidad	07001405
Miembro del jurado 1	
Nombres y apellidos	Frank Edmundo Escobedo Bailón
Tipo de documento	DNI
Número de documento de identidad	41671087
Miembro del jurado 2	
Nombres y apellidos	Juan Gamarra Moreno
Tipo de documento	DNI
Número de documento de identidad	20039857
Miembro del jurado 3	
Nombres y apellidos	Fany Yexenia Sobero Rodríguez
Tipo de documento	DNI

Número de documento de identidad	20120467
Datos de investigación	
Línea de investigación	C.0.3.22. Ingeniería de software
Grupo de investigación	Gididoc
Agencia de financiamiento	Sin financiamiento.
Ubicación geográfica de la investigación	Universidad Nacional Mayor de San Marcos País: Perú Departamento: Lima Provincia: Lima Distrito: Lima Latitud: -12.056445 Longitud: -77.085994
Año o rango de años en que se realizó la investigación	2021 - 2022
URL de disciplinas OCDE	Otras ingenierías y tecnologías https://purl.org/pe-repo/ocde/ford#2.11.02

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
Universidad del Perú. Decana de América
Facultad de Ingeniería de Sistemas e Informática
Vicedecanato de Investigación y Posgrado
Unidad de Posgrado

ACTA DE SUSTENTACIÓN VIRTUAL DE TESIS PARA OPTAR EL GRADO ACADÉMICO DE MAGÍSTER EN INGENIERÍA DE SISTEMAS E INFORMÁTICA CON MENCIÓN EN INGENIERÍA DE SOFTWARE

A los doce (12) días del mes de mayo de 2023, siendo las 15:45 pm., se reunieron en la sala virtual ***https://meet.google.com/mjd-qosv-bwz*** *el Jurado de Tesis conformado por los siguientes docentes:*

Dr. Cayo Víctor León Fernández (Presidente)
Dr. Frank Edmundo Escobedo Bailón (Miembro)
Mg. Juan Gamarra Moreno (Miembro)
Mg. Fany Yexenia Sobero Rodríguez (Miembro Asesor)

Se inició la Sustentación invitando al candidato a Magíster ***WALDO JEFFERSON TREBEJO LOAYZA,*** *para que realice la exposición oral y virtual de la tesis para optar el Grado Académico de Magíster en Ingeniería de Sistemas e Informática con mención en Ingeniería de Software, siendo la Tesis intitulada:*

"HERRAMIENTA PARA EL MODELADO Y GENERACIÓN DE CÓDIGO DE ARQUITECTURAS DE SOFTWARE BASADAS EN MICROSERVICIOS"

Concluida la exposición, los miembros del Jurado de Tesis procedieron a formular sus preguntas que fueron absueltas por el graduando; acto seguido se procedió a la evaluación correspondiente, habiendo obtenido la siguiente calificación:

.....***DIECINUEVE (19) - EXCELENTE*** .....

Por tanto, el presidente del Jurado, de acuerdo con el Reglamento General de Estudios de Posgrado, otorga al Bachiller ***WALDO JEFFERSON TREBEJO LOAYZA*** *el Grado Académico de Magíster en Ingeniería de Sistemas e Informática con mención en Ingeniería de Software.*

Siendo las 16:46 horas, el presidente del Jurado de Tesis, da por concluido el acto académico de Sustentación de Tesis.

Dr. Cayo Víctor León Fernández
(Presidente)

Dr. Frank Edmundo Escobedo Bailón
(Miembro)

Mg. Juan Gamarra Moreno
(Miembro)

Mg. Fany Yexenia Sobero Rodríguez
(Miembro Asesor)

Firmado digitalmente por LEON FERNANDEZ Cayo Victor FAU 20148092282 soft
Motivo: Soy el autor del documento
Fecha: 25.01.2023 14:45:02 -05:00

Universidad del Perú. Decana de América
Facultad de Ingeniería de Sistemas e Informática
Vicedecanato de Investigación y Posgrado
Unidad de Posgrado

Lima, 25 de Enero del 2023

INFORME N° 000002-2023-UPG-VDIP-FISI/UNMSM

INFORME DE EVALUACIÓN DE ORIGINALIDAD

1. Autoridad Académica que emite el Informe de Originalidad:	*Director de la Unidad de Posgrado*
2. Apellidos y Nombres de la autoridad académica:	*Dr. Cayo Víctor León Fernández*
3. Operador del programa informático de similitudes:	*Dr. Cayo Víctor León Fernández*
4. Documento evaluado:	*Tesis para Posgrado* *Título: "Herramienta para el modelado y generación de código de Arquitecturas de Software basadas en Microservicios"*
5. Autor del documento:	*Waldo Jefferson Trebejo Loayza*
6. Fecha de recepción de documento	*20/01/2023*
7. Fecha de aplicación del programa detector de similitudes:	*20/01/2023*
8. Software utilizado:	*Turnitin*
9. Configuración del programa detector de similitudes:	*i. Excluye textos entrecomillados: SI* *ii. Excluye biografías: SI* *iii. Excluye cadenas menores a 40 palabras: SI* *iv. Otro criterio (especificar): NO*
10. Porcentaje de similitudes según programa detector de similitudes	*Tres por ciento (3%)*
11. Fuentes originales de las similitudes encontradas	*Se adjuntan en cuatro (04) fojas al presente informe*
12. Observaciones:	*Ninguna*
13. Calificación de originalidad *i. Documento cumple criterios de originalidad, sin observaciones.* *ii. Documento cumple criterior de originaldiad, con observaciones.* *iii. Documento no cumple criterios de originalidad.*	*Documento cumple criterio de originalidad, sin observaciones.*
14. Fecha del Informe:	*23/01/2023*

DR. CAYO VICTOR LEON FERNANDEZ
DIRECTOR DE LA UNIDAD DE POSGRADO(e)

CLF/rda

Esta es una copia auténtica imprimible de un documento electrónico archivado por la Universidad Nacional Mayor de San Marcos, aplicando lo dispuesto por el Art. 25 de D.S. 070-2013-PCM y la Tercera Disposición Complementaria Final del D.S. 026- 2016-PCM. Su autenticidad e integridad pueden ser contrastadas a través de la siguiente dirección web: **https://spsgd.unmsm.edu.pe/verifica/inicio.do** e ingresando el siguiente código de verificación: **BOXVAQR**

Dedicatoria

Dedico esta tesis a mi familia quienes han estado apoyándome en todo momento, por sus palabras de aliento, sus consejos, paciencia y motivación, dándome las energías positivas para lograr mi objetivo propuesto de alcanzar el grado de magister.

Agradecimiento

A la profesora Mg. Fany Sobero Rodríguez, por su tiempo y apoyo constante en la asesoría y revisión del presente trabajo para culminarlo satisfactoriamente.

Al profesor Mg. Efraín Bautista Ubillús, por su orientación y paciencia para iniciar el proceso de elaboración del presente trabajo.

A los profesores del programa de maestría de la UNMSM, por compartir experiencias y conocimientos durante el desarrollo del programa, que fue de gran apoyo para la elaboración del presente trabajo.

A Dios, por darme la oportunidad de alcanzar un logro más en mi vida profesional.

Índice general

Índice general iv
Lista de cuadros vii
Lista de figuras viii
Resumen x
Abstract xii
CAPITULO 1: INTRODUCCIÓN 13
1.1. Situación Problemática 13
1.2. Formulación del problema 16
1.2.1. Problema General 16
1.2.2. Problemas Específicos 16
1.3. Justificación teórica 17
1.4. Justificación práctica 18
1.5. Objetivos 19
1.5.1. Objetivo General 19
1.5.2. Objetivos Específicos 19
CAPITULO 2: MARCO TEÓRICO 20
2.1. Antecedentes de Investigación 20
2.2. Bases Teóricas 48
2.2.1. Software 48
2.2.2. Ingeniería de Software 48
2.2.3. Arquitectura de Software 49
2.2.4. Diseño Arquitectónico 49
2.2.5. Patrones de Diseño 49
2.2.6. Modelo 50
2.2.7. EventStorming 50
2.2.8. Domain Driven Design – DDD 51
2.2.9. Diseño Estratégico 53
2.2.9.1. Domino 53
2.2.9.2. Subdominio 54
2.2.9.3. Modelo de Dominio 54
2.2.9.4. Bounded Context 54
2.2.9.5. Context Map 55
2.2.10. Diseño Táctico 55

2.2.10.1. Agregados 56
2.2.10.2. Entidades 56
2.2.10.3. Objetos de Valor 56
2.2.11. Microservicios 56
2.2.12. Api 57
2.2.13. Api Rest 57
2.2.14. Api Gateway 58
2.2.15. RESTful 59
2.2.16. Comunicación entre Microservicios 59
2.2.17. Mensajería Asíncrona 59
2.2.18. Event Store 59
2.2.19. Event Bus 60
2.2.20. Balanceo de Carga 60
2.2.21. CQRS 60
2.2.22. Event Sourcing 61
2.2.23. Base de Datos 62
CAPÍTULO 3: METODOLOGÍA 63
3.1. Tipo y diseño de la investigación 63
3.2. Unidad de análisis 63
3.3. Población de estudio 63
3.4. Tamaño de la muestra 63
3.5. Selección de muestra 64
3.6. Técnicas de recolección de Datos 64
3.7. Análisis e interpretación de la información 64
CAPÍTULO 4: IMPLEMENTACIÓN DE LA HERRAMIENTA 65
4.1. Definición del requerimiento 65
4.1.1. Herramientas y tecnologías a emplear 65
4.2. Modelado de procesos de la herramienta propuesta 65
4.3. Metodología para el desarrollo 67
4.3.1. Definición de Roles 67
4.3.2. Definición de Historias de Usuario 68
4.3.3. Definición de Sprint Backlog 70
4.4. Desarrollo e incremento del sprint 1 71
4.4.1. Diagrama de Casos de Uso 71
4.4.2. Modelos de Datos 72

4.4.3. Diseño de la Arquitectura 72
4.4.4. Estructura del Proyecto 73
4.4.5. Interfaz de la herramienta (Aplicación Web) 82
4.5. Desarrollo e incremento del sprint 2 83
4.6. Desarrollo e incremento del sprint 3 84
4.7. Desarrollo e incremento del sprint 4 86
4.8. Desarrollo e incremento del sprint 5 88
4.9. Resumen de Capítulo 90
CAPÍTULO 5: RESULTADOS Y DISCUSIÓN 92
5.1. Análisis, interpretación y discusión de resultados 92
5.1.1. Objetivos de la validación de la herramienta 92
5.1.2. Diseño de la validación 92
5.1.3. Ejecución de la validación 93
5.2. Presentación de resultados 94
CONCLUSIONES 100
RECOMENDACIONES 102
REFERENCIAS BIBLIOGRÁFICAS 103
ANEXOS 111
Anexo 1. Operacionalización de las variables 111
Anexo 2. Matriz de Consistencia 112
Anexo 3. Encuesta de Satisfacción de usuario Post-Uso 113
Anexo 4. Capturas del taller de validación de la herramienta 114

Lista de cuadros

Cuadro 1 Subdominio del sistema de contratación electrónica 21
Cuadro 2 Servicios para las aplicaciones MGM, Original y LGM. 39
Cuadro 3 Técnicas e Instrumentos.....64
Cuadro 4 Definición de roles de Scrum para el proyecto.....67
Cuadro 5 Definición de Historias de Usuario 69
Cuadro 6 Definición del Sprint Backlog – Scrum 70
Cuadro 7 Tiempo de esfuerzo tomado con el uso de la herramienta.....94
Cuadro 8 Resultados de la encuesta de satisfacción de usuario post-Uso.....97
Cuadro 9 Validación de confiabilidad del instrumento. 97
Cuadro 10 Resultados de la encuesta por subescala.....98

Lista de figuras

Figura 1 Modelo operativo de forma inteligente.....23
Figura 2 Proceso de aprobación del formulario. 24
Figura 3 Modelo jerárquico.....25
Figura 4 Mapa de contexto de ejemplo de escenario de seguros.....26
Figura 5 Mapeador de contexto: metamodelo estratégico de DDD (diagrama de clases UML).....27
Figura 6 Mapa de navegación para diseño controlado por dominio en MSA 29
Figura 7 Ejemplo de ejecución basado en DDD. 30
Figura 8 Metamodelo para modelado basado en DDD.....32
Figura 9 Metamodelo enriquecido con capacidades semánticas a través de DomainConcept.....33
Figura 10 El diagrama de clases UML para el ejemplo de carga original basado en DDD. 36
Figura 11 El diagrama de clases UML para los MGM del ejemplo de carga. 37
Figura 12 El diagrama de clases UML para los LGM del ejemplo de carga 38
Figura 13 El diagrama de clases UML para los MGM del ejemplo de eShopOnContainers. 39
Figura 14 Patrón básico de la arquitectura de Microservicios 42
Figura 15 Reemplazo de la dependencia del código con llamadas de servicio..44
Figura 16 Strangler Application 45
Figura 17 Arquitectura de microservicios.....47
Figura 18 Visión simplificada del desarrollo de sistemas 48
Figura 19 Descripción general del lenguaje de patrones 52
Figura 20 Esquema de comunicación de la Arquitectura 53
Figura 21 Bounded Contexts 55
Figura 22 Arquitectura de API Gateway.....58
Figura 23 Patrón CQRS 61
Figura 24 Modelado de los procesos de la herramienta propuesta. 66
Figura 25 El Marco Scrum 67
Figura 26 Diagrama general de caso de uso 71
Figura 27 Modelo de datos 72
Figura 28 Arquitectura de la herramienta.....73
Figura 29 Estructura del proyecto.....73
Figura 30 Estructura de la capa Dominio.....74
Figura 31 Estructura de la capa Persistencia.....74
Figura 32 Componentes necesarios para migración de los datos.....75

Figura 33 Implementación de la capa Persistencia....75
Figura 34 Definición de la conexión a la base de datos. 76
Figura 35 Listado de tablas creados desde el proyecto. 77
Figura 36 Diagrama de la base de datos....77
Figura 37 Estructura de la capa Aplicación del proyecto 78
Figura 38 Estructura de la capa Utilitario....78
Figura 39 Estructura de la capa WebApp 79
Figura 40 Arquitectura de aplicación Web con el patrón MVC 79
Figura 41 Estructura del modelo en la capa WebApp....80
Figura 42 Estructura de la vista en la capa WebApp 81
Figura 43 Estructura del controlador en la capa WebApp....81
Figura 44 Estructura del proyecto generator-springboot....82
Figura 45 Pantalla principal de la herramienta web 83
Figura 46 Formulario de registro de Dominio y Lenguaje Ubicuo....83
Figura 47 Formulario de registro de Subdominios 84
Figura 48 Formulario de registro de Contexto Limitado 84
Figura 49 Formulario de registro de Entidades....85
Figura 50 Formulario de registro de Objetos de Valor, Eventos y Comandos....85
Figura 51 Formulario para ingreso de parámetros....86
Figura 52 Formulario para Generación de Código....87
Figura 53 Formulario para Generación de Agregados - Entidades 87
Figura 54 Opción de descarga de código generado 88
Figura 55 Microservicios generados con la herramienta....89
Figura 56 Ejecución del microservicio solicituddecredito 89
Figura 57 Resultado de la ejecución usando Swagger 90
Figura 58 Dominio analizado con el taller Event Storming....93
Figura 59 El PSSUQ versión 3. 97
Figura 60 Resultado de encuesta de satisfacción por subescalas....99

Resumen

Construir un buen software toma un tiempo determinado según los requerimientos y la rapidez en el desarrollo. En la actualidad diferentes empresas que ofrecen servicios digitalizados para sus clientes y usuarios tienen la necesidad de contar con software robustos y de fácil uso, con diferentes atributos de calidad que les permitan realizar diferentes operaciones de manera rápida y segura, a su vez, mantener el software en constante mejora, constante actualización y con alternativas de migrar a nuevas tecnologías, pero estos deben ser construidos en el menor tiempo posible para lanzar nuevas versiones mejoradas. Muchas empresas siguen usando sistemas legados como la principal herramienta para los procesos de negocios, otras empresas usan sistemas obsoletos que limitan a realizar actualizaciones o migraciones a tecnologías modernas, convirtiendo este escenario en una amenaza de seguridad a la información que operan, comprometiendo el rendimiento y productividad de las empresas.

En la presente tesis se implementa una herramienta web para el modelado y generación de código de arquitecturas de software basada en Microservicios empleando el enfoque Diseño guiado por el dominio (DDD) con el propósito de facilitar y agilizar la construcción de proyectos de software y migración de sistemas legados en nuevos servicios independientes, empleando tecnologías modernas para contar con la Interoperabilidad, Seguridad, Escalabilidad, Modularidad, entre otros atributos de calidad que se requieren, con ello garantizar la productividad permanente de las empresas sin afectar la operatividad de los sistemas existentes, sino realizar mejoras de manera independiente por cada proceso de negocio.

Para comprobar el aporte de la herramienta web, se realizó la validación con profesionales expertos en desarrollo de software teniendo como resultado un tiempo de esfuerzo empleado de 26.5 minutos por participante siendo este un resultado muy aceptable. También se realizado una encuesta de satisfacción de usuario post-Uso, obteniendo la aprobación de la usabilidad, la eficiencia y eficacia que aporta la herramienta en la fase de modelado y generación de

código de arquitectura, desde la descomposición del dominio hasta la obtención de la estructura del proyecto en un corto tiempo.

Palabras Clave: Arquitectura de software; microservicios; diseño arquitectónico.

Abstract

Building good software takes a certain amount of time depending on the requirements and the speed of development. At present, different companies that offer digitized services for their clients and users have the need to have robust and easy-to-use software, with different quality attributes that allow them to carry out different operations quickly and safely, in turn, maintain the software in constant improvement, constant updating and with alternatives to migrate to new technologies, but these must be built in the shortest possible time to launch new improved versions. Many companies continue to use legacy systems as the main tool for business processes, other companies use obsolete systems that limit updates or migrations to modern technologies, turning this scenario into a security threat to the information they operate, compromising performance and business productivity.

In this thesis, a web tool for the modeling and code generation of software architectures based on Microservices is implemented using the Domain Driven Design (DDD) approach with the purpose of facilitating and speeding up the construction of software projects and the migration of legacy systems into new independent services, using modern technologies to have Interoperability, Security, Scalability, Modularity, among other quality attributes that are required, thereby guaranteeing the permanent productivity of companies without affecting the operation of existing systems, but rather make improvements independently for each business process.

To verify the contribution of the web tool, validation was carried out with professional experts in software development, resulting in an effort time of 26.5 minutes per participant, this being a very acceptable result. A post-Use user satisfaction survey was also carried out, obtaining the approval of the usability, efficiency and effectiveness that the tool provides in the phase of modeling and generation of architecture code, from the decomposition of the domain to obtaining the approval. project structure in a short time.

Keywords: Software architecture; microservices; architectural design.

CAPITULO 1: INTRODUCCIÓN

1.1. Situación Problemática

Construir un producto software potente con diferentes atributos de calidad y capacidad para cubrir las necesidades de los procedimientos de negocio de una entidad es el objetivo principal en la ejecución de proyectos de software; puesto que, el procedimiento de construcción de software es flexible si se considera un enfoque de microservicios (López & Maya, 2017). Sin embargo, no siempre se alcanza el objetivo deseado por diferentes factores que se pueden presentar en el proceso de análisis conceptuales, funcionales, ecológicos, técnicos y formales, diseño de la arquitectura y la construcción del software (Gatell, 2019).

A medida que la tecnología avanza, las necesidades de los clientes y usuarios son más exigentes con los servicios digitales que se ofrecen; es decir, es necesario realizar actividades que promuevan una mejor comunicación cliente-vendedor, que indiquen un servicio de alta calidad (Cañarte & Morán, 2021). Por otro lado, cada organización empresarial que brinda servicios digitales se ven obligados a mantener los sistemas de información operativos, actualizados, en constante mejora y siempre disponibles para controlar, automatizar y optimizar los procesos de negocio.

Este hecho genera la necesidad de contar con softwares cada vez más robustos y de fácil uso por lo que deben ser construidos en el menor tiempo posible para lanzar nuevas versiones mejoradas. En este orden de ideas, la tecnología de desarrollo de un software hoy penetra en diversos lugares económicos, su uso permite tratar pacientes en clínicas, prevenir accidentes de aeropuertos, tránsito, energía, administrar sistemas de transporte público, etc (Villadiego, 2020).

La parte principal de aplicaciones en los teléfonos y tabletas inteligentes es el software, ya que facilita conectarse y realizar diversas operaciones del día a día, como son las transacciones bancarias, en

adquirir servicios o productos y en realizar todo tipo de actividades (Villadiego, 2020).

En la actualidad, los desarrollos de software se enfocan en ambientes web lo cual requieren una serie de características a cumplir como la mensajería o seguridad, control de usuarios entre otros, los días se vuelven más complejas en el desarrollo y su mantenimiento (Sayago, 2018). Contar con servicios en tecnologías web requiere diferentes procesos a tener en cuenta desde su entendimiento de los requerimientos, el análisis, el desarrollo, el despliegue en producción y el mantenimiento basándose en los resultados esperados de su aplicación (Ministerio de vivienda, 2021). Por ello, es muy importante en un principio realizar el modelado de la arquitectura de software que garantice y cubra todas las necesidades que requiere un software de calidad para conocer las características de las distintas tecnologías en disponibilidad del mercado y entender a fondo el modelo de negocio al cual se automatizará.

Para diseñar una arquitectura de software exitoso y desarrollar el producto software de calidad, es necesario entender los requerimientos de los usuarios. Asimismo, varios investigadores han propuesto técnicas y modelos para obtener información de los requisitos y las técnicas y recomendaciones más detalladas para mantenerlos, pero pocos especifican cómo documentarlos en esta etapa (Sommerville, 2005).

Aunque la importancia de la planificación de requisitos y la obtención de información es ampliamente reconocida en la sociedad como un paso importante en este proceso de ingeniería de software, se han registrado vacíos de trabajos para guiar a ingenieros en la documentación de esta obtención de información. Por tanto, la importancia de una apropiada documentación permite un mejoramiento de comprensión de las necesidades que presenta los clientes o usuarios; en este orden apoya a los ingenieros en conocer mejor el

problema para así modelar una solución que se manifieste correctamente especificado (Serna & Hernán, 2016).

En el desarrollo de software, uno debe enfrentar varios problemas, como la velocidad de uso del software, el alto nivel de avance y complejidad de los sistemas actuales, y otros problemas que se desarrollan cada vez más avanzando dificultosos (Bachmann et al., 2000). En este contexto, la existencia de un modelo arquitectónico en la fase inicial se vuelve indiscutible, la expectativa de requerimientos del sistema permite un modelo avanzado de solución alternativa a las exigencias generadas, que en continuas mejoras conduce al producto de aspecto final (Meaurio & Schmieder, 2013).

En los inicios del uso de la Red, no se contaba con numerosas aplicaciones de internet, solo se codificaban aplicaciones para los sistemas operativos existentes como Windows, Linux y Macintosh. Entregar el software consistía en copiar un código en un servidor o generar un instalador para el sistema operativo y grabarlos en un CD-ROM para luego comercializarlos. Se realizan lanzamientos cada año o en más tiempo, si había errores tomaban corregirlos meses; las versiones desarrolladas eran monolíticas y cada versión debía ser mejor que la anterior. Con la llegada de las aplicaciones de software como servicios (SaaS) es más rápido gestionar el desarrollo y lanzamiento. Las aplicaciones SaaS cuentan con una API y pueden comunicarse con diferentes tecnologías. Además, pueden corregirse, actualizarse y desplegarse en minutos en lugar de meses, lo que podemos observar que esta tecnología ha cambiado esencialmente la manera en que se entrega y desarrolla el software (Hodges, 2022).

Las empresas que utilizan Software as a Service (SaaS) han obtenido ventajas como: acceso a las mejores soluciones tecnológicas para diferentes compañías sin considerar la dimensión, servicio continuo, actualizaciones y perfeccionamientos gratuitas, precio más económico al cambiar del modelo fijo a otro modelo de suscripción que permite escalabilidad, una fácil unificación con otros sistemas, ahorro de costos

para otros y beneficios; de esta forma, el uso de software en la nube en los próximos años impulsará repentinamente sectores que cambian su dinámica de mercado, donde incluso los datos que proporciona jugarán un papel clave en las nuevas estrategias comerciales de la compañía (Jiménez, 2021).

En síntesis, para desarrollar un producto software que tengan los principales atributos de calidad como escalabilidad, interoperabilidad, desplegabilidad, seguridad, modularidad entre otros, es importante diseñar correctamente la arquitectura de software; es importante, entender bien el negocio y dividirlos en varias partes para conocer su proceso de funcionamiento. Ante esta necesidad, surge la alternativa de emplear un estilo arquitectónico como es el de Microservicios con el cual se podrá crear aplicaciones modernas con servicios independientes y migrar sistemas legados a nuevas tecnologías; pero todo ello, no será posible si no se parte desde el análisis de los requerimientos y el respectivo modelado de las arquitecturas para alcanzar un producto software con la calidad requerida en un menor tiempo posible.

1.2. Formulación del problema

1.2.1. Problema General

¿Es posible implementar una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios para el proceso de desarrollo de proyectos de software y la migración de sistemas legados?

1.2.2. Problemas Específicos

¿Es posible desarrollar un proyecto software y la migración de sistemas legados de manera Eficaz mediante una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios?

¿Es posible desarrollar un proyecto software y la migración de sistemas legados de manera Efectiva mediante una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios?

¿Es posible desarrollar un proyecto software y la migración de sistemas legados mediante una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios con facilidad de uso?

¿Es posible validar la herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios?

1.3. Justificación teórica

El presente estudio propone desarrollar una herramienta tecnológica en entorno web para el modelado y la generación de códigos de arquitecturas de software basadas en microservicios tomando como base la exploración del negocio mediante el Diseño Guiado por el Dominio (DDD). Para ello, se realiza la revisión de diferentes artículos científicos relacionados al estilo arquitectónico de microservicios y domain driven design (DDD) para evaluar la superioridad e inferioridad que conserva el estilo arquitectónico de microservicios y los componentes que conforman el domain driven design (DDD), donde se pudo observar gran porcentaje de trabajos que emplean el DDD para construir microservicios de forma rápida y con buenos resultados.

La herramienta propuesta está orientada para la creación de nuevos proyectos y migraciones de sistemas legados a tecnologías modernas donde ya existe el dominio, comenzando con el análisis del negocio hasta obtener el diseño de arquitectura, apoyando a los arquitectos de software y desarrolladores en tomar mejores decisiones para optar las diferentes técnicas y tecnologías a emplear en un proyecto de desarrollo planificado para garantizar un software de calidad. Asimismo, toda

empresa busca contar con software de apoyo en sus procesos de negocios, por ello es necesario modernizar, migrar o realizar nuevos desarrollos de software en el menor tiempo posible, es ahí donde la herramienta propuesta entra a cumplir un rol muy importante de orientar, facilitar y agilizar la generación de código para una arquitectura adecuada del software que se requiere obtener empleando diferentes tecnologías.

1.4. Justificación práctica

La herramienta propuesta partirá empleando el enfoque Diseño Guiado por el Dominio (DDD) desde el proceso de descomposición del dominio, identificación de los subdominios, seguido con el registro de cada dominio y subdominio identificado, definición y registro de los contextos limitados (Bounded Context), todo ello por la herramienta web (formularios) para finalmente identificar cada microservicio y obtener el código de la arquitectura lista para compilar y ejecutar. Hoy en día, el arquitecto o el diseñador de software tienen a disposición una gran cantidad de plantillas y lenguajes de plantillas, todos los cuales son el resultado de la experiencia y los grandes avances en esta ciencia, que permite dar a conocer diversos problemas de arquitectura con mayor calidad y rapidez a todo momento (Jimenez-Torres et al., 2014).

Los arquitectos de software y desarrolladores podrán usar la herramienta propuesta para modelar y generar código de la arquitectura basadas en microservicios para tener como base una estructura de la solución, con el cual podrán continuar con el desarrollo del proyecto software adicionando nuevas funcionalidades y tecnologías para completar los componentes del proyecto, con la ventaja de contar con diferentes alternativas para diseñar la estructura final, lo que permitirá ahorrar tiempo en la entrega del producto software final.

1.5. Objetivos

1.5.1. Objetivo General

Implementar una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios para el desarrollo de proyectos de software y la migración de sistemas legados.

1.5.2. Objetivos Específicos

Desarrollar un proyecto software y la migración de sistemas legados de manera Eficaz mediante una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios.

Desarrollar un proyecto software y la migración de sistemas legados de manera Efectiva mediante una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios.

Desarrollar un proyecto software y la migración de sistemas legados mediante una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios con facilidad de uso.

Validar la herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios.

CAPITULO 2: MARCO TEÓRICO

2.1. Antecedentes de Investigación

Se han hallado diferentes investigaciones referentes a implementar herramientas tecnológicas para mecanizar los procesos de generación de modelos y códigos de software, para ello es importante la evaluación y revisión de otras investigaciones realizadas.

2.1.1. Fajar et al. (2020) desarrollaron un trabajo de investigación titulado "***Design and Implementation of Microservices System Based on Domain-Driven Design***" tomando como base un software existente de una empresa de TI que ha sido desarrollado un producto básico de sistema de aplicaciones de contratación electrónica para cubrir necesidades de automatización en el proceso de contratación interna de una empresa, donde hacen mención las diferentes desventajas que se presenta al querer mejorar y actualizar el software. Como obras relacionadas hacen mención diferentes conceptos relacionados al negocio de la empresa y se centran como una alternativa de mejora al estilo arquitectónico de microservicos haciendo una comparación con los monolitos, para tener como alternativa el empleo de DDD en el enfoque para el modelado de los microservicos. Con los análisis que realizan, y viendo las dificultades que genera una arquitectura monolítico a los grandes cambios y avances para los nuevos enfoques de desarrollo, se ven con la necesidad de trasladar la complejidad al nivel de coordinación con la arquitectura de microservicios el cual les ayudara a la empresa a integrar los procesos comerciales existentes, respaldar la infraestructura de tecnología de la indagación y también los componentes de servicio que se pueden reutilizar y combinar de acuerdo con las prioridades comerciales.

Como resultado y discusión del trabajo de investigación, lo clasifican en 4 grupos, partiendo desde el análisis comercial de contratación electrónica, análisis de dominio, definir contexto acotado y el diseño arquitectónico de microservicios.

A. Proceso comercial de contratación electrónica: donde describen el flujo de módulos general del sistema de contratación electrónica existente en la empresa. Se detallan las funcionalidades de cada módulo para entender todo el flujo que se requiere migrar a la nueva arquitectura.

B. Análisis de dominio: hacen mención el enfoque DDD que puede ayudar a diseñar los microservicios, DDD comienza con el modelado del dominio empresarial y la generación de modelos de dominio, ya que es un guía abstracto del dominio empresarial. Este proceso consiste en organizar y filtrar el conocimiento del dominio y proporcionar un lenguaje común para desarrolladores.

Después del análisis del dominio realizado, se propone la identificación del dominio que se puede dividir en varios subdominios, que los representan en el siguiente cuadro.

Cuadro 1 **Subdominio del sistema de contratación electrónica**

No	Subdomain	Domain Type
1	Budget	Core Domain
2	Purchase Request	Core Domain
3	Purchase Order	Core Domain
4	Bidding	Core Domain
5	Item	Core Domain
6	Catalog	Supporting Domain
7	Vendor management	Core Domain
8	Goods Receipts	Supporting Domain
9	Goods Retum	Supporting Domain
10	Asset	Supporting Domain
11	Approval Management	Generic Domain

Realizan las divisiones de los subdominios clasificándolos como principales, de apoyo y genéricos.

C. Definir contexto acotado: en este proceso realizan la identificación de los contextos limitados, donde hacen mención de que este proceso

proporcionará claridad sobre qué modelo se utiliza, dónde debe ser coherente y qué se debe ignorar. Mediante un gráfico, explican los límites de cada subdominio existente, los cuales se convertirán en un referente en el modelado de dominios y determinará los servicios a construir.

Luego de la identificación de los servicios, los clasifican en una serie de procesos comerciales, funciones comerciales y resultados de la gestión del desarrollo de funciones comerciales y de la identificación de microservicios con los resultados de los estudios de modelado con el enfoque DDD. Seguidamente listan los servicios que se desarrollaran en la fase de definición de la arquitectura de microservicios.

D. Propuesta de diseño arquitectónico de microservicios: con los servicios identificados, realizaron la propuesta del diseño de la arquitectura de microservicios agregando diferentes elementos como complemento.

Concluyeron mencionando, que el análisis y bosquejo de la arquitectura de microservicios empleando el enfoque de diseño impulsado por dominios puede ayudar en el proceso de identificación de microservicios, y aplicar microservicios en el sistema de la empresa en estudio es de gran ayuda ya que permitirá dividir los módulos en servicios independientes para un fácil mantenimiento y nuevos desarrollos de la aplicación.

2.1.2. Chen et al. (2019) elaboraron un artículo titulado "***Business Generator Model Based on Domain Driven Design***" donde proponen un esquema de diseño de modelo de generador de negocios que se logra mediante la metodología de diseño de modelos controlados por dominios y la implementación de códigos controlados por modelos. Se basan en el enfoque del diseño basado en dominios (DDD) comparando los resultados obtenidos de la arquitectura de sistemas construido con DDD el cual es mejor que una arquitectura heredada y

que la escalabilidad, el rendimiento y la calidad del código del nuevo sistema han mejorado significativamente.

Como primera parte, emplearon el Modelo Operativo del Formulario Inteligente, con esta tecnología los desarrolladores y administradores pueden crear sistemas de aplicaciones de forma rápida y sencilla.

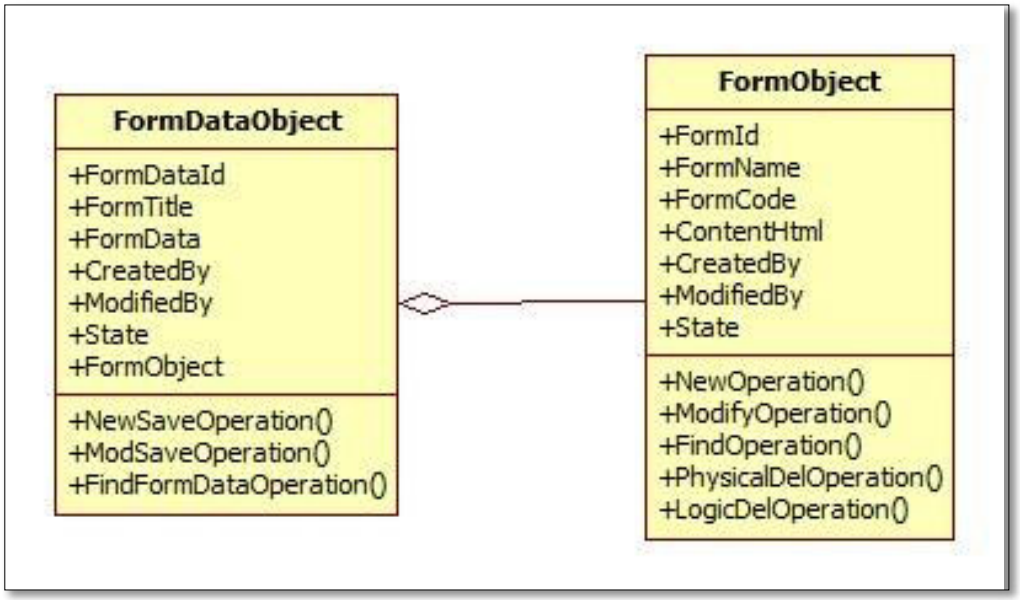


Figura 1 **Modelo operativo de forma inteligente.**

En el modelo de funcionamiento del formulario, incluyeron las operaciones básicas como la inserción, eliminación y modificación de los formularios inteligentes, y para el modelo de manipulación de datos del formulario detallaron que se incluye la creación y guardado de operaciones de datos del formulario, modificación y guardado de operaciones de datos del formulario y la búsqueda de operaciones de formulario.

Como segunda parte, emplearon el Modelo de Operación de Flujo de Trabajo, lo que se puede considerar como una operación compuesta sobre las operaciones del objeto comercial. Como propuesta de ejemplo, mencionan que un proceso de aprobación de formulario puede verse como la inserción de una operación de proceso durante la creación de un nuevo formulario del objeto de negocio, luego de que se

pasa la aprobación, el objeto de formulario se crea correctamente. Una vez que falla la aprobación, falla la creación del objeto de formulario.



Figura 2 **Proceso de aprobación del formulario.**

Como tercera parte, emplearon el Modelo de Generador de Negocios DDDBizModel, donde mencionan que es un conjunto de modelos de generador de negocios basado en un diseño impulsado por dominios, el cual consiste en entidad, objeto de valor, agregación, almacenamiento, repositorio y servicio. De los cuales, la entidad, el objeto de valor y la agregación no solo tienen datos, sino que también tienen comportamientos comerciales enriquecedores.

Según el enfoque de diseño dirigida por dominios, el modelo de generador de negocios se divide en capa de dominio, aplicación, presentación de infraestructura, siendo el núcleo la capa de dominio.

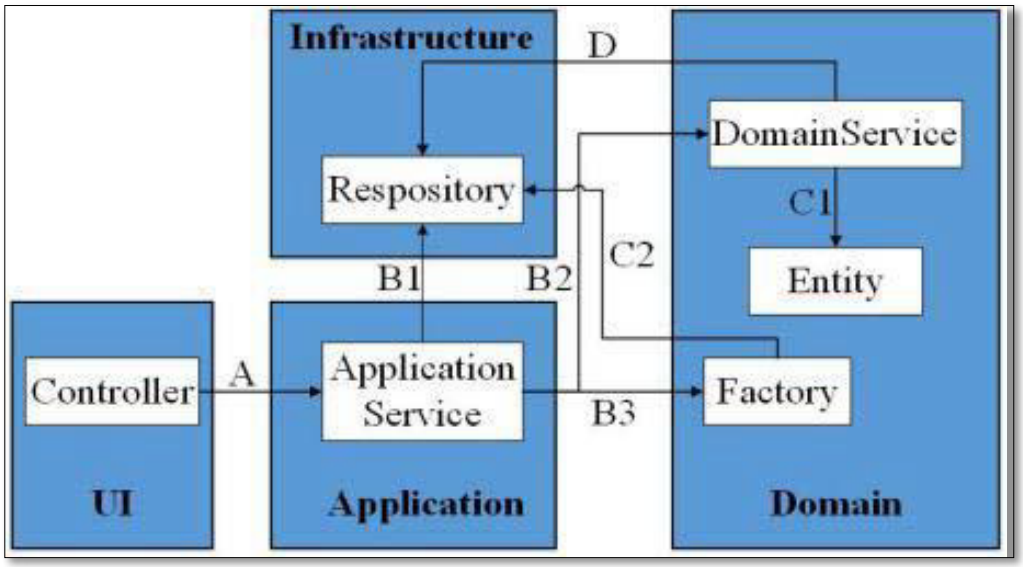


Figura 3 **Modelo jerárquico.**

Los autores concluyeron mencionando que el articulo presentado propone un modelo de generador de negocio DDDBozModel centrado en un diseño dirigido por dominios, donde incluyen el siguiente punto:

Modelización de campo de operaciones sobre objetos de negocio y unificación de operaciones, se presenta un modelo generador de negocio centrado en DDD y de acuerdo con el modelo, se implementa el marco de desarrollo y se encapsulan las lógicas comerciales públicas, como el flujo de trabajo, las operaciones de formulario y las operaciones de datos de formulario, lo que puede respaldar mejor el diseño y el desarrollo orientado al dominio, mejorar la eficiencia del desarrollo y reducir los defectos.

2.1.3. Kapferer & Zimmermann (2020) elaboraron un estudio titulado "***Domain-specific Language and Tools for Strategic Domain-driven Design, Context Mapping and Bounded Context Modeling***" donde presentan un lenguaje especifico de dominio (DSL) para los patrones estratégicos de DDD y, como otra contribución, extraen un metamodelo que suministra una definición concisa de patrones y su aplicabilidad, además mencionan que con la herramienta elaborada pueden identificar servicios con cohesión y acoplamiento rozables.

Los autores parten desde la investigación de un metamodelo DDD estratégico con reglas semánticas que describen las interpretaciones de los patrones que realizan, luego presentan la sintaxis del DSL para seguidamente analizar los escenarios de uso y aplicabilidad del enfoque presentado.

Como un ejemplo, hacen un análisis mencionando que el diseño estratégico impulsado por el dominio (DDD) se puede utilizar para descomponer el dominio del problema de un sistema de software en múltiples subdominios y los llamados Contextos delimitados. También

permite a los arquitectos definir las relaciones entre contextos delimitados, por ejemplo, cómo trabajan juntos.

Presentan una ilustración del Mapa de Contexto del escenario inspirado en las visualizaciones de Vernon, Brandolini y Plöd; con el propósito de explicar en qué consiste un mapa de contexto, por ello detallan: “Un mapa de contexto proporciona una vista global de todos los contextos delimitados relacionados con el que está trabajando un equipo.”

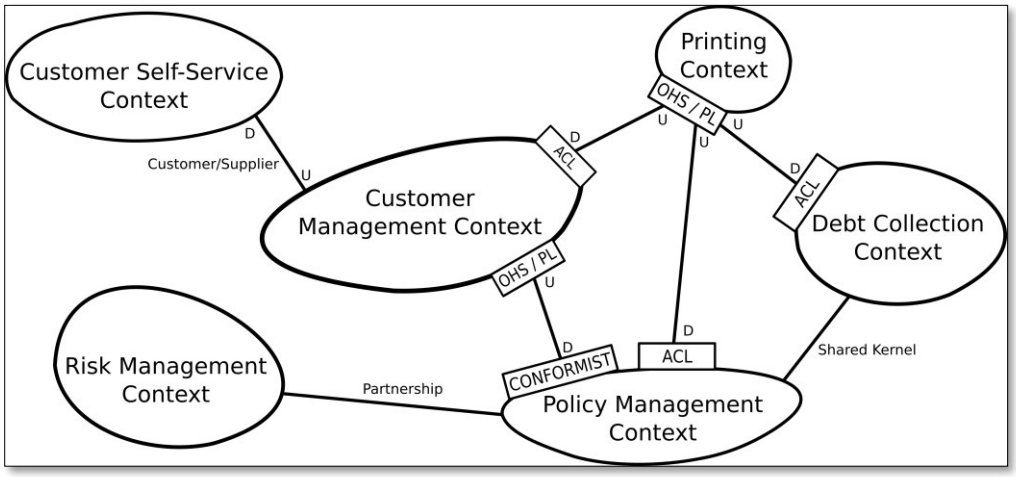


Figura 4 **Mapa de contexto de ejemplo de escenario de seguros**

De la ilustración anterior, realizan varias menciones para dar a entender el mensaje, donde mencionan que las relaciones aguas arriba-aguas abajo están marcadas con un U para aguas arriba y un D para aguas abajo, estos términos río arriba y río abajo se utilizan en DDD para describir relaciones en las que solo un contexto limitado influye en el otro; la corriente arriba influye en la corriente abajo. Por lo tanto, el contexto acotado descendente depende del modelo de dominio del contexto acotado ascendente, pero no al revés. Por otra parte, hacen mención de los patrones Published Language (PL), Open Host Service (OHS), Anticorruption Layer (ACL) and Conformist (CF) los cuales se utilizan para describir la interacción entre contextos delimitados en una relación ascendente descendente.

Los autores, también hacen mención del metamodelo que han implementado, que se basa en los patrones estratégicos del DDD, el

cual tiene como elemento central el mapa de contexto donde se muestra los contextos delimitados y sus relaciones.

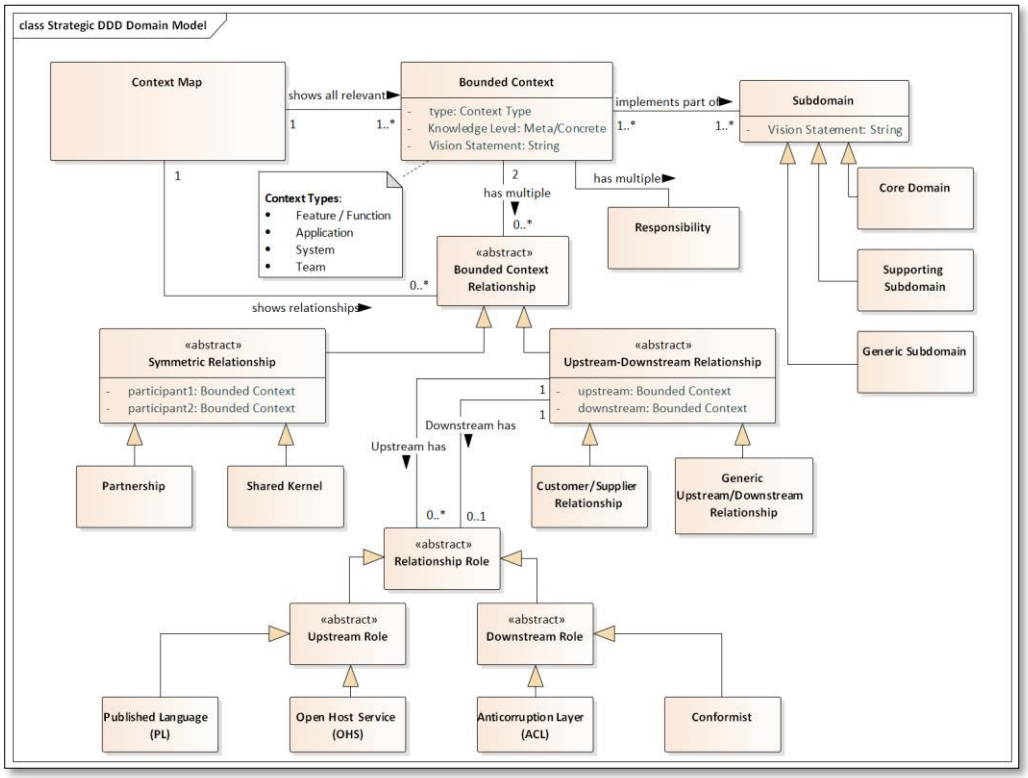


Figura 5 **Mapeador de contexto: metamodelo estratégico de DDD (diagrama de clases UML)**

Desarrollaron un conjunto de reglas semánticas además del metamodelo. Estas reglas definen las relaciones válidas entre los patrones, así como las combinaciones no permitidas en nuestro idioma. Además, presentan la herramienta de mapeador de contexto DS, el cual permitirá a los arquitectos de software modelar sistemas en base al metamodelo DDD y realizan un listado de las sintaxis del mapeo de contexto y sintaxis de la relación como ejemplo.

Finalmente, mencionan los escenarios de uso y aplicabilidad le la herramienta, donde especifican tres escenarios de uso principales y roles de usuario:

1) Los analistas de negocios pueden usar el lenguaje de modelado para analizar y comprender un dominio, y establecer un vocabulario común para él.

2) Los arquitectos pueden describir, comunicar y desarrollar diseños de sistemas y conexiones con otros sistemas (por ejemplo, integración de aplicaciones empresariales).

3) Los adoptantes de arquitecturas de microservicios pueden modelar descomposiciones de sistemas y servicios individuales.

Tiene la visión de utilizar el lenguaje como una herramienta para evolucionar y mejorar la arquitectura con refactorizaciones y transformaciones de modelos de manera iterativa, además, el lenguaje se puede utilizar como entrada para generadores que producen otras representaciones de los modelos o como entrada para otras herramientas.

Como conclusión, los autores mencionaron las contribuciones de investigación y propusieron: a) un metamodelo y reglas semánticas que apuntan a una especificación concisa de cómo se pueden combinar los patrones de DDD. b) un DSL y herramientas de apoyo para modelar contextos delimitados y sus relaciones, así como patrones tácticos de DDD como Agregados. Y a futuro, plantean mejorar la herramienta para que los arquitectos de software puedan desarrollar arquitecturas de sistemas con más transformaciones y refactorizaciones.

2.1.4. Diepenbrock et al. (2017) elaboraron un artículo de investigación titulada "***An Ontology-based Approach for Domain-driven Design of Microservice Architectures***" En el artículo abordan la falta de soporte de DDD para expresar semántica en modelos basados en DDD mediante ontologías y OWL. Por lo que plantean las siguientes contribuciones:

Primero, para permitir la expresión de relaciones semánticas en un diseño de MSA impulsado por dominios, definen un metamodelo para DDD que comprende conceptos para DM, su asignación a BC y la definición de SM entre BC.

Segundo, amplían el metamodelo con capacidades semánticas que permiten la expresión de la semántica de DM y SM y sus relaciones a través de BC.

La aplicación del metamodelo permite utilizar DDD para modelar BC, DM y SM y la base de microservicios relacionados con el negocio; enriquecer DM y SM con semántica para asegurar una comprensión común de los conceptos de dominio en el desarrollo distribuido de MSA.

Proyecto basado en dominios de arquitecturas de microservicios o diseño de microservicios basándose en el enfoque del DDD, detalla los conceptos de modelado de DDD relevantes para el diseño de MSA impulsado por dominios.

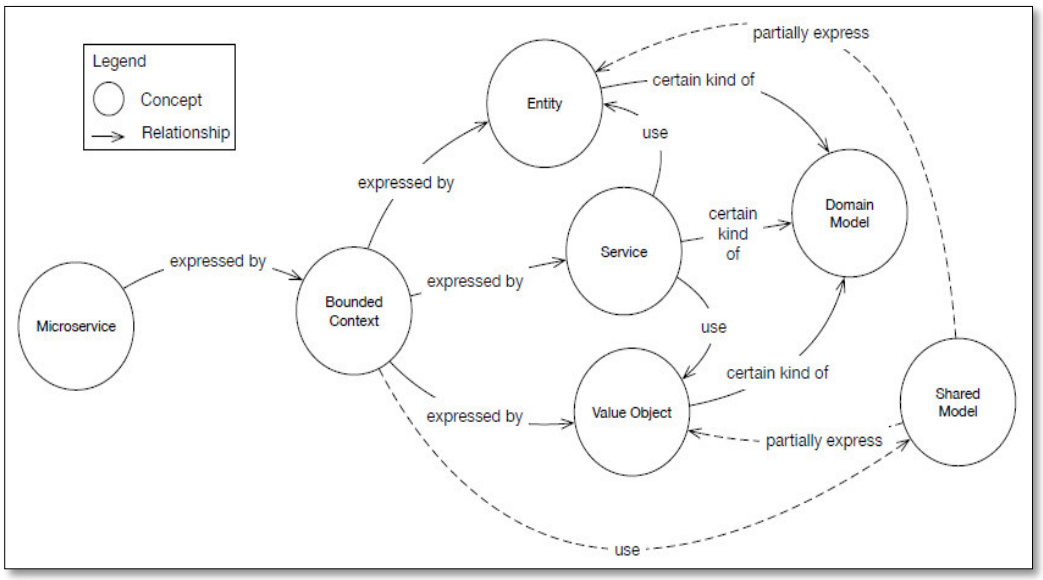


Figura 6 **Mapa de navegación para diseño controlado por dominio en MSA**

Seguido a lo anterior, presentan un ejemplo en ejecución que utiliza para la identificación de desafíos de modelos de dominio fragmentados

en un MSA. El ejemplo es un modelo basado en DDD de una aplicación llamada Research Management System (RMS), que usan en el grupo de investigación para administrar conferencias telefónicas y ponencias.

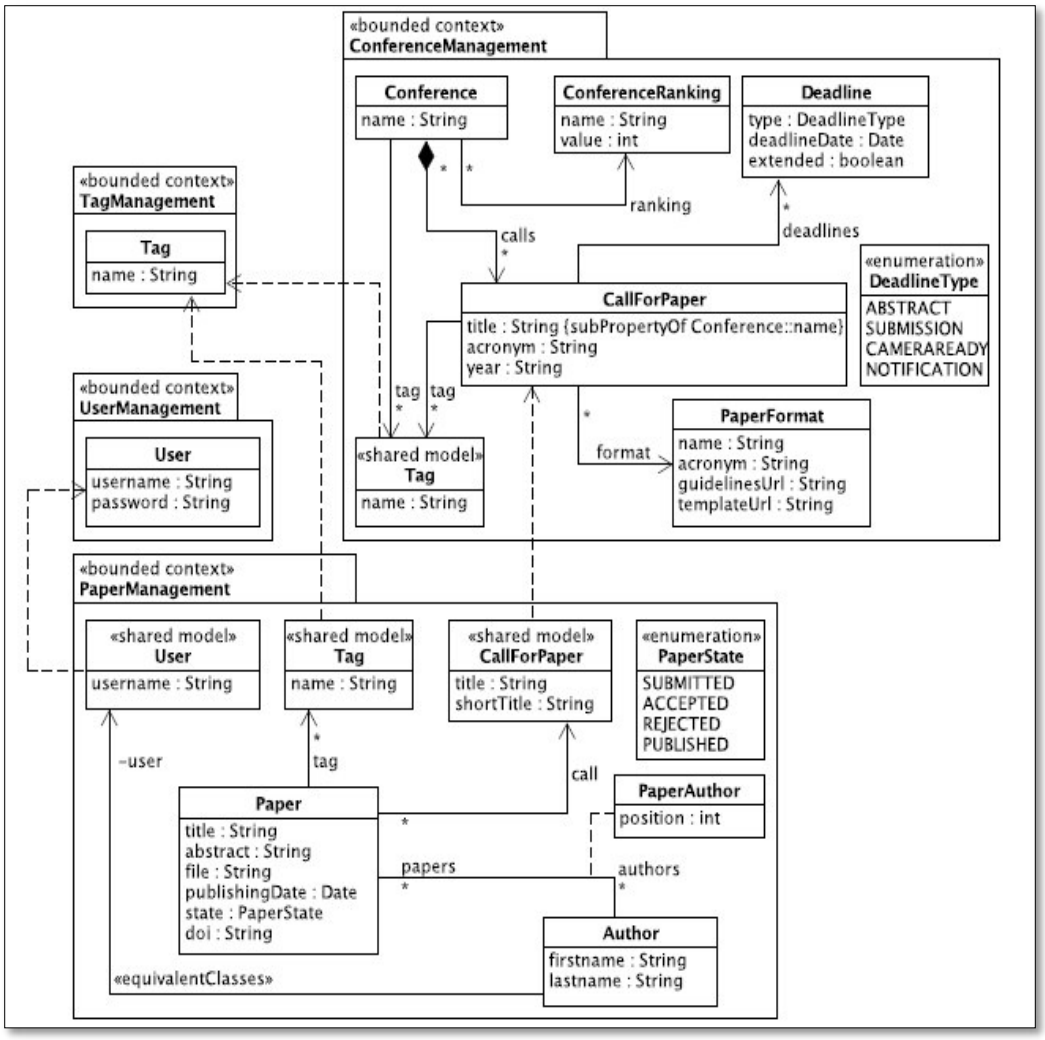


Figura 7 **Ejemplo de ejecución basado en DDD.**

Del ejemplo anterior, el diseño basado en dominios está compuesto por TagManagement, UserManagement, PaperManagement y ConferenceManagement. Para gestionar las Etiquetas correspondientes de Conferencia, CallForPaper y Paper pueden tener varias Etiquetas asignadas. La etiqueta DM es parte de TagManagement BC. Un Paper de BC PaperManagement se asigna a

un CallForPaper de ConferenceManagement BC, también el documento puede tener varios autores que están representados por Author en PaperManagement. La gestión de un Usuario se liquida en el BC UserManagement para fines de autenticación.

Porque BC PaperManagement y BC ConferenceManagement tienen que interactuar con otros Microservicios, estos tienen elementos de Modelo Compartido definidos que se expresan como clases con el estereotipo <<shared model>>. En PaperManagement el SM usuario se define en relación con el usuario DM de UserManagement BC. Esto permite que la relación de las instancias de Paper a través del Autor con el Usuario. También la etiqueta SM en BC se define PaperManagement que representa la etiqueta DM del TagManagement BC. La relación de DM Paper con CallForPaper se realiza mediante SM CallForPaper dentro de PaperManagement BC. La realización del Usuario SM, así como CallForPaper dentro de BC PaperManagement son diferentes a los respectivos DM con los que se relacionan. También BC ConferenceManagement define una etiqueta SM que se relaciona con la etiqueta DM del BC Gestión de etiquetas. La relación entre SM y el DM correspondiente son visualizado por la línea discontinua y la flecha apunta al DM respectivo.

Seguidamente, mencionan los desafíos de los modelos de dominio fragmentados en MSA como Desafío semántico: Atributos del modelo de dominio derivado, comprensión diferente de los conceptos de dominio compartido y conceptos de dominio distribuidos semánticamente equivalentes.

Los autores presentan su metamodelo para modelar un MSA siguiendo el enfoque DDD y enriquecer con conocimiento semántico:

Integración de ontologías en contextos acotados.

A. Un metamodelo para el diseño basado en dominios de arquitecturas de microservicios

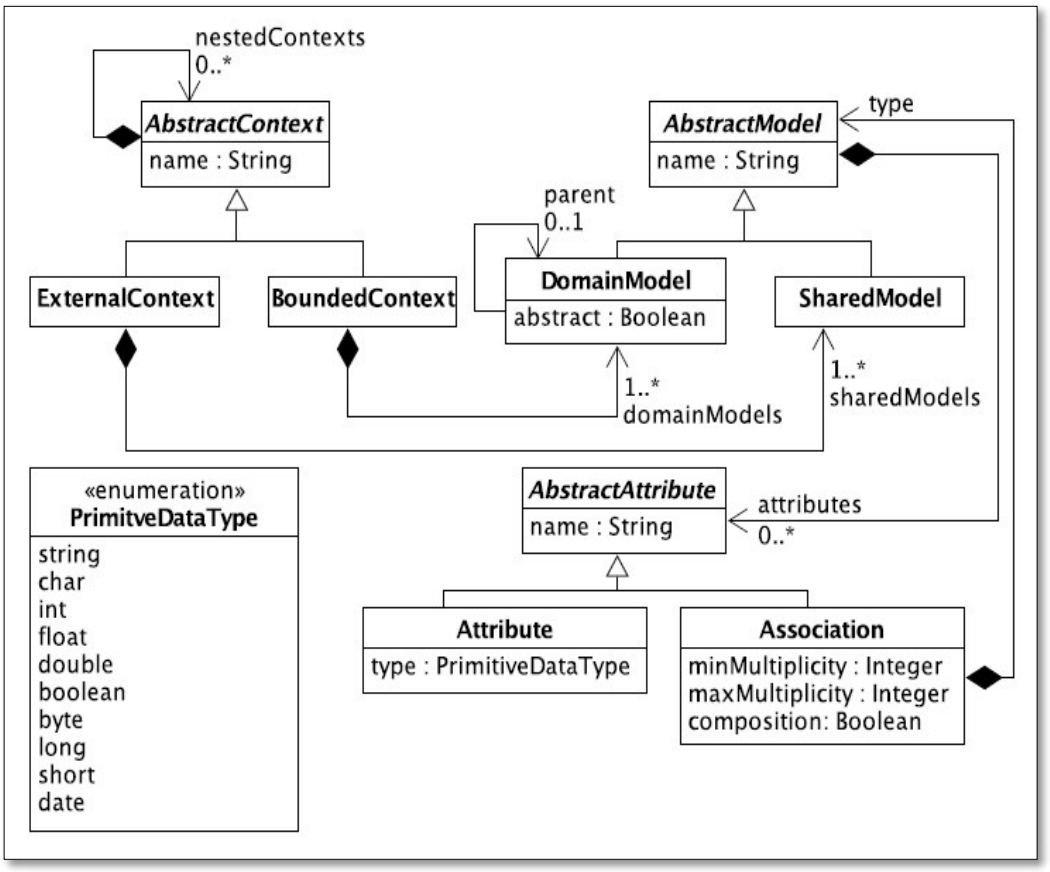


Figura 8 **Metamodelo para modelado basado en DDD.**

El metamodelo que presentan en la imagen anterior está diseñado para modelar un MSA siguiendo los principios de DDD, pero carece de soporte para expresar conocimiento semántico. Para enriquecer el metamodelo con capacidades semánticas distinguen entre características semántica de conceptos relacionados con modelos, es decir, DomainModel, etc., y conceptos relacionados con atributos, es decir, atributos, etc. modelos y atributos. Las capacidades semánticas para modelos relacionados los conceptos son: expresión de equivalencia de modelos distintos, desarticulación de distintos modelos y herencia de modelos. Para las capacidades relacionadas con atributos se componen de: herencia de atributos y equivalencia entre atributos distintos.

B. Enriqueciendo el metamodelo con capacidades semánticas

Para expresar el conocimiento semántico en el metamodelo, ampliaron con un nuevo concepto abstracto llamado DomainConcept como se muestra a continuación.

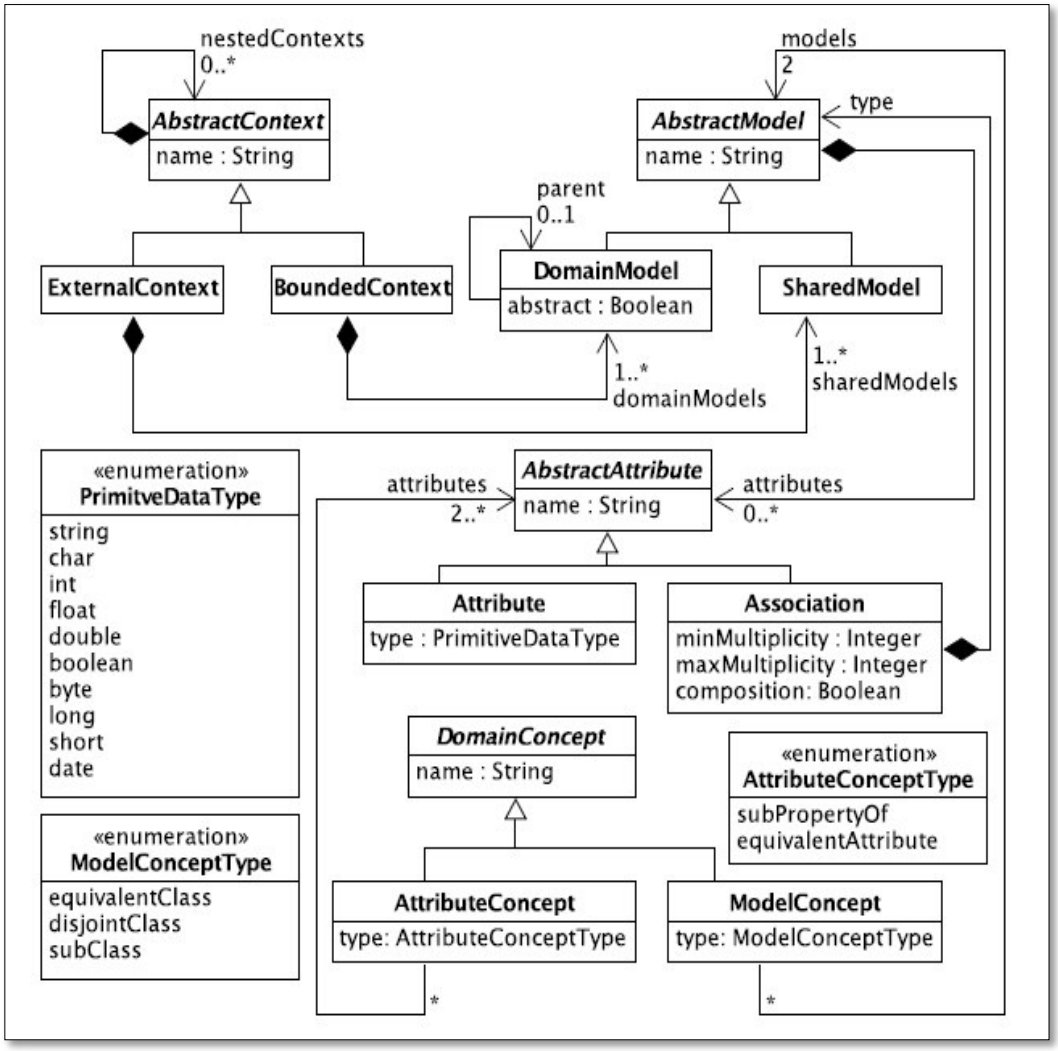


Figura 9 **Metamodelo enriquecido con capacidades semánticas a través de DomainConcept**

Las instancias concretas de DomainConcept son ModelConcept y AttributeConcept que diferencian entre conceptos semánticos de modelos y atributos para abordar las capacidades semánticas relacionadas con el modelo y con los atributos. Un DomainConcept se expresa mediante un nombre. ModelConcept tiene una relación con ModelConceptType y AttributeConcept se relacionan con un

AttributeConceptType. Con estas dos enumeraciones de ConceptType se pueden expresar la característica semántica. El ModelConcept se relaciona exactamente con dos instancias de AbstractModel con la relación de modelos. Para la expresión semántica de atributos se utiliza el AttributeConcept, que se relaciona al menos con dos instancias de AbstractAttribute. Usando el DomainConcept y las instancias concretas ModelConcept y AttributeConcept permiten la expresión del conocimiento semántico.

Concluyeron mencionando que presentaron un enfoque que aborda la falta de soporte para expresar conocimiento semántico al modelar MSA con diseño impulsado por dominios. Describieron el proceso de diseño de arquitecturas de microservicio aprovechando DDD. Seguidamente presentaron un metamodelo que admite el diseño de MSA controlado por dominios en combinación con la semántica y distinguieron entre el contexto externo, que se utiliza para describir modelos de dominio de microservicios externos, y el contexto acotado para diseñar el microservicio en sí. Asimismo, presentaron el DomainConcept concepto para expresar semántica en un MSA que está diseñado utilizando los principios de DDD y como representación formal de la semántica emplearon OWL.

Para trabajos futuros mencionan que tienen planeado implementar generadores que produzcan código para modelos de microservicio basados en el metamodelo analizado y evaluar el enfoque aplicándolo a varios proyectos del mundo real.

2.1.5. Vural & Koyuncu (2021) publicaron un artículo de investigación titulada "***Does Domain-Driven Design Lead to Finding the Optimal Modularity of a Microservice?"*** el cual se basa en dos ejemplos de diseño generados en estudios previos que aplicaron el diseño impulsado por dominios en la propuesta de microservicios. Ambos ejemplos se modifican para generar microservicios más y menos

granulares. Los valores de acoplamiento y cohesión de los ejemplos originales se comparan con los de los microservicios más y menos granulares.

El objetivo del estudio analizado fue investigar si el diseño impulsado por dominios (DDD) conduce a la granularidad óptima de un microservicio y comprender si los cálculos de cohesión y acoplamiento pueden ayudar a identificar el contexto acotado óptimo. Por lo tanto, esperan obtener respuestas empíricas a cuatro preguntas de investigación (CR) para aclarar los objetivos mencionados:

- ✓ RQ1: Si el tamaño de un microservicio es pequeño, ¿esto implica que está débilmente acoplado?
- ✓ RQ2: Si aumenta el tamaño de un microservicio, ¿aumentará la cohesión?
- ✓ RQ3: ¿El valor de acoplamiento mínimo y el valor de cohesión más grande definen la granularidad de microservicio óptima?
- ✓ RQ4: ¿DDD puede conducir a la granularidad óptima de microservicios?

Los autores mencionan que el microservicio óptimo se define como tener un equilibrio entre el acoplamiento mínimo y los valores máximos de cohesión. Por lo tanto, RQ3 intenta obtener la selección óptima entre las medidas de acoplamiento y cohesión aplicando el proceso de jerarquía analítica (AHP). El objetivo de RQ1 y RQ2 es determinar si el acoplamiento y la cohesión cambian sin problemas en función de la granularidad (tamaño). ¿Es posible concluir directamente que los módulos más pequeños siempre tienen menos acoplamiento? Dado que el acoplamiento y la cohesión son medidas en conflicto, ¿Podemos esperar siempre que un microservicio con granularidad media sea el mejor caso que proporcione el mejor equilibrio entre los valores de acoplamiento y cohesión? ¿O hay un punto óptimo que es difícil de detectar a menos que lo midamos con cuidado? RQ4 apunta a ver si

los ejemplos de DDD seleccionados conducen a un bajo acoplamiento y una alta cohesión.

Para la elaboración de este artículo los autores realizaron búsqueda de diferentes artículos relacionamos con el tema de granularidad, después de varias selecciones, realizaron comparaciones con los diferentes ejemplos que obtuvieron para medir el acoplamiento y la cohesión, los cuales los analizan en diagramas obtenidos de los ejemplos. También realizaron 2 casos de estudio para ilustrar los desafíos del diseño de microservicos basados en DDD.

A. Primer caso:

Describieron los estudios de otros autores como ejemplo de microservicios basado en DDD. Asimismo, tomaron como el ejemplo original de microservicio basado en DDD, el que se representa en la siguiente figura:

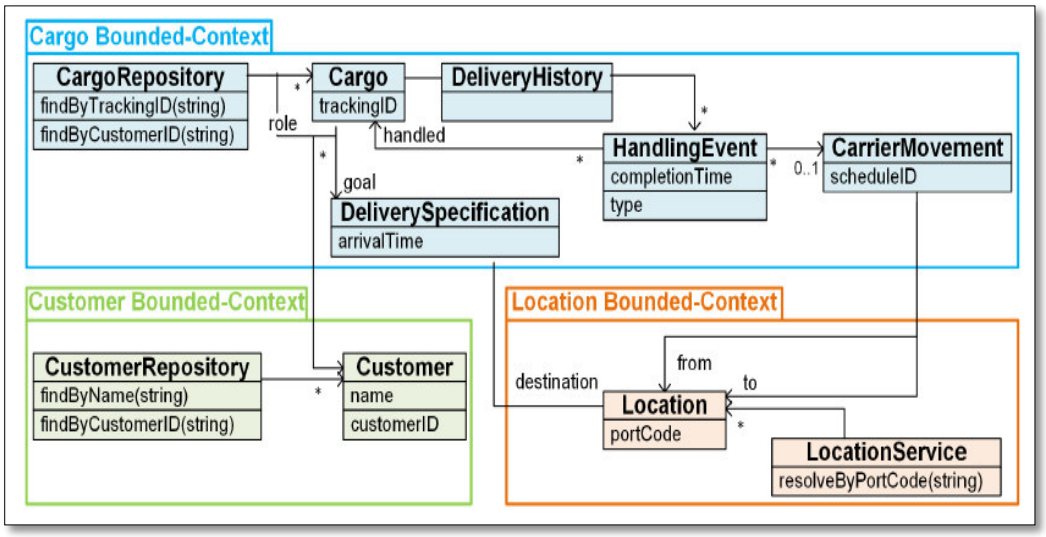


Figura 10 **El diagrama de clases UML para el ejemplo de carga original basado en DDD.**

Fuente. Rademacher et al. (2018)

El diagrama UML anterior muestra tres diferentes contextos acotados ('Carga', 'Cliente' y 'Ubicación') los cuales pueden ser interpretados como microservicios (Rademacher et al., 2018). El ejemplo original se

dividió en construcciones más pequeñas, el que se representa en la siguiente figura:

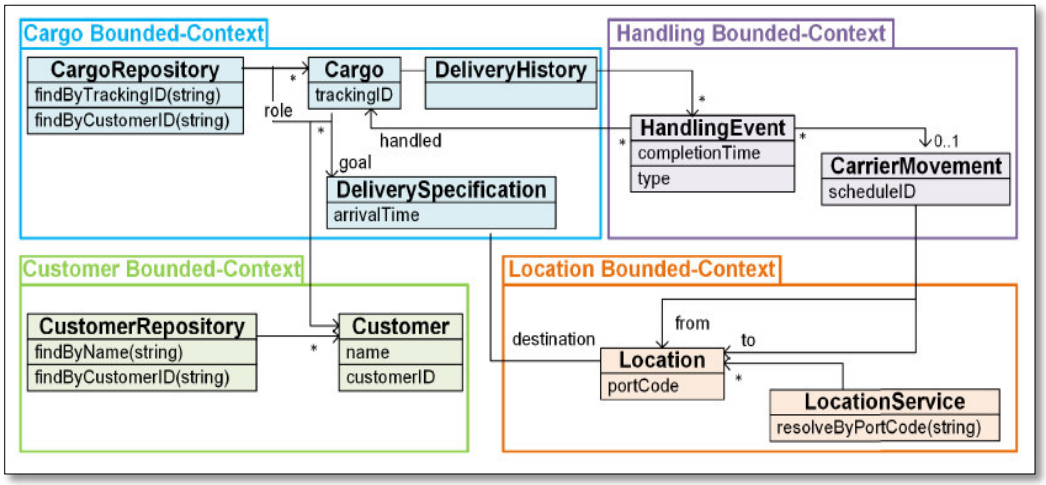


Figura 11 **El diagrama de clases UML para los MGM del ejemplo de carga.**

Fuente**.** Rademacher et al. (2018)

La muestra resultante se denomina ejemplo MGM. Con esta presentación, el propósito de los autores fue examinar el resultado de la modificación de diseño sobre los valores de acoplamiento y cohesión.

El ejemplo original se combina en construcciones más grandes para obtener otra alternativa de diseño, lo que se representa en la siguiente figura:

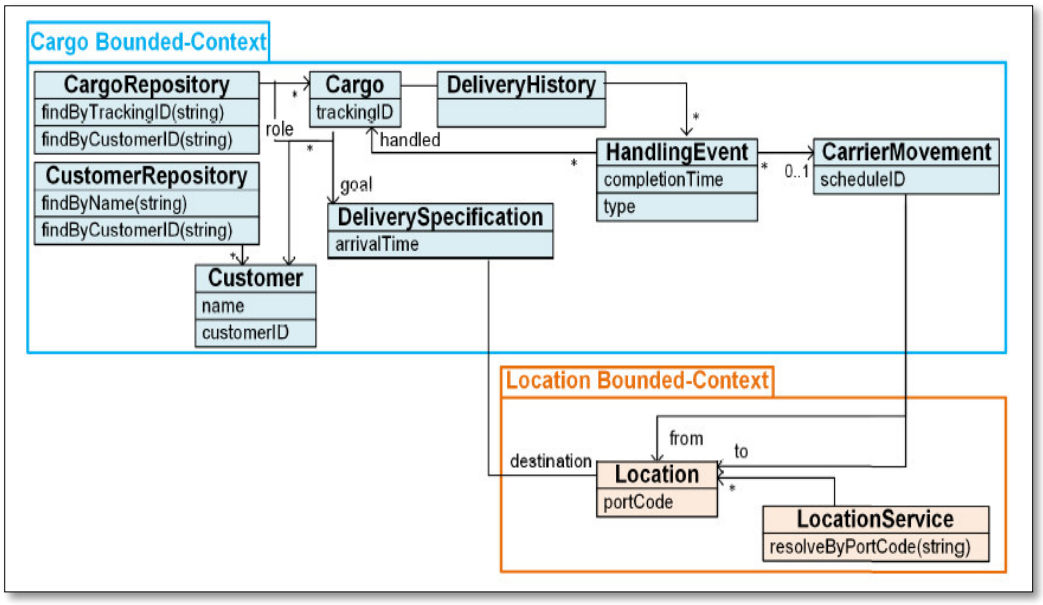


Figura 12 **El diagrama de clases UML para los LGM del ejemplo de carga**

Fuente. Rademacher et al. (2018)

La muestra resultante se denomina ejemplo de microservicio menos granular (LGM). Si existe una relación entre la granularidad y el acoplamiento, puede haber una relación entre la granularidad y la cohesión. El objetivo de este diseño es observar los valores de cohesión y acoplamiento cuando se reduce la granularidad de los servicios.

En los resultados que obtuvieron de los ejemplos, los valores de acoplamiento aferentes y eferentes son los más pequeños para la muestra original. El valor de acoplamiento es el peor (acoplamiento más alto) para el LGM. Donde, los valores de cohesión aumentan medidas que generan el tamaño del contexto delimitado. El LGM tiene la mejor cohesión relacional, seguido de cerca por el ejemplo original.

B. Segundo caso:

Para este caso los autores seleccionan como ejemplo de microservicio una aplicación de comercio electrónico para compras en línea, el ejemplo se ejecuta en Docker y ha sido construido en base a conceptos

del DDD, además, cuenta con 7 microservicios, de los 7 servicios, 4 de ellos fueron los más importantes para los autores del caso de estudio.

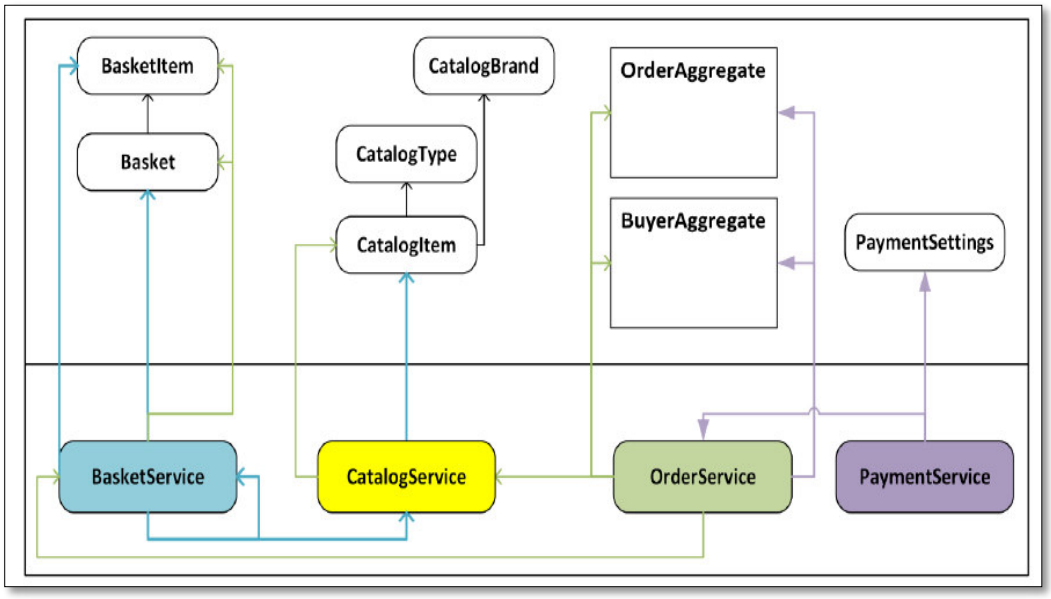


Figura 13 **El diagrama de clases UML para los MGM del ejemplo de eShopOnContainers.**

Para el caso de estudio los autores modifican el ejemplo original para obtener dos nuevos ejemplos, ejemplos LGM y MGM como se muestra a continuación:

Cuadro 2 **Servicios para las aplicaciones MGM, Original y LGM.**

Application name	Names of the services	Total # of services
MGM	Basket, Catalog, Identity, Location, Marketing, Order, Buyer, Payment	8
eShopOriginal	Basket, Catalog, Identity, Location, Marketing, Order, Payment	7
LGM	Basket, Catalog, Identity, Location, Marketing, Order	6

Realizaron varias modificaciones para obtener un ejemplo más granular dividiendo la aplicación original en 2 microservicos más, y para el ejemplo menos granular combinan 2 microservicios reduciendo los servicios. Empleando una herramienta para el análisis de código estático el cual, ayudado a reducir el error humano, lograron medir las 3 métricas para las 3 aplicaciones de microservicio de ejemplo.

Para el análisis que realizaron, primero exploraron el estudio de caso de carga y luego el estudio de caso de eShopOnContainers. El objetivo principal fue identificar el resultado óptimo basado en las medidas de acoplamiento y cohesión.

En la investigación, los autores recopilaron tres mediciones, con esas mediciones, los procedimientos esperados de una definición de buen diseño son:

- ✓ Acoplamiento aferente: Se espera que los valores de acoplamiento aferente sean bajos.
- ✓ Acoplamiento eferente: Se espera que los valores de acoplamiento eferente sean bajos.
- ✓ Cohesión relacional: Se espera que la cohesión sea alta.

Concluyeron, mencionando que el objetivo fue investigar un medio apropiado para identificar el contexto delimitado óptimo y la granularidad óptima de un microservicio. Para lograr esto, seleccionaron dos aplicaciones de muestra impulsadas por DDD como casos originales. Para cada ejemplo, la aplicación original se dividió en MGM y se combinó en LGM para construir aplicaciones adicionales. Luego, midieron los valores promedio de acoplamiento aferente, acoplamiento eferente y cohesión relacional a nivel de clase. Los resultados se compararon aplicando el AHP para responder a cuatro preguntas de investigación.

Las posteriores conclusiones surgieron en base a los resultados empíricos:

- ✓ El uso de DDD demostró ser un buen medio para identificar la granularidad óptima de microservicios.
- ✓ El cálculo de los valores de acoplamiento y cohesión ayudó a identificar el contexto acotado óptimo.

En los ejemplos analizados, observaron que el diseño basado en dominios aportó un buen resultado final para encontrar microservicios modulares.

2.1.6. López & Maya (2017) en una publicación de un artículo titulada *"**Arquitectura de Software basada en Microservicios para Desarrollo de Aplicaciones Web**"* realizan un análisis de los diferentes problemas que se presentan en la fase de construcción de software presentada por una institución dedicada a tecnologías de información y comunicación**,** donde se utiliza una arquitectura de software tradicional que es más conocido como monolítico el cual consiste en aplicaciones que empaquetan todas las funcionalidades en uno solo archivo de ejecución lo que ha traído como dificultad para el mantenimiento, escalabilidad, entregas en tiempo récord entre otras atributos de calidad.

Su objetivo fue identificar nuevas tecnologías, métodos y arquitecturas utilizadas por la institución dedicada a tecnologías de información y comunicación en estudio para desarrollar proyectos web, así como tecnologías existentes para implementar microservicios que resuelvan las dificultades encontradas. Utilizaron como base de su estudio una investigación de tipo descriptiva cualitativa y un enfoque de diseño documental, la cual se aplica a los funcionarios de la región de desarrollo de la institución dedicada a tecnologías de información y comunicación, en su revisión de estado del arte se basaron en conocer los métodos de implementación de los microservicios, sus requerimientos y las necesidades relacionadas con la construcción de aplicaciones web.

La propuesta para aplicar una arquitectura de software que describen, se presenta en la siguiente figura:

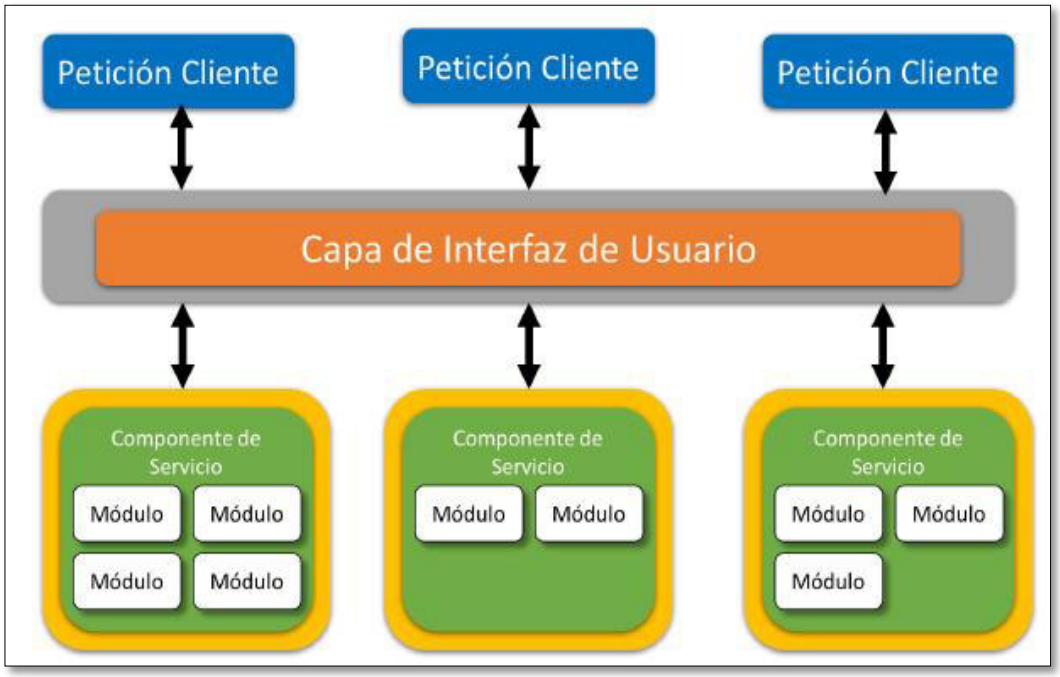


Figura 14 **Patrón básico de la arquitectura de Microservicios**
Fuente**.** Richards (2015)

Concluyeron mencionando que el actual problema en el desarrollo de software con la arquitectura empleada dificulta el mantenimiento, escalabilidad del sistema y adición de nuevas funcionalidades por ser una aplicación monolítica, por ello se basan en la evaluación de aplicar el uso de nuevas tecnologías como la arquitectura de microservicios con el cual pretender mejorar como institución.

2.1.7. Carvalho & Vendramel (2019) elaboraron un artículo titulado *"**Padrões de migração de sistemas legados para arquitetura baseada em microsserviços**"* donde hacen referencia que el microservicio es el estilo de arquitectura de software que ha ido consolidándose en proyectos de desarrollo de aplicaciones, compuesto por un conjunto de pequeños servicios, además mencionan que existe una gran demanda para migrar los sistemas heredados construidos en arquitectura

monolítica a este nuevo estilo con el propósito de aprovechar las tecnologías de los sistemas distribuidos. Este artículo tuvo como objetivo enumerar los patrones que se pueden usar en proyectos que tienen como objetivo migrar una aplicación desarrollada utilizando un enfoque monolítico a un nuevo camino de microservicios.

Microservicios es el modelo de arquitectura de software centrado en la nube de la arquitectura donde se desarrolla un sistema software como un conjunto de pequeños servicios (Kratzke & Quint, 2017).

Cada uno de estos servicios es capaz de instalarse de forma independiente y en distintas plataformas, ejecutando su propio proceso y comunicándose entre sí a través de mecanismos ligeros como las API RESTful. En este contexto, cada servicio realiza una función de negocio y cada de ellos se pueden implementar utilizando distintas lenguas de programación y almacenamiento de datos (Lewis & Fowler, 2014).

Los autores tuvieron como objetivo de la investigación identificar patrones de migración en la literatura que traen beneficios y facilidades en proyectos que tienen como objetivo migrar aplicaciones desarrollada en arquitectura monolítica a una arquitectura de microservicio, para lograr alcanzar el objetivo del estudio realizaron investigación bibliográfica para obtener bases teóricas, además realizaron encuesta a profesionales del mercado de tecnologías de información con el sentido de evaluar la aplicabilidad y relevancia de las normas en la práctica diaria.

Mencionaron que los Patrones de migración, aunque la arquitectura de microservicio aporta muchos beneficios, también aporta complejidades al sistema, que necesita adoptar nuevos componentes. La dificultad para descomponer grandes servicios en servicios más pequeños, la necesidad de monitorear y administrar los servicios son algunos de los factores que hacen que la migración a microservicios sea una tarea desafiante.

Tomando en cuenta el aumento en la cantidad de servicios que trae la arquitectura de microservicios y la complejidad en la gestión del sistema en un entorno de producción, se necesita un mecanismo que ayude a mantener este entorno siempre preparado para la entrega continua, es decir, la actualización e instalación de nuevas funcionalidades.

Algunos problemas son comunes en aplicaciones monolíticas; es decir, la complejidad de un código que se vuelve complicado y difícil de entender, las distintas partes del sistema que tienen requisitos innecesarios no funcionales y la necesidad de una instalación completa de la aplicación incluso cuando el cambio se refiere solo a una pequeña parte son solo algunos ejemplos que se puede resolver mediante microservicios. Para ello, existe la opción de rediseñar la solución monolítica para un conjunto de servicios basados en la metodología de Domain-Driven Design (DDD). Cabe mencionar que se recomienda comenzar con un número reducido de servicios, dos o tres, e ir aumentando en función del aumento del conocimiento empresarial (Evans, 2015).

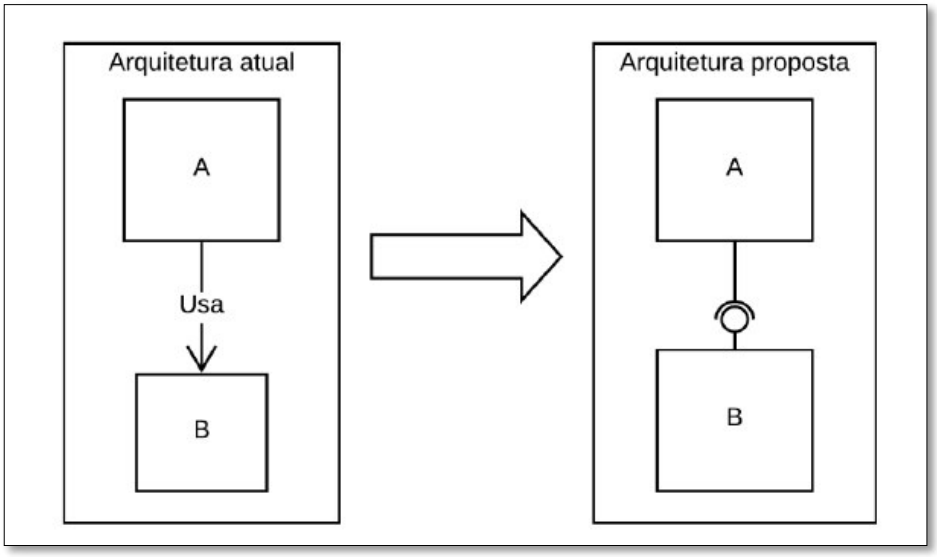


Figura 15 **Reemplazo de la dependencia del código con llamadas de servicio**

Fuente**.** Adaptado de Balalaie et al. (2018)

El estándar Strangler Application tiene como objetivo crear una capa o aplicación que intercepte las llamadas realizadas a la aplicación a migrar y redirija a nuevos microservicios, cuando ya están implementados (Lewis & Fowler, 2014).

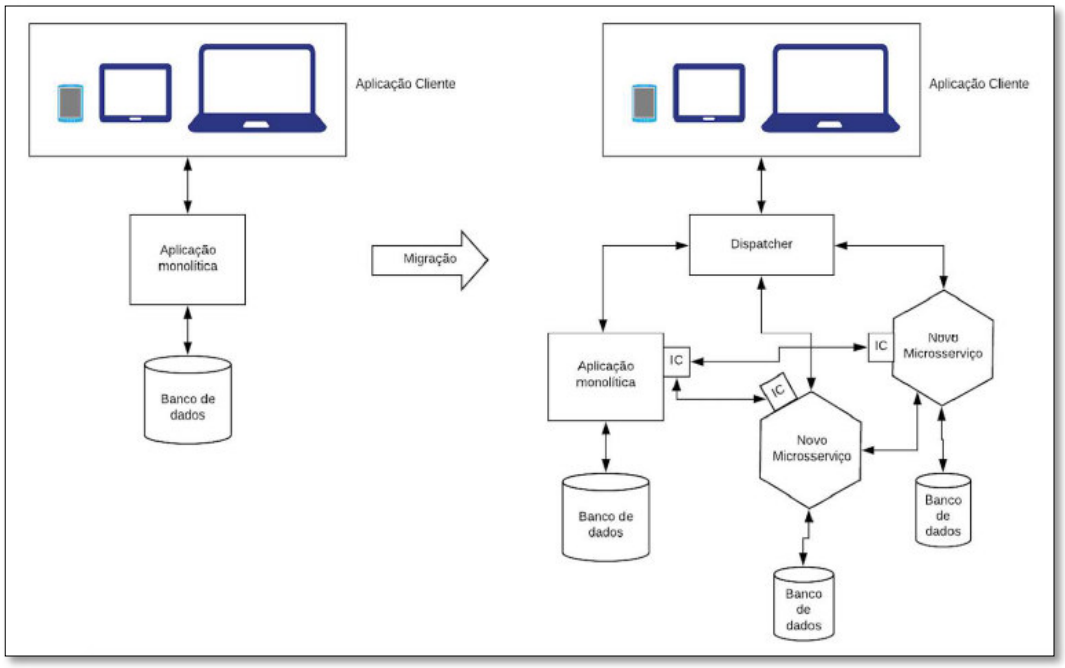


Figura 16 **Strangler Application**
Fuente. Adaptado por Santis et al. (2016)

Para completar el proceso de migración, los autores también elaboraron registros de servicios, elaboraron un balanceador de carga, crearon un servidor de configuración. Asimismo, mencionan que para la investigación se realizó una búsqueda web con 40 profesionales en el entorno de tecnologías de la Información.

Concluyeron, mencionando que, en base a los resultados presentados, se observa que los patrones migratorios enumerados por la investigación bibliográfica en este estudio son conocidos por la mayoría de los profesionales encuestados. Los estándares se consideran relevantes en proyectos de migración de aplicaciones heredada apoyada en arquitecturas monolítica para una nueva aplicación desarrollada manejando la arquitectura de microservicios. Cabe mencionar el nivel de experiencia de los profesionales involucrados en la investigación, con un 52,5% de los profesionales poseen más de diez

años de experiencia y solo un 12,5% tienen menos de dos años de experiencia en el ambiente de TI, lo que indica que las respuestas aportan un nivel de confianza consistente, dado el nivel de experiencia de los participantes. También es de destacar que el nivel de adherencia a la investigación por parte de los profesionales que trabajan directamente en desarrollo de software y bosquejo de arquitectura fue alto, con el 82,5% del total de la muestra, trayendo beneficios a la investigación, ya que estos profesionales trabajan directamente vinculados a la temática de investigación tratado en este artículo.

2.1.8. Barrios (2018) realiza un análisis del artículo titulado "***Arquitectura de microservicios***", hace referencia a la arquitectura de microservicios, mencionando que es un nuevo termino que estuvo examinado en un taller para arquitectos de software el año 2011 (Fowler, 2015). Esto, surge obteniendo una mayor popularidad para el año 2014, refiriéndose que es un conjunto de servicios desplegables e independientes. Una de las referencias más importantes se centra en el estilo monolítico que si bien es cierto su desarrollo trae resultados exitosos, pero frente a un cambio constante en la tecnología modificar toda la aplicación lo que genera es más complicaciones en la misma. Frente a la misma problemática los microservicios permiten más ventajas en este tipo de arquitectura, construida a manera iterativa e incremental un diseño sencillo y enfocado a resolver el problema.

Una de las grandes características entre el enfoque de microservicios y monolítico, el primero busca dividir una aplicación en partes separando desde los equipos, servidores y base de datos los cuales genera que se deba separar el trabajo de otros equipos; así mismo el código fuente crecerá frecuentemente por adicionar una nueva funcionalidad la que conllevara que se realice nuevos cambios; mientras que el otro enfoque busca una organización interdisciplinaria pues mezcla la interfaz de usuarios, base de datos y lógica de negocio; es decir el equipo deber tener diferentes habilidades, a diferencia del

segundo punto el enfoque de microservicios nos propone en que se deba aislar la especialidad de la lógica lo que permite no hacer crecer el componente a diferencia del otro enfoque que tiene más habilidades.

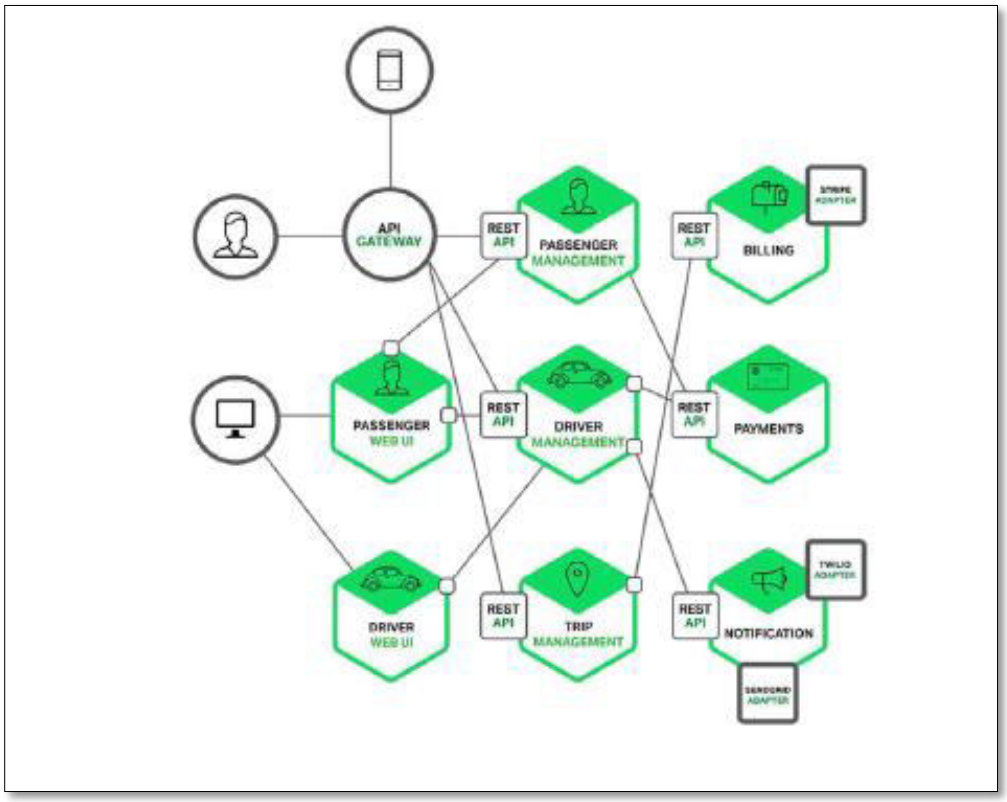


Figura 17 **Arquitectura de microservicios**
Fuente. Richardson (2015)

Los microservicios son un sistema desarrollo software lo cual influye de manera positiva en aspectos de tiempo, rendimiento y estabilidad para el desarrollo de los proyectos ya que se funciona a través de pequeños servicios de manera autónoma e independiente lo cual nos facilita el desarrollo de los procesos.

2.2. Bases Teóricas

2.2.1. Software

Un software es importante porque facilita casi a todas las actividades de nuestras vidas, y está presente en nuestro comercio, cultura, etc. (Pressman, 2010). Por otro lado, el software es como una expresión simplificada de la construcción del sistema de software, puede mostrarse como un movimiento hacia una solución técnica a un problema específico o una forma de resolverlo, este paso a menudo se asocia con restricciones de tiempo, costo y calidad (Cervantes et al., 2016).

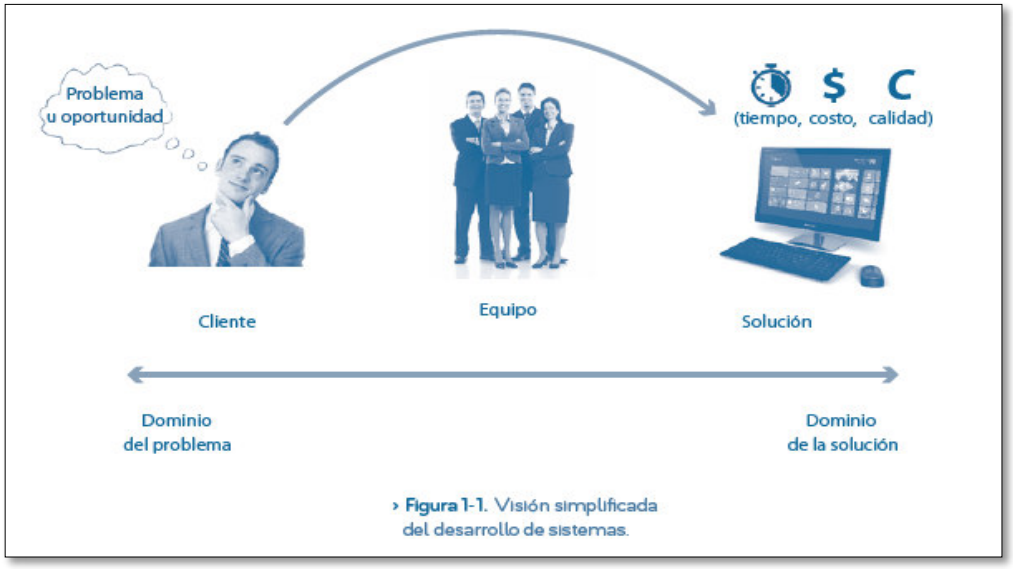


Figura 18 **Visión simplificada del desarrollo de sistemas**
Fuente. Obtenido de (Cervantes et al., 2016).

2.2.2. Ingeniería de Software

Según Sommerville (2011) la ingeniería de software viene a ser la especialidad de ingeniería que se ocupa de todas las formas y aspectos en la construcción de un software, partiendo de las fases iniciales de definición de un sistema, culminando en el mantenimiento después de su implementación. Por tanto, es parte de la ingeniería que se enfoca en la construcción del software desde los requerimientos del usuario hasta la puesta en producción del producto software, con el objetivo de mostrar satisfacción a las necesidades de los usuarios que los usan para agilizar

o automatizar procesos de negocios gestionando la información que el software recibe, procesa, transforma y entrega.

2.2.3. Arquitectura de Software

Una arquitectura de software se considera un puente entre la fase de diseño y la planificación de requisitos (Sommerville, 2011). Por otro lado, Fowler (2019) menciona cuando las personas en la industria del software hablan de "arquitectura", se refieren a una noción definida de los aspectos más importantes del diseño interno de un sistema de software. Una buena arquitectura es importante, de lo contrario se volverá más lenta y costosa agregar nuevas capacidades en el futuro.

Hay muchas definiciones de arquitectura de software, pero cuando analizas las definiciones existentes, puedes ver que son de diferentes autores, pero todos están de acuerdo en que la arquitectura de software se enfoca en la distribución del sistema, sus módulos y estructura de los enlaces entre ellos, por lo que se pueden agregar algunos puntos para mencionar que la arquitectura de software es el esquema altamente avanzado de la organización de un sistema, que radica en una agrupación de patrones y abstracciones que brindan un marco transparente para la implementación de los sistemas (Blancarte, 2020).

2.2.4. Diseño Arquitectónico

Según Sommerville (2011) el diseño arquitectónico es como la parte fundamental de un sistema de software donde se define como deberá estructurarse el sistema a desarrollar y como se relacionarán los componentes que los conforman. Es importante entender la definición de la arquitectura para tener el conocimiento necesario, el cual facilitara el diseño del modelo a proponer como solución.

2.2.5. Patrones de Diseño

Según Gamma et al. (2003) el patrón de diseño denomina, agrega y reconoce formas clave de una distribución de esquema habitual y los utiliza para formar un bosquejo orientado a objetos que se pueden volver

a utilizar; el modelo de diseño distingue clases y casos colaboradores, sus roles, colaboraciones, y el repartimiento de compromisos, cada uno de los patrones de diseño se enfoca en un determinado problema y refiere en qué momento debe aplicarse y si posee la razón de efectuar, asumiendo otras limitaciones de diseño e implicaciones de uso, ventajas y desventajas.

2.2.6. Modelo

Pons et al. (2010) el modelo es la representación del sistema que se va a desarrollar facilitando la comprensión del mismo para realizar un diseño final del sistema, esto ayudara a tener conocimiento de cómo está conformado cada parte del sistema.

2.2.7. EventStorming

Es un formato de taller centrado en eventos para comprender un dominio complejo e implica reunir a expertos en el dominio en una habitación, muchas notas adhesivas y una superficie muy grande (una pizarra o un rollo de papel) para pegar las notas; por ello, el resultado de la tormenta de eventos es un modelo de dominio centrado en eventos que consta de agregados y eventos (Richardson, 2018).

EventStorming consta de tres pasos principales:

1. Lluvia de ideas sobre eventos: solicite a los expertos del dominio que hagan una lluvia de ideas sobre los eventos del dominio; los eventos de dominio se representan mediante notas adhesivas de color naranja dispuestas en una línea de tiempo aproximada en la superficie de modelado (Richardson, 2018).

2. Identifique los desencadenantes de eventos: solicite a los expertos del dominio que identifiquen el desencadenante de cada evento, que es uno de los siguientes: Acciones del usuario, representadas como un comando usando una nota adhesiva azul, sistema externo, representado por una nota adhesiva morada, otro evento de dominio y paso del tiempo (Richardson, 2018).

3. Identificar agregados: solicite a los expertos del dominio que identifique el agregado que consume cada comando y emite el evento correspondiente. Los agregados están representados por notas adhesivas amarillas (Richardson, 2018).

2.2.8. Domain Driven Design – DDD

El diseño guiado por el dominio (DDD) es el enfoque para la construcción de software complejos, el cual se basa en el dominio central del negocio, explora el modelo en ayuda de los expertos del dominio y quienes desarrollan softwares, además usa un idioma ubicuo dentro de un contexto delimitado para entender las definiciones, DDD se divide en diseño estratégico y diseño táctico, y ambos deben combinarse para alcanzar el éxito en la aportación de un sistema (Evans, 2015).

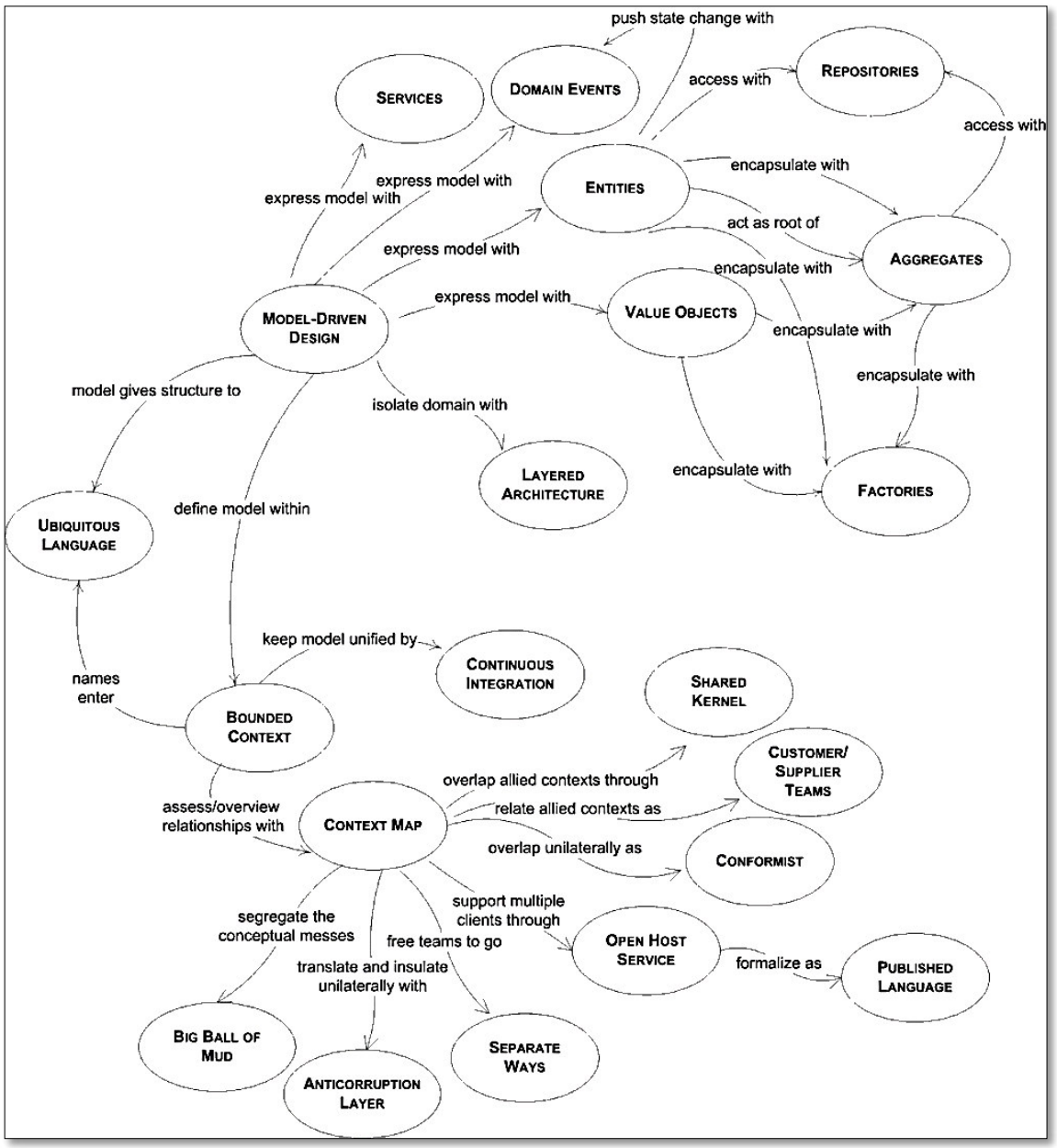


Figura 19 **Descripción general del lenguaje de patrones**

Fuente. Obtenido de (Evans, 2015).

Por lo tanto, el propósito de la arquitectura apoyada en Domain Driven Design (DDD) es alcanzar un piloto orientado a objetos que muestre el juicio del dominio entregado, por ende, es totalmente autónomo ante cualquier concepto de declaración, así sea con las partes de infraestructura como de interfaz de usuario, etc. (Torre et al., 2010).



Figura 20 **Esquema de comunicación de la Arquitectura**
Fuente. Obtenido de (Torre et al., 2010).

2.2.9. Diseño Estratégico

2.2.9.1. Domino

El dominio se intenta tan a menudo y tan raramente se logra que su negación merece una discusión por derecho propio (Evans, 2015). Por ello, un dominio es una esfera de conocimiento que actividad e influye; es decir, es el área comercial de enfoque para el desarrollador de aplicaciones, por tanto, la premisa subyacente de DDD es que la complejidad está en el dominio, no en la tecnología. Los marcos y las herramientas han eliminado la complejidad (Betts, 2018). Asimismo, los términos dominio y subdominio tienden a usarse indistintamente con bastante frecuencia, esto puede ser confuso para el espectador casual; dado que, los subdominios tienden a ser bastante complejos y jerárquicos, un subdominio puede ser un dominio por derecho propio (Chandrasekaran & Krishnan, 2022).

2.2.9.2. Subdominio

En esencia, del DDD proporciona medios para abordar la complejidad y los ingenieros hacen esto dividiendo los problemas complejos en otros más manejables llamados subdominios; lo cual, facilita una mejor comprensión y hace que sea más fácil llegar a una solución (Chandrasekaran & Krishnan, 2022).

2.2.9.3. Modelo de Dominio

Es una parte integral del dominio diseño, ya que la mayoría de los objetos de cualquier forma no funcionarán con total independencia, cuando se piensa en asociaciones entre objetos, la asociación en sí misma puede contener conocimiento de dominio que queremos tener incorporado en el mismo modelo (Chandrasekaran & Krishnan, 2022).

2.2.9.4. Bounded Context

Bounded context es una unidad organizativa destinada a sostener la coherencia de los modelos grandes; También puede representar la funcionalidad y comunicación del sistema para un procedimiento del exterior o un grupo de elementos diseñados para efectuar actividades de manera óptima. (Torre et al., 2010). También, Bounded context es un contexto que usa un modelo que limita explícitamente la organización del equipo, que se usa en ciertas partes de la aplicación y las manifestaciones físicas, como el código base en el esquema de bases de datos (Evans, 2015).

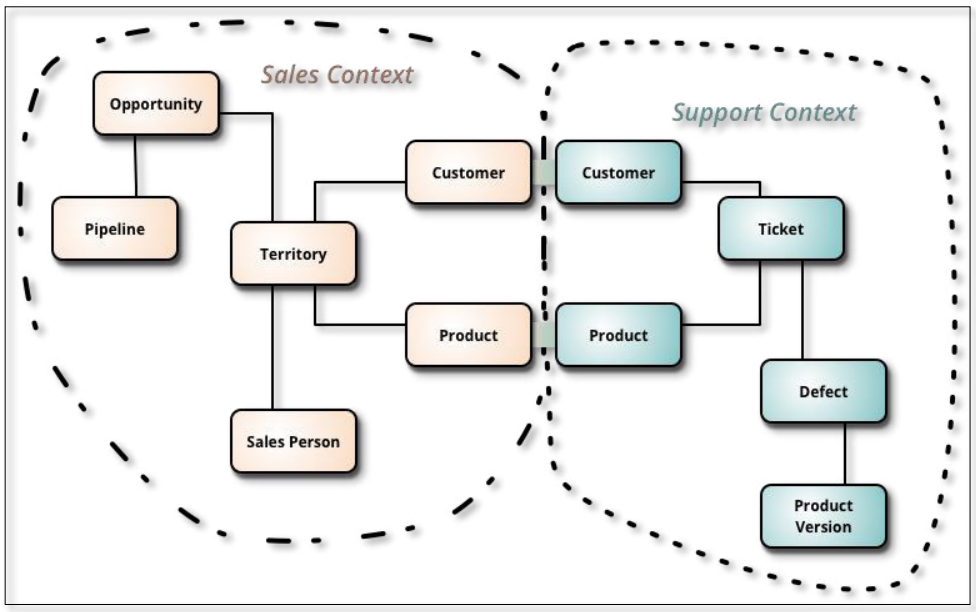


Figura 21 **Bounded Contexts**
Fuente. Obtenido de (Fowler, 2014).

2.2.9.5. Context Map

El contexto map describe los puntos de contacto entre los modelos, destacando la traducción explícita de cualquier comunicación, destacando cualquier intercambio, mecanismos de aislamiento y niveles de influencia; por tanto, mapea el terreno existente y retoma las transformaciones más tarde (Evans, 2015). Por otro lado, context map es un artefacto poderoso que puede traer nuevos equipos, los miembros se ponen al día rápidamente y brindan una advertencia temprana sobre posibles puntos críticos de problemas; por tanto, los mapas de contexto también pueden revelar problemas con la comunicación y los flujos de trabajo dentro de la empresa (Millett & Tune, 2015).

2.2.10. Diseño Táctico

El diseño táctico apunta a superar las deficiencias del lenguaje de programación, el paradigma o el entorno operativo (Eden, 2005). Por otro lado, el diseño táctico, es una actividad relacionada con los requisitos locales que gobiernan lo que hace una solución, como algoritmos, diseño patrones, lenguajes de programación, refactorización y bajo nivel implementación (Jaiswal, 2019). Por tanto, los elementos de diseño

táctico de DDD brindan un conjunto de técnicas para ayudar a lidiar y recuperarse de estas situaciones casi desesperadas de manera pragmática, sin tener que adoptar potencialmente un enfoque de big bang (Chandrasekaran & Krishnan, 2022).

2.2.10.1. Agregados

Un agregado es un límite de continuidad alrededor de una o más propiedades (Jaiswal, 2019). Asimismo, los agregados también se denominan raíces agregadas, es decir, el objeto que está en la raíz de la jerarquía de entidades; además, es un gráfico de objetos cuyo estado cambia de manera constante y atómica (Chandrasekaran & Krishnan, 2022).

2.2.10.2. Entidades

Una entidad es un objeto con una identidad única y encapsula los datos en el comportamiento de sus atributos; es decir, puede ser conveniente ver las entidades como una colección de otras entidades y objetos de valor que deben agruparse (Chandrasekaran & Krishnan, 2022).

2.2.10.3. Objetos de Valor

Un objeto de valor no tiene identidad, se define solo por los valores de las propiedades inmutables; para actualizar un objeto de valor, es fundamental crear una nueva instancia para reemplazar la anterior (Jaiswal, 2019). Por otro lado, los objetos de valor pueden tener métodos que encapsulan la lógica del dominio, pero no deben afectar el estado del objeto (Chandrasekaran & Krishnan, 2022).

2.2.11. Microservicios

La arquitectura de Microservicios propone dividir o dividir las aplicaciones en pequeños componentes, donde cada componente realiza su propia función de proceso de negocio, esta forma de trabajar se diferencia de una arquitectura monolítica, que es de un solo componente, la separación de componentes en microservicios, la complejidad de su

integración aumenta, por lo que se decide utilizar comunicación asíncrona basada en eventos para separar las interfaces de integración entre servicios (Alshuqayran et al., 2018). Un aspecto importante que los implementadores de esta arquitectura de microservicios deben considerar es identificar el límite de cada microservicio, especialmente el límite de los microservicios que se comunican a través de interfaces, donde los cambios pueden afectar a otros microservicios (Velepucha et al., 2019).

Según Roldán et al. (2018) las arquitecturas basadas en microservicios surgen para dar respuesta a los problemas de escalado de aplicaciones, en particular cuando se trata de aplicaciones distribuidas. Por otro lado, los microservicios son un paradigma arquitectónico moderno que se usa ampliamente para una variedad de casos de uso. Su objetivo es componer la funcionalidad de un sistema a partir de un conjunto de servicios pequeños y altamente independientes, o microservicios (Lähtevänoja, 2021).

2.2.12. Api

Una Api es un término vagamente definido que la gente usa de varias maneras para transmitir información de interfaz sobre un elemento, a veces, las personas reúnen una colección de firmas y las llaman la Api de un elemento (Clement et al., 2010).

Por otra parte, un Api es una especificación destinada a utilizarse como interfaz entre elementos de software para comunicarse entre sí; es decir, una API puede contener definiciones de rutinas, estructuras de datos, objetos de clase y variables (Best, 2018). Asimismo, una Api es el medio por el cual una pieza de código se comunica con otra pieza de código; por ejemplo, en bibliotecas Java para recopilación, entrada/salida o secuencias que proporcionan una variedad de API para realizar tareas específicas (Sharma, 2021).

2.2.13. Api Rest

El Api Rest es más flexible en su implementación porque depende únicamente de HTTP; esto le brinda flexibilidad, pero necesita estándares y convenciones para una verificación y validación estrictas (Sharma, 2021). Por otro lado, el api Rest es una de varias interfaces web que puede usar para acceder a sus datos de Salesforce sin usar la interfaz de usuario de Salesforce; es decir, con el acceso a la API, puede realizar operaciones e integrar Salesforce en sus aplicaciones como desee (Salesforce, 2022).

2.2.14. Api Gateway

El Api Gateway brinda un interfaz donde distintos clientes pueden tener acceso a los servicios individuales y resolver varios problemas, como qué hacer cuando desea enviar diferentes respuestas a diferentes clientes para el mismo servicio (Sharma, 2021). Por tanto, la Api Gateway maneja todas las tareas involucradas en aceptar y procesar hasta infinitas llamadas de Api simultáneas. Estas tareas incluyen la gestión del tráfico, la autorización y el acceso, control, seguimiento y gestión de versiones de API (Amazon Web Services, 2022).

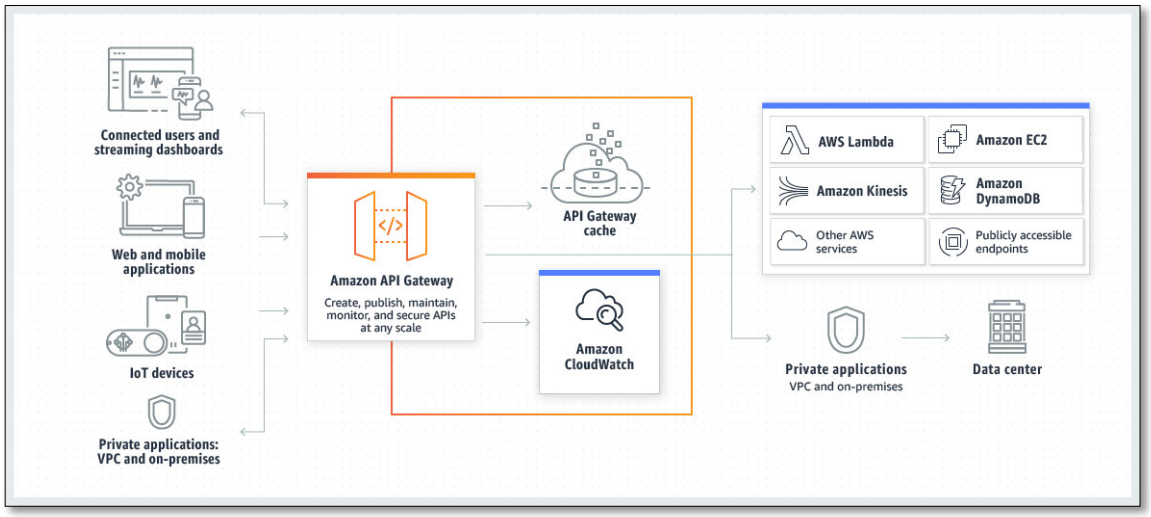


Figura 22 **Arquitectura de API Gateway**

Fuente. Obtenido de (Amazon Web Services, 2022).

2.2.15. RESTful

RESTful es un estilo arquitectónico de Api que utiliza solicitudes HTTP para recuperar y acceder a datos. se puede acceder a esos datos utilizando los tipos de datos GET, PUT, POST y DELETE; relacionados con las acciones de lectura, actualización, creación y eliminación asociadas con los recursos (Gillis, 2016).

2.2.16. Comunicación entre Microservicios

La comunicación entre Microservicios un sistema distribuido en varias máquinas, cada instancia de servicio suele ser un proceso o una máquina, por lo que es necesario implementar un mecanismo de comunicación entre procesos (Rodríguez et al., 2018).

2.2.17. Mensajería Asíncrona

La comunicación asíncrona ocurre cuando el diálogo entre dos agentes que se comunican tiene lugar sobre un número de conexiones discretas, los agentes no se convierten necesariamente sincronizados durante su intercambio de mensajes; también, se puede describir como una manera de comunicación donde una de las partes decide comunicarse con otra parte enviándole un mensaje sin necesidad de sincronizar con la otra parte o para establecer una conexión permanente (Lähtevänoja, 2021).

2.2.18. Event Store

Es un híbrido de intermediario de mensajes y de almacén de datos, este se comporta como un depósito de datos porque tiene una Api para insertar y recuperar eventos de un agregado por clave principal; por tanto, se comporta como un broker de mensajes porque tiene una Api para suscribirse a eventos (Richardson, 2018).

2.2.19. Event Bus

Una solución al problema de KWIC utiliza la arquitectura Event bus; es decir, los componentes que están interesados en ciertos eventos reciben una notificación cuando esos eventos están disponibles. Eso significa que los componentes se llaman implícitamente (Bijlsma et al., 2011). Mientras tanto, cuando Event bBus se distribuye para trabajar dentro de su propio proceso, no se puede garantizar la atomicidad, cada operación numerada funciona como una transacción independiente; por tanto, significa que pueden fallar de forma independiente, lo que puede dar lugar a incoherencias en los datos (Chandrasekaran & Krishnan, 2022).

2.2.20. Balanceo de Carga

El balanceo de capacidad del lado del servidor implica el enfoque clásico de equilibrio de carga en el que el tráfico se distribuye utilizando un distribuidor de carga que se coloca frente a los servidores; es decir, este distribuidor enruta la carga por igual o de acuerdo con ciertas reglas a los servidores que luego realizan el trabajo (Richardson, 2018). Por otra parte, el balanceo de carga es el proceso de compartir el tráfico de red entrante en tiempo simultáneo o discreto entre servidores denominados granja de servidores o grupo de servidores; por tanto, este proceso de compartir se puede escalar uniformemente o se puede realizar de acuerdo con ciertas reglas (Chandrasekaran & Krishnan, 2022).

2.2.21. CQRS

CQRS no es un patrón arquitectónico para aplicar a todos los contextos limitados de un sistema (Millett & Tune, 2015). Más bien, es una generalización basada en eventos del enfoque popular de usando RDBMS como sistema de registro y un motor de búsqueda de texto, como Elasticsearch, para manejar consultas de texto; pero la diferencia es que CQRS utiliza una gama más amplia de tipos de bases de datos, no solo un motor de búsqueda de texto (Richardson, 2018). Así, CQRS no es de ninguna manera una panacea, es un cambio de paradigma cuando se aplica a los problemas de software convencionales; por ello,

cualquier otra decisión de arquitectura, debe aplicar la debida diligencia al elegir adoptar CQRS para su situación (Chandrasekaran & Krishnan, 2022).

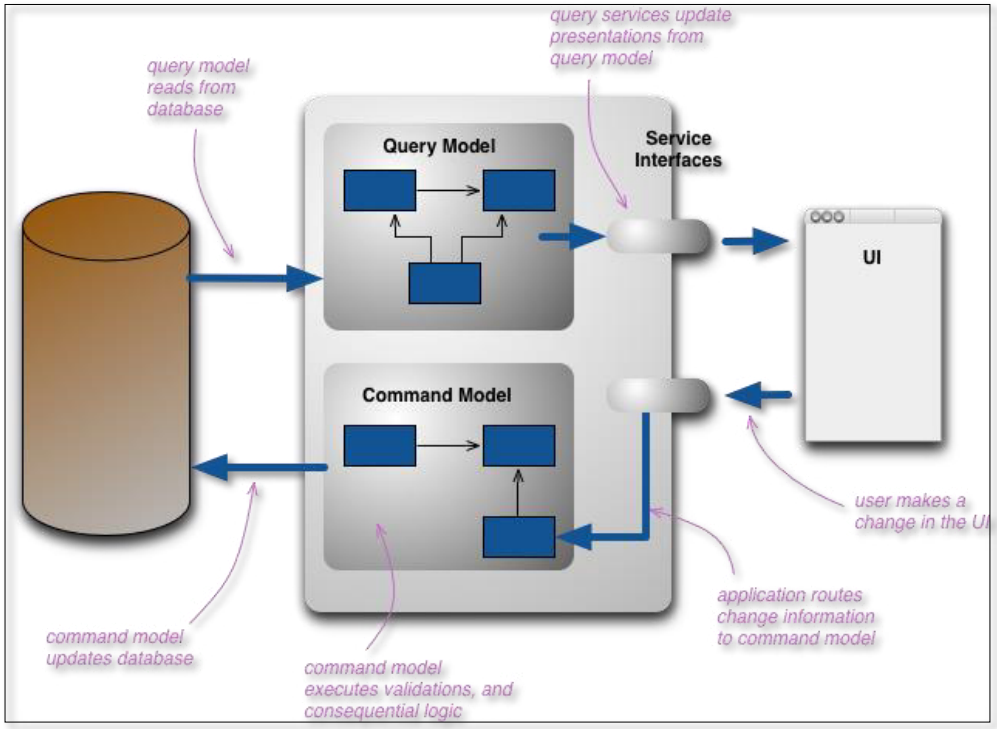


Figura 23 **Patrón CQRS**

Fuente. Obtenido de (Fowler, 2011).

2.2.22. Event Sourcing

Evento Sourcing es una forma diferente de estructurar la lógica empresarial y los agregados persistentes; es decir, persiste un agregado como una secuencia de eventos, donde cada evento representa un cambio de estado del agregado y una aplicación recrea el estado actual de un agregado reproduciendo los eventos (Richardson, 2018). Por otro lado, Evento Sourcing requiere un cambio de paradigma, lo que significa que los equipos comerciales y de desarrollo tendrán que dedicar tiempo y esfuerzo a comprender cómo funciona (Chandrasekaran & Krishnan, 2022).

2.2.23. Base de Datos

Según Evans (2003) la base de datos no solo interactúa con los objetos; está almacenando la forma persistente de los datos que componen los objetos mismos; a su vez, los desafíos técnicos de asignar objetos a tablas relacionales y almacenarlos y recuperarlos de manera efectiva. Existen tres casos comunes:

1. La base de datos es principalmente un depósito para los objetos.
2. La base de datos está delineada para otros sistemas.
3. La base de datos está trazada para este sistema, pero cumple otras funciones además del almacén de objetos.

CAPÍTULO 3: METODOLOGÍA

3.1. Tipo y diseño de la investigación

Este trabajo es de diseño *No Experimental y de tipo aplicado*, debido a que se implementara una herramienta para la generación y modelado de código de arquitecturas de software basadas en microservicios y no se realizaran cambios en las variables.

Hernández et al. (2014) mencionan, en la exploración no experimental existen variables independientes sin que se pueden manipular, estas variables no se pueden controlar directamente y no se puede influir porque ya han ocurrido, al igual que sus efectos.

Por otro lado, es de enfoque *Cuantitativo*, debido a que se busca medir la satisfacción del usuario post-Uso (usabilidad).

El alcance es *correlacional*, debido a que en la presente investigación se busca describir y predecir cómo se relacionan las variables en la realidad sin que el investigador pueda alterarlos en los resultados obtenidos al implementar la herramienta propuesta como propuesta de mejora.

3.2. Unidad de análisis

La unidad de análisis para el presente trabajo son los *Profesionales de tecnologías de información**.***

3.3. Población de estudio

Está formado por los *Profesionales de tecnologías de información dedicados al desarrollo de proyectos de software.*

3.4. Tamaño de la muestra

Se considera tipo de muestreo como ***No probabilístico***.

Y selección de muestra por conveniencia, profesionales que estén dispuestos a participar en el estudio.

Muestra:

10 profesionales conformados por arquitectos y desarrolladores de software.

Hernández et al. (2014) mencionan que, para un proceso cuantitativo, una muestra es el subconjunto de la población de interés de la que se recopilan datos que deben estar bien definidos y limitados de antemano y ser representativos de la población.

3.5. Selección de muestra

- Cada profesional elegido deberá tener experiencia en construcción de software mayor a 5 años.
- Deberán contar con experiencia en diseño de arquitecturas de software mayor a 5 años.
- Deberán poseer conocimientos básicos del DDD.

3.6. Técnicas de recolección de datos

✓ Para la obtención de datos se empleó *la encuesta,* con ello, se busca recopilar información de los resultados de experiencia de usuario obtenidos después de usar la herramienta.

Cuadro 3 **Técnicas e Instrumentos**

Técnica	Instrumento
Cuantitativa (aplicar la encuesta)	Cuestionario (Encuesta)

Fuente. Elaboración propia

3.7. Análisis e interpretación de la información

Se manejó el programa estadístico SPSS para realizar validación de confiablidad y el estudio de los datos recopilados, luego se realizó la explicación de los resultados para un mejor entendimiento del estudio.

CAPÍTULO 4: IMPLEMENTACIÓN DE LA HERRAMIENTA

El presente capítulo detalla el procedimiento de construcción de la herramienta planteada, tomando como base el marco de trabajo Scrum, el cual facilita una rápida finalización del proyecto propuesto.

4.1. Definición del requerimiento

Contar con una herramienta con entorno web que permita el modelado y la generación de códigos de arquitectura de software basado en microservicios, empleando el enfoque para construcción de software llamado Diseño Guiado por el Dominio (DDD).

4.1.1. Herramientas y tecnologías a emplear

IDE: Ms Visual Studio 2022 / Visual Studio Code
Lenguaje de Programación: C# con Net Core / Java – SpringBoot
Entorno Web: ASP.NET Core 6.0
Patrón: MVC
Librería: JQuery
Manejadores de Base de Datos: MySQL / MariaDB
Modelador de clases: StarUML
Modelado de proceso: Bizagi Modeler
Control de versiones: GitHub

4.2. Modelado de procesos de la herramienta propuesta

Como definición de la solución, se diseñó un modelado de procesos en la cual se expone el proceso detallado de la funcionalidad de la herramienta.

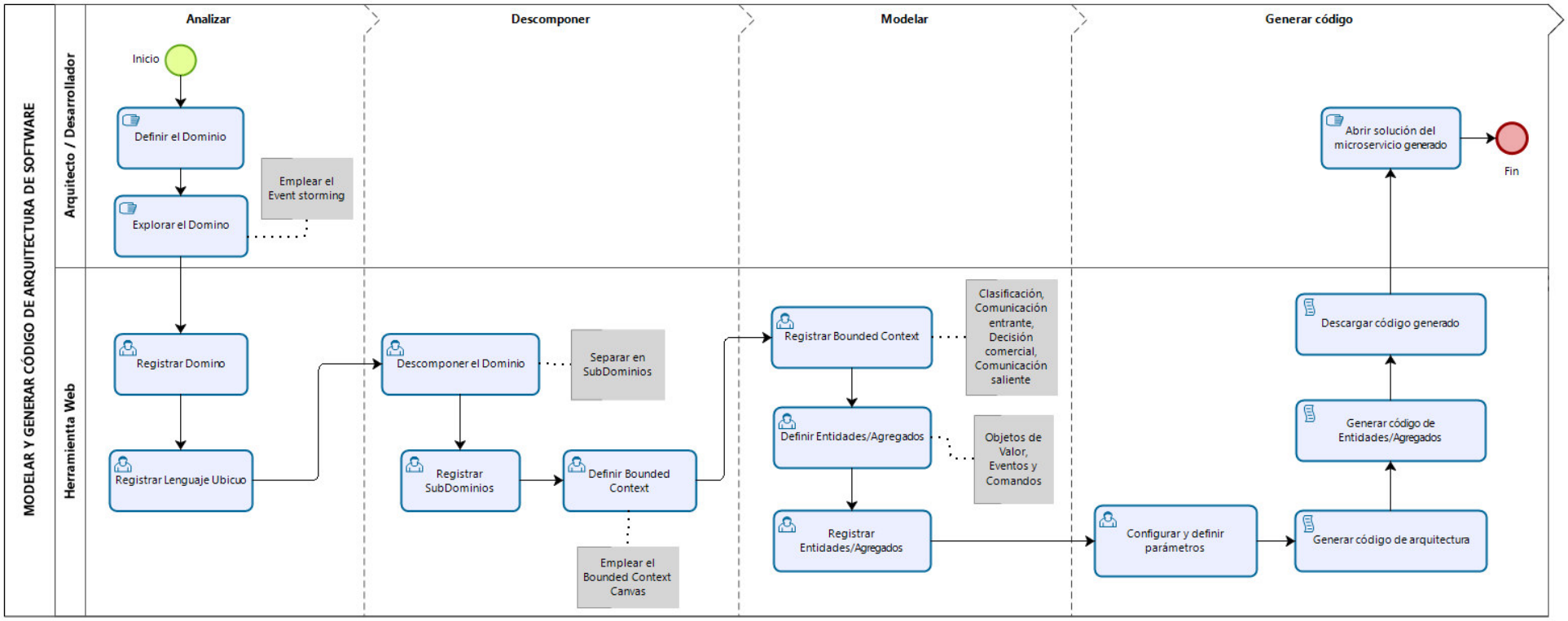


Figura 24 **Modelado de los procesos de la herramienta propuesta.**

Fuente. Elaboración propia

4.3. Metodología para el desarrollo

Para empezar el uso del scrum, se debe tener en cuenta Scrum como una forma de hacer el trabajo en equipo en pequeñas fracciones simultáneamente, con pruebas continua y ciclos de retroalimentación en el camino para aprender y mejorar a medida que avanza. Scrum ayuda a los equipos y personas a crear valor de forma incremental y de forma colaborativa. A modo de marco ágil, Scrum provee la estructura suficiente para que las personas y los equipos se compongan en su forma de trabajar, al tiempo que agrega las prácticas correctas para optimizar sus necesidades específicas (scrum.org).

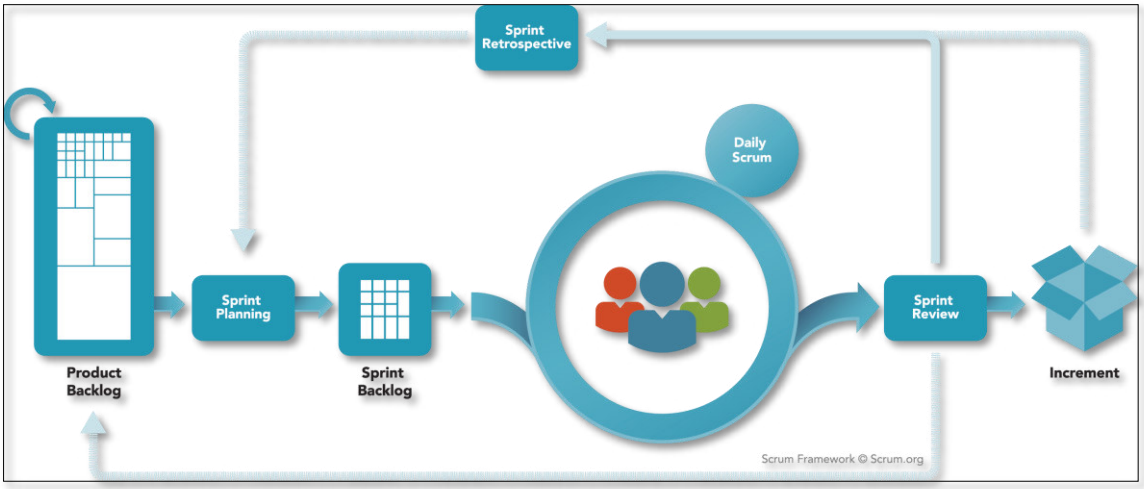


Figura 25 **El Marco Scrum**

Fuente: scrum.org

4.3.1. Definición de Roles

Cuadro 4 **Definición de roles de Scrum para el proyecto**

Definición de Roles	Encargado
Product Owner: Dueño del Producto encargado de expresar las necesidades del cliente.	El autor
Scrum Master: Líder del equipo de trabajo.	El autor
Development Team: Equipo de desarrollo capacitados en diferentes tecnologías para el desarrollo del producto software.	El autor

Fuente. Elaboración propia

4.3.2. Definición de Historias de Usuario

En seguida, se detalla la definición de los requerimientos de la herramienta a desarrollar, para ello se elabora en base a las historias de usuario con el cual se definirá cada sprint, la prioridad de desarrollo, el responsable de cada tarea y el control de los estados.

Se han dividido en 8 Historias de usuario, cada historia cuenta con sus respectivos criterios de aceptación:

- *Aplicación Web:* Desarrollo de la herramienta propuesta.

- *Modelado de arquitectura de software con Diseño Estratégico del DDD:* Desarrollo de los formularios web para la definición de Dominio, Subdominio y el Bounded Context (Contexto Limitado).

- *Modelado de arquitectura de software con Diseño Táctico del DDD:* Desarrollo de formularios web para la definición de los agregados, entidades y objetos de valor.

- *Configuración, definición de parámetros:* Ingreso de parámetros para definir la creación del código de arquitectura.

- *Generación de código de Arquitectura:* Proceso de generación de código de arquitectura propuesta para Spring Boot.

- *Generación de agregados y entidades:* Proceso de creación de Entidades y Agregados dentro de código generado.

- *Descargar código:* Proceso para descargar código fuente generado.

- *Importar y abrir código de microservicio*: Proceso complementario de importar y abrir el código fuente descargado de la herramienta, para posteriormente compilar, ejecutar y probar el microservicio.

Cuadro 5 **Definición de Historias de Usuario**

ID_HU	Titulo	Como	Necesito	Para	Criterios de Aceptación	Prioridad	Sprint	Story Points	Estado	Observaciones	Responsable
HU001	Aplicación Web	Arquitecto/ Desarrollador	Navegar en la aplicación Web	Seleccionar opciones de modelado y codificación.	Ingresar a la aplicación Web	1	1		Finalizado		El autor
					Visualizar el menú de opciones de modelado y codificación	1	1		Finalizado		El autor
HU002	Modelado de arquitectura de software con Diseño Estratégico del DDD	Arquitecto/ Desarrollador	Modelar la arquitectura de software en base al Diseño Estratégico del DDD	Entender los diferentes contextos limitados y lenguaje ubicuo del negocio.	Registrar el dominio y lenguaje ubicuo.	1	2		Finalizado		El autor
					Descomponer el dominio, identificar Subdominios a incluir y excluir.	1	2		Finalizado		El autor
					Registrar subdominios	1	2		Finalizado		El autor
					Registrar los contextos limitados	1	2		Finalizado		El autor
HU003	Modelado de arquitectura de software con Diseño Táctico del DDD	Arquitecto/ Desarrollador	Modelar la arquitectura de software en base al Diseño Táctico del DDD	Definir entidades, objetos de valor y agregados	Definir los agregados	1	3		Finalizado		El autor
					Registrar las entidades	1	3		Finalizado		El autor
					Registrar los objetos de valor, eventos y comandos.	1	3		Finalizado		El autor
HU004	Configuración, definición de parámetros	Arquitecto/ Desarrollador	Definir los parámetros	Generación de código del microservicio	Definir, nombre de servicio, nombre de paquete, base de datos, características y herramienta de compilación	2	4		Finalizado		El autor
HU005	Generación de código de Arquitectura	Arquitecto/ Desarrollador	Generar código de arquitectura	Para obtener la solución en código del microservicio	Ejecutar proceso de generación de código	2	4		Finalizado		El autor
HU006	Generación de agregados y entidades	Arquitecto/ Desarrollador	Generar agregados y entidades en la arquitectura	Completar la estructura del microservicio	Ejecutar proceso de generación de agregados y entidades	2	4		Finalizado		El autor
HU007	Descargar código	Arquitecto/ Desarrollador	Descargar código generado	Revisar la estructura de la solución	Permitir la descarga del código generado	3	5		Finalizado		El autor
HU008	Importar y abrir código de microservicio	Arquitecto/ Desarrollador	Importar y abrir código de microservicio generado	Compilar y probar la funcionalidad	Abrir microservicio y compilar.	4	5		Finalizado		El autor

Fuente. Elaboración propia

4.3.3. Definición de Sprint Backlog

Cuadro 6 **Definición del Sprint Backlog – Scrum**

ID Historias de Usuario	Descripción de item (Product Backlog)	Sprint	Estado	Responsable	Tiempo estimado (días)
HU001	Aplicación Web	1	Finalizado	Autor de proyecto	7
HU002	Modelado de arquitectura de software con Diseño Estratégico del DDD	2	Finalizado	Autor de proyecto	6
HU003	Modelado de arquitectura de software con Diseño Táctico del DDD	3	Finalizado	Autor de proyecto	4
HU004	Configuración, definición de parámetros	4	Finalizado	Autor de proyecto	1
HU005	Generación de código de Arquitectura	4	Finalizado	Autor de proyecto	3
HU006	Generación de agregados y entidades	4	Finalizado	Autor de proyecto	1
HU007	Descargar código	5	Finalizado	Autor de proyecto	1
HU008	Importar y abrir código de microservicio	5	Finalizado	Autor de proyecto	1
				Total, tiempo estimado	**24**

Fuente. Elaboración propia

4.4. Desarrollo e incremento del sprint 1

En la etapa del sprint 1, se construye la herramienta web, desde la definición de arquitectura, su implementación, hasta su ejecución para ser probado y validado.

4.4.1. Elaboración del diagrama de Casos de Uso

Se elabora el diagrama de casos de uso para definir los procesos a considerar en el desarrollo del proyecto.

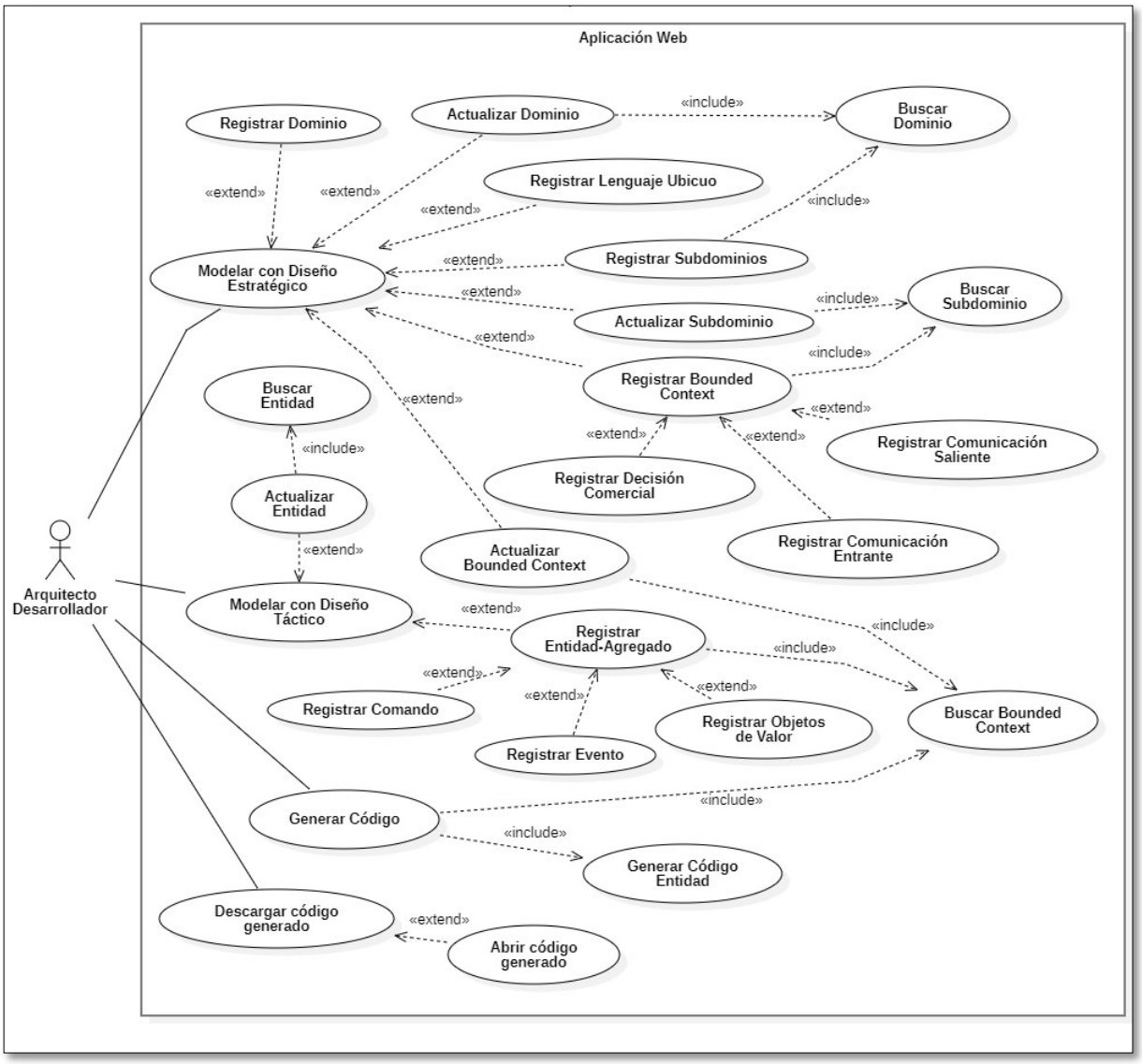


Figura 26 **Diagrama general de caso de uso**

Fuente. Elaboración propia

4.4.2. Modelos de Datos

Se elabora el modelo de datos con el cual se va a trabajar durante el desarrollo del proyecto propuesto.

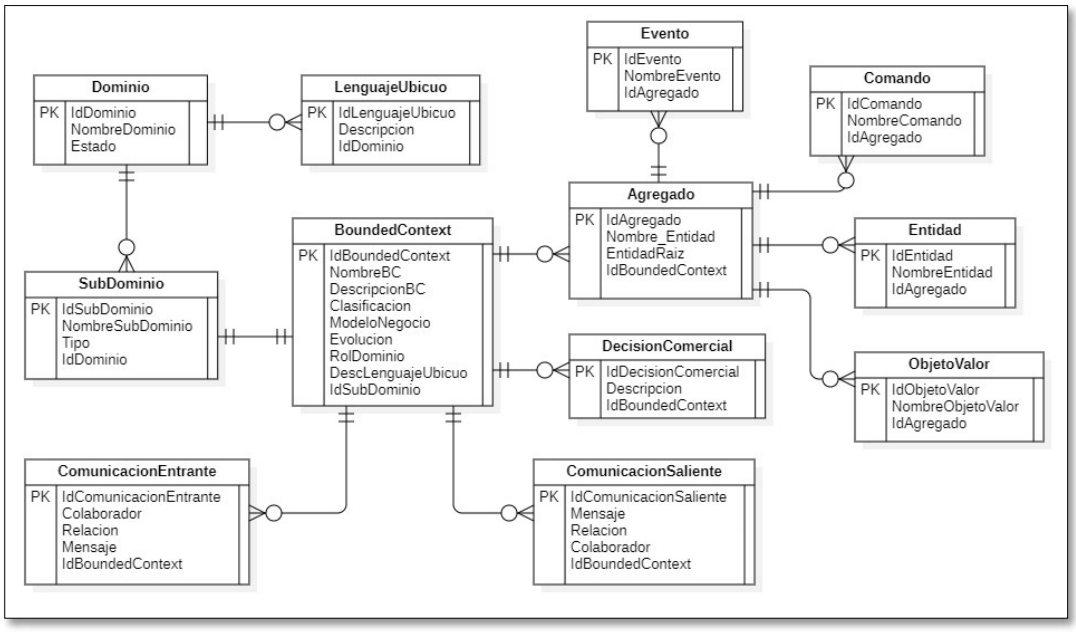


Figura 27 **Modelo de datos**

Fuente. Elaboración propia

4.4.3. Diseño de la Arquitectura

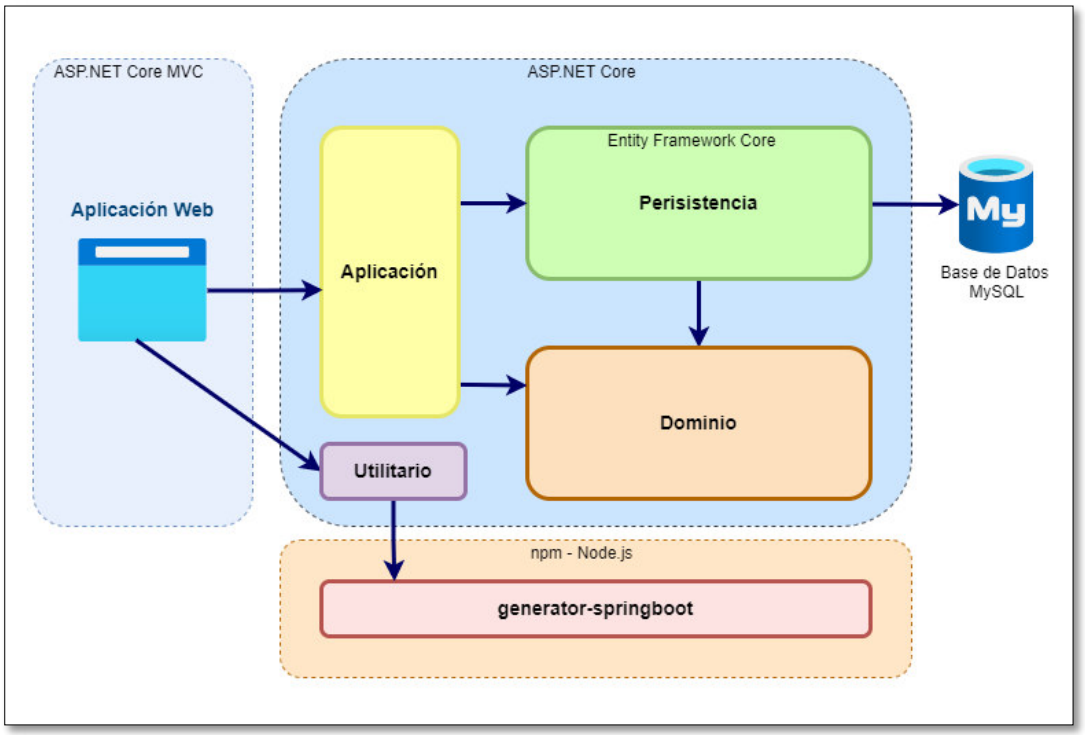


Figura 28 **Arquitectura de la herramienta**

Fuente. Elaboración propia

4.4.4. Estructura del Proyecto

Seguidamente, se muestra el desarrollo de la organización del proyecto separados por capas en base a la arquitectura diseñada. Aplicación, Dominio, Persistencia, Utilitario y WebApp (Aplicación Web).

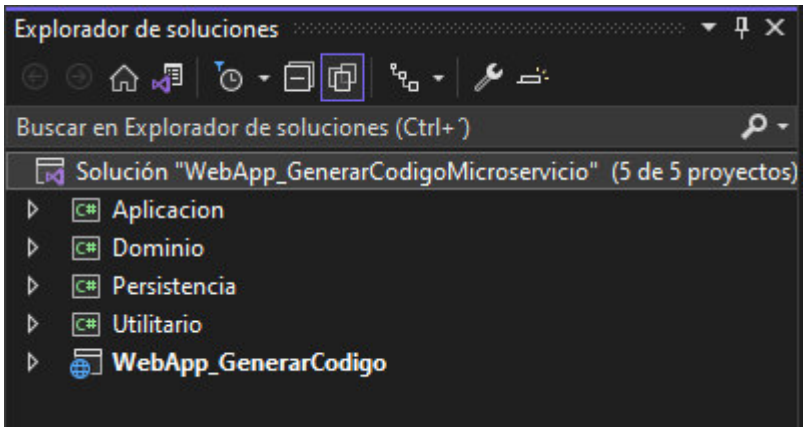


Figura 29 **Estructura del proyecto**

Fuente. Elaboración propia

Capa de Dominio

Es donde se define las entidades en base al modelo de negocio, los cuales serán mapeados para la generación de base de datos mediante uso del Entity Framework Core.

Se conoce a Entity Framework Core como una versión liviana de código abierto y multiplataforma de la muy conocida tecnología de acceso a datos Entity Framework, se adapta con diferentes motores de bases de datos (Microsoft, 2022).

En el presente proyecto, se definen los dominios: Aggregate, BoundedContext, Comando, ComunicaciónEntrante, ComuicaciónSaliente, DecisionComercial, Dominioo, Evento, LenguajeUbicuo, ObjetoValor y SubDominio.

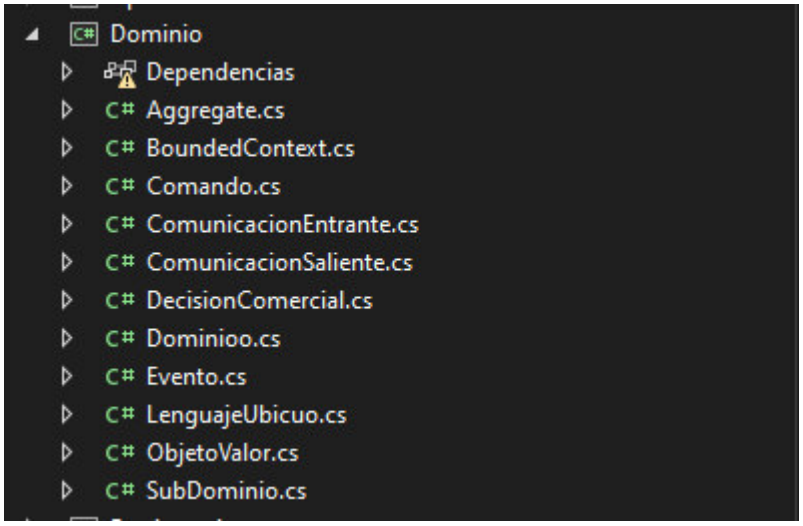


Figura 30 **Estructura de la capa Dominio**

Fuente. Elaboración propia

Capa de Persistencia

En esta parte de la estructura, se define la clase GenerarCodigoDbContext encargado de comunicarse con la base de datos.

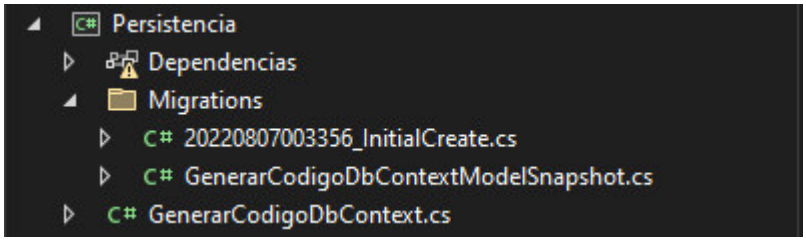


Figura 31 **Estructura de la capa Persistencia**

Fuente. Elaboración propia

Previo a la configuración, se instala los componentes necesarios a usar para realizar la operación de definición y creación de la base de datos con las respectivas tablas del proyecto en desarrollo.

- Microsoft.EntityFrameworkCore 6.0.9
- Microsoft.EntityFrameworkCore.Design 6.0.9
- Microsoft.EntityFrameworkCore.Tools 6.0.9
- Pomelo.EntityFrameworkCore.MySql 6.0.2

Este último componente orientado para la conocida base de datos MySQL que se emplea en el presente proyecto.

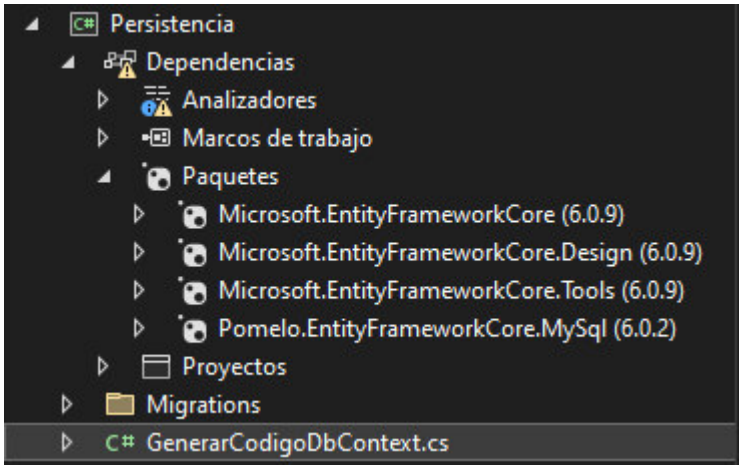


Figura 32 **Componentes necesarios para migración de los datos**

Fuente. Elaboración propia

Clase creada para definir los Dominios, de los cuales se crearán las tablas en la base de datos.

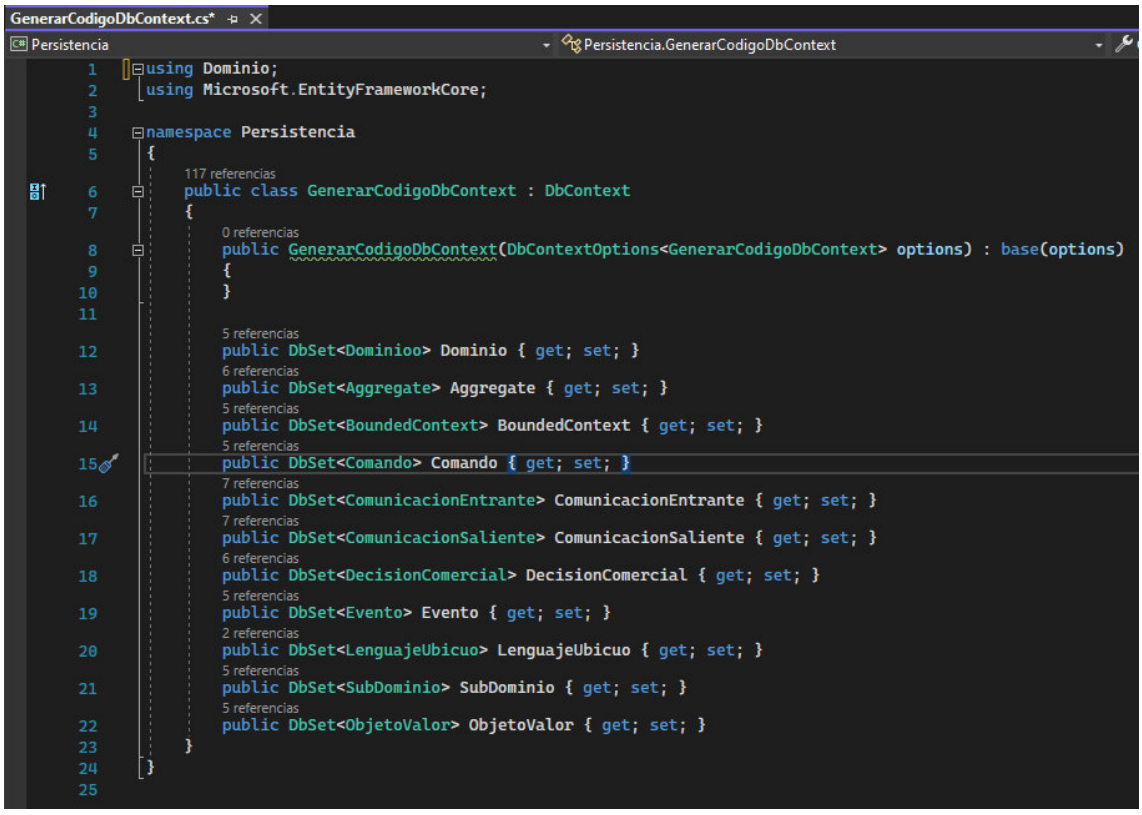


Figura 33 **Implementación de la capa Persistencia**

Fuente. Elaboración propia

Previo a la generación de las tablas, se define el acceso hacia la base de datos creada, esto dentro del archivo appsettings.json de la capa WebApp_GenerarCodigo.

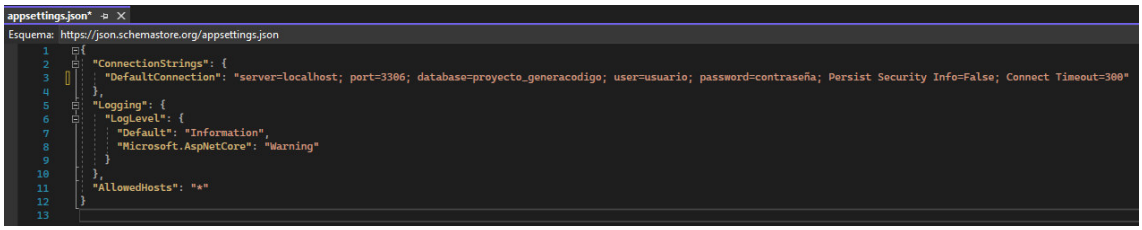


Figura 34 **Definición de la conexión a la base de datos.**
Fuente. Elaboración propia

Mediante el uso del IDE Visual Studio 2022, se procede a abrir la herramienta administradora de paquetes NuGet, Consola del Administrador de Paquetes para ejecutar comandos.

Para generar el directorio Migrations con las respectivas clases de creación de tablas, se procede a ejecutar el comando:

dotnet ef migrations add InitialCreate -p Persistencia/ -s WebApp_GenerarCodigo/

Para generación de tablas en la base de datos, se continua con la ejecución del comando:

dotnet ef database update -p Persistencia/ -s WebApp_GenerarCodigo/

Seguidamente, se obtiene el resultado de la generación de las tablas:

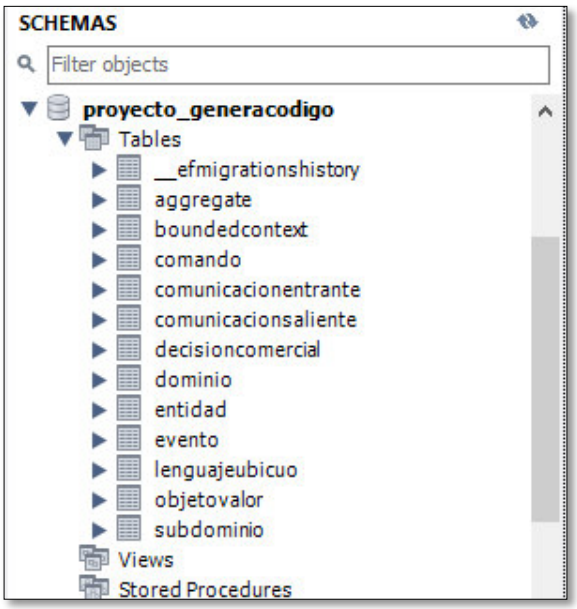


Figura 35 **Listado de tablas creados desde el proyecto.**

Fuente. Elaboración propia

Diagrama de Base de Datos

Esquema obtenido desde la base de datos creada dentro del el proyecto.

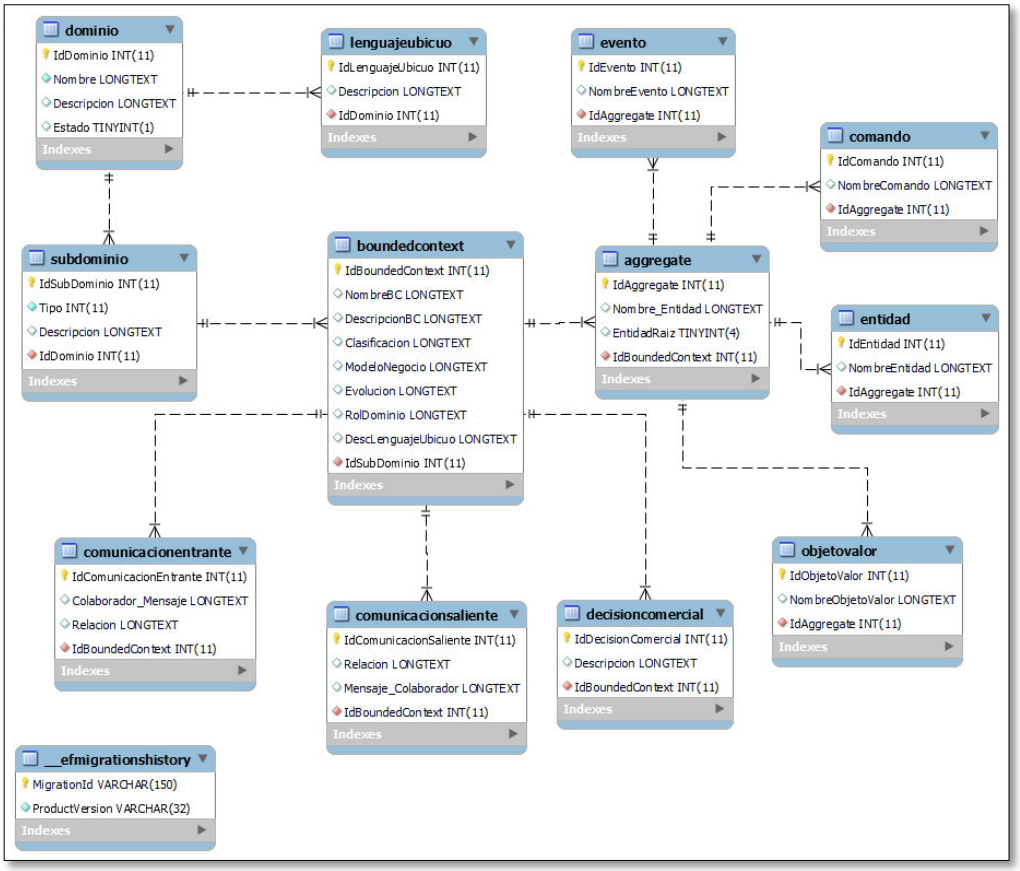


Figura 36 **Diagrama de la base de datos**

Fuente. Elaboración propia

Capa de Aplicación

No deberá contener lógica de negocio, solo proceso de relación referentes a requerimientos técnicos de aplicación, los cuales pueden ser combinación de invocaciones a repositorios para guardar datos, trasformaciones de formato de datos e invocaciones a componentes de la infraestructura para realizar procesos complementarios de la aplicación (Torre et al., 2010).

En esta capa se definen los componentes intermediarios iniciando en la capa interfaz de usuario, llegando hasta la capa persistencia de datos y dominio.

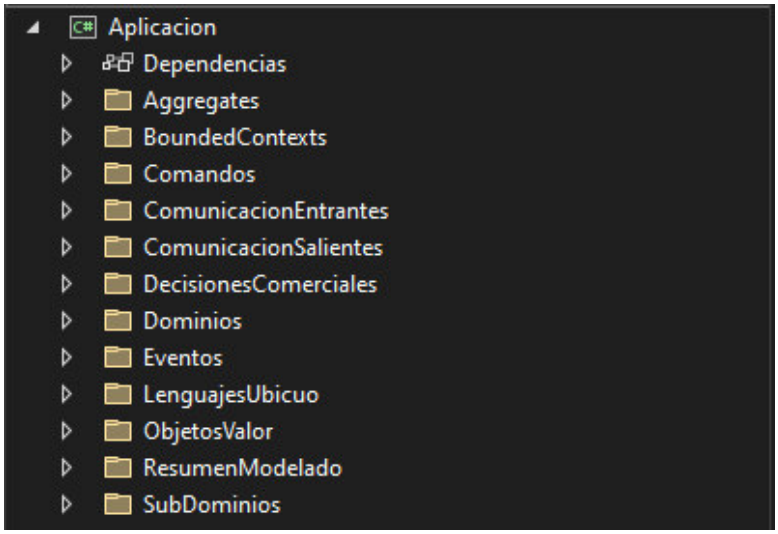


Figura 37 **Estructura de la capa Aplicación del proyecto**

Fuente. Elaboración propia

Capa Utilitario

En este componente del proyecto se define una nueva capa que ejecuta otro proyecto complementario de generación de código (generator-springboot).

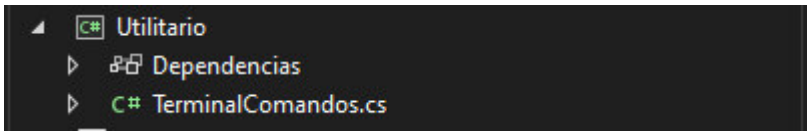


Figura 38 **Estructura de la capa Utilitario**

Fuente. Elaboración propia

Capa Interfaz de Usuario (WebApp)

Parte del proyecto con el que interactúan los usuarios para tener acceso a las funcionalidades que se han desarrollado. Para su desarrollo, se emplea el patrón MVC.

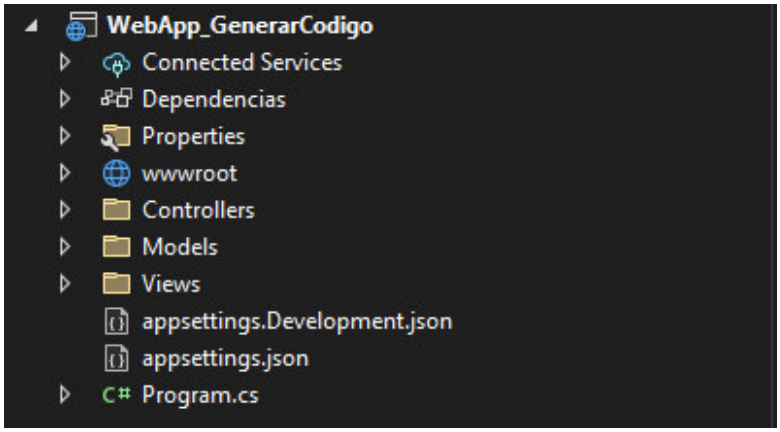


Figura 39 **Estructura de la capa WebApp**

Fuente. Elaboración propia

Patrón MVC:

Un patrón MVC, es uno de los patrones más importantes en la historia de desarrollo y en la actualidad es el más usado para construir software, pertenece al conjunto de patrones en la lista de estilos arquitectural de presentación apartada, este patrón separa el código comprometido de la grafía de los datos en pantalla del código delegado de la acción de modelo de negocio. Se divide en 3 tipos: Modelo, vista y controlador (Torre et al., 2010).

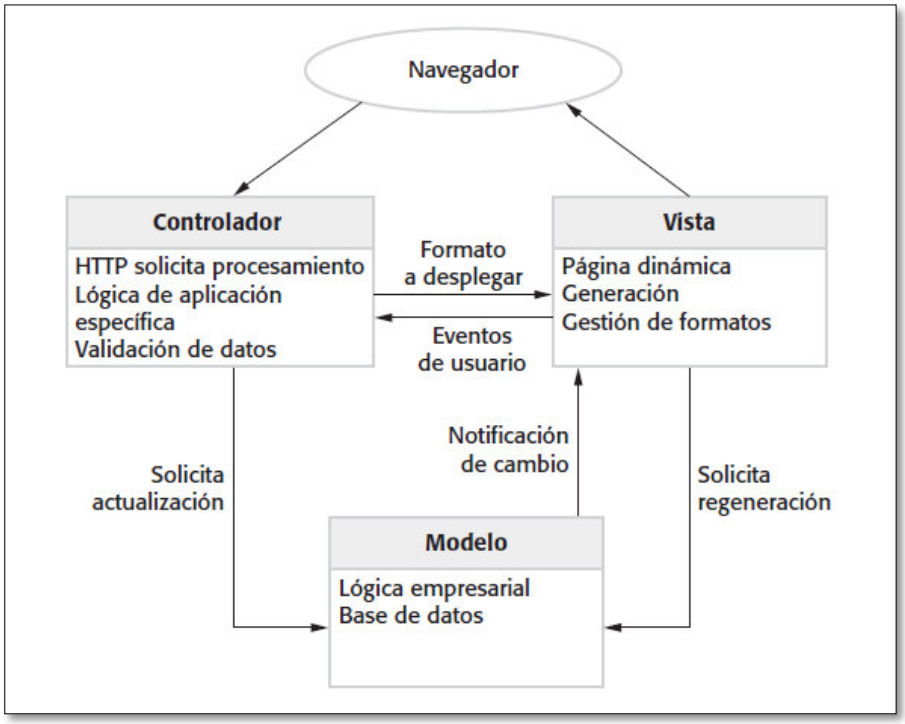


Figura 40 **Arquitectura de aplicación Web con el patrón MVC**

Fuente. Sommerville (2011).

Para el presente proyecto se han definido Modelos, Vistas y Controladores los cuales forman parte de la capa de presentación (aplicación web) que se ha propuesto desarrollar.

Modelo

Forman parte del estudio que efectúan el conocimiento para el dominio de datos, recobran y depositan el estado de modelo en base de datos, se puede mencionar como ejemplo a un determinado objeto que recupera información desde la base de datos, trabaja y vuelve a escribir la información que se ha actualizado (Microsoft, 2022).

Para el presente proyecto se ha definido modelos que son usados entre el Controlador y la Vista.

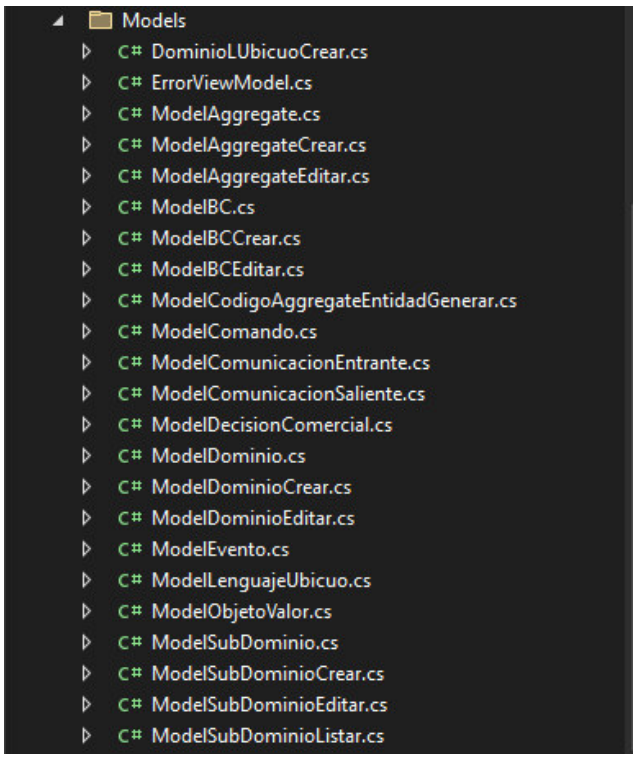


Figura 41 **Estructura del modelo en la capa WebApp**

Fuente. Elaboración propia

Vista

Son módulos que exponen la capa de presentación de una aplicación, generalmente, se construyen desde de los datos del modelo (Microsoft, 2022).

Para el presente proyecto, se define las siguientes vistas: DEBoundedContext, DEDominio, DESubDominio, DTAggregate y GeneraCodigo. Como complemento se configura las vistas Home y Shared.

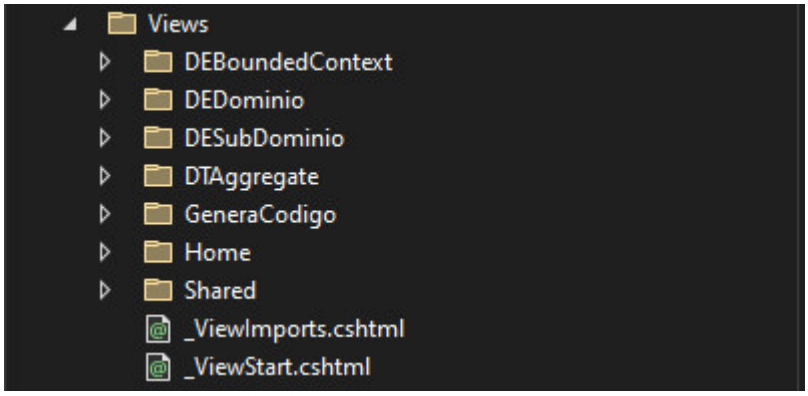


Figura 42 **Estructura de la vista en la capa WebApp**
Fuente. Elaboración propia

Controlador

Controlan la interacción de los usuarios trabajando con los modelos y eligen la vista para publicar la interfaz de usuario. En MVC, la vista se encarga solo de exponer información, el controlador interviene y se manifiesta a la interacción y los datos que ingresan los usuarios (Microsoft, 2022).

Para el presente proyecto, se definen los siguientes controladores: DEBoundedContextController, DEDominioController, DESubDominioController, DTAggregateController y GeneraCodigoController. Como complemento se adiciona el HomeController.

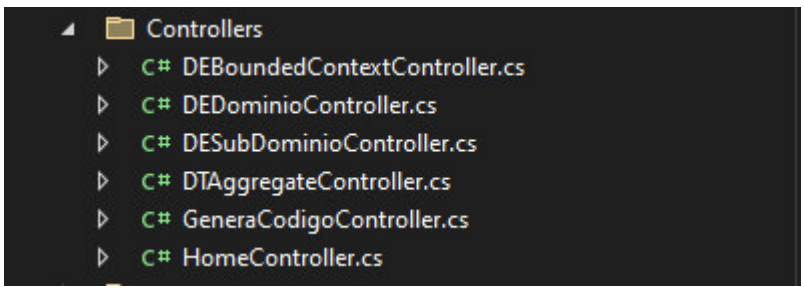


Figura 43 **Estructura del controlador en la capa WebApp**
Fuente. Elaboración propia

Estructura del proyecto generator-springboot

generator-springboot es un proyecto generador Yeoman para generar Microservicios con SpringBoot (K. Siva Prasad, 2020).

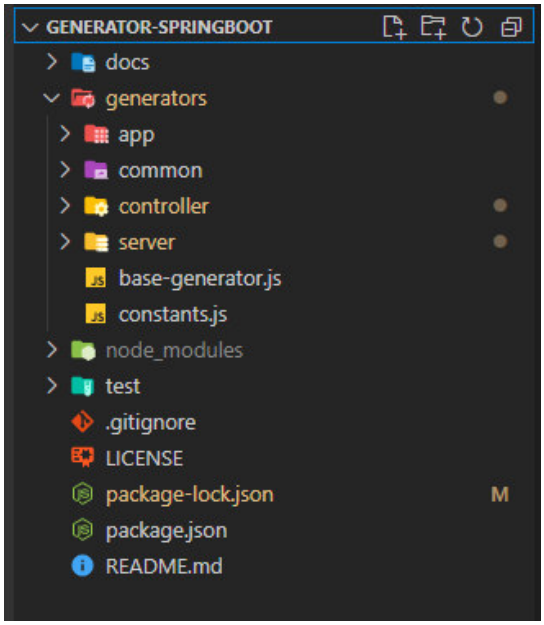


Figura 44 **Estructura del proyecto generator-springboot**

Fuente. Adaptado de (K. Siva Prasad, 2020).

4.4.5. Interfaz de la herramienta (Aplicación Web)

Ejecución de la herramienta aplicación web para mostrar los resultados obtenidos con el desarrollo del sprint 1:



Figura 45 **Pantalla principal de la herramienta web**

Fuente. Elaboración propia

4.5. Desarrollo e incremento del sprint 2

En la etapa del sprint 2, se desarrolla las funcionalidades de modelado de la herramienta en base al DDD diseño estratégico.

Registrar el Dominio y Lenguaje Ubicuo

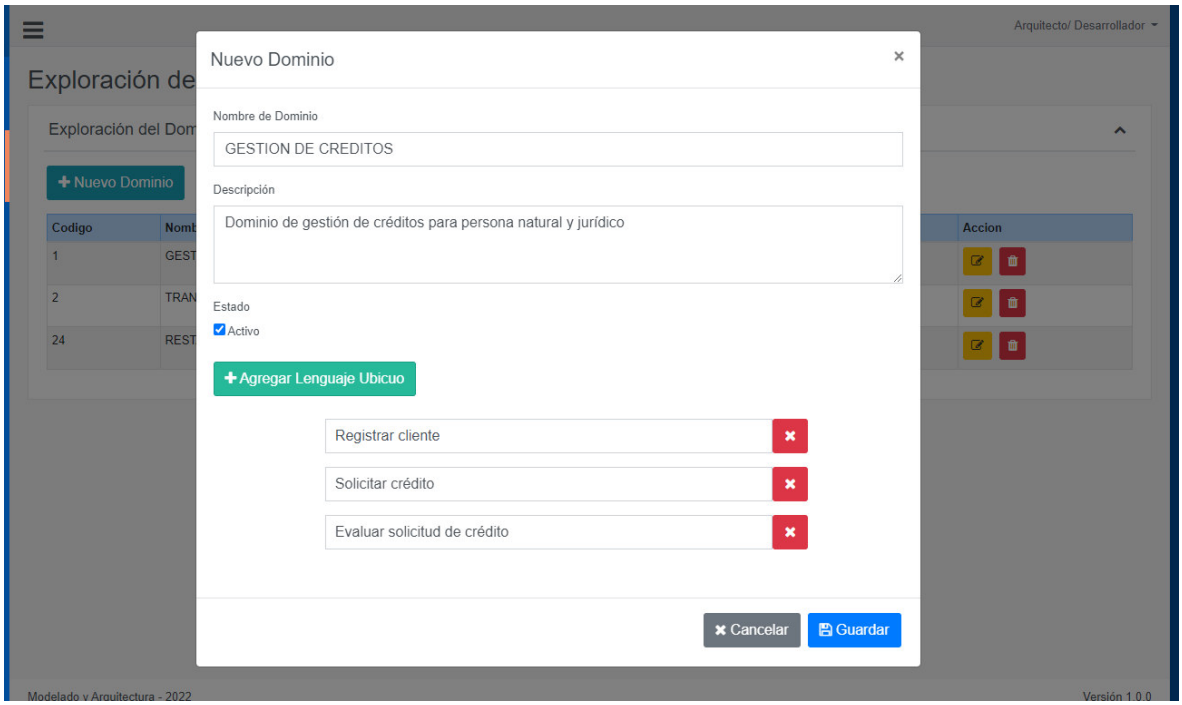


Figura 46 **Formulario de registro de Dominio y Lenguaje Ubicuo**

Fuente. Elaboración propia

Descomponer el Dominio – Registrar Subdominios

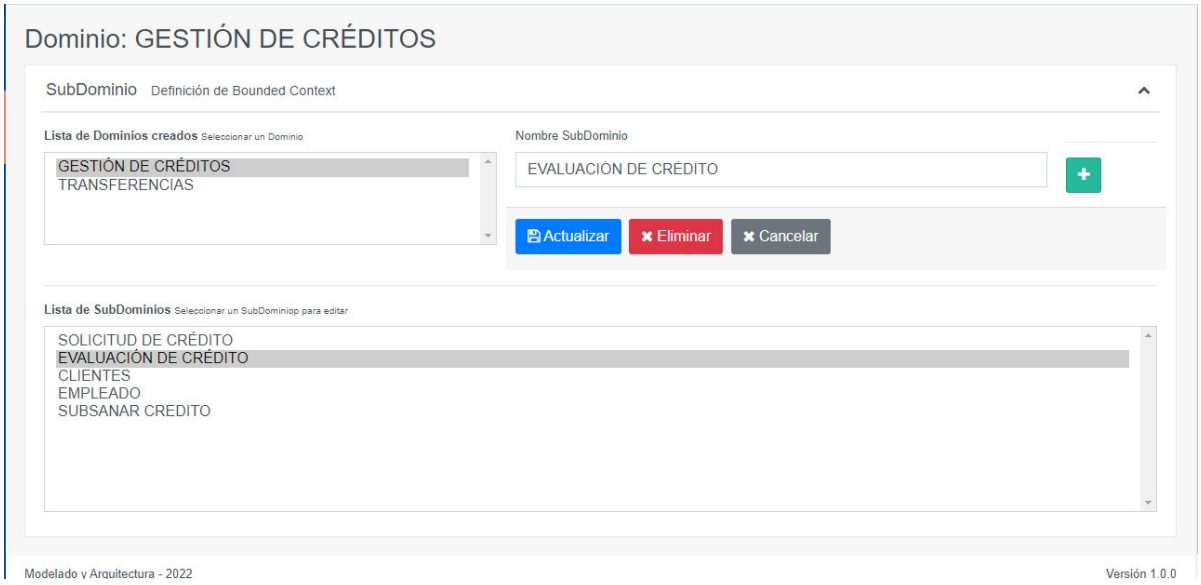


Figura 47 **Formulario de registro de Subdominios**

Fuente. Elaboración propia

Registrar el Contexto Limitado (Bounded Context Canvas)

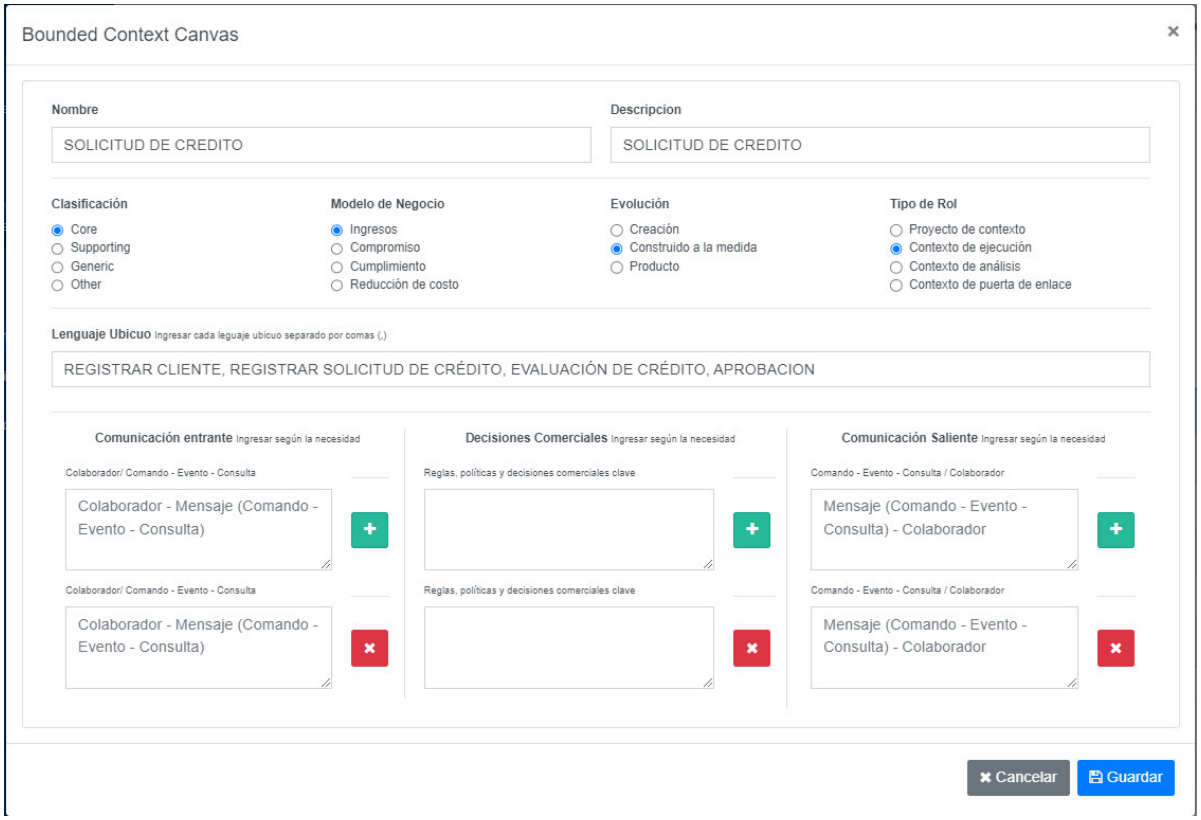


Figura 48 **Formulario de registro de Contexto Limitado**

Fuente. Adaptado de (Baas, Lindhard, Plöd, & Sanglan-Charlier, s.f.)

4.6. Desarrollo e incremento del sprint 3

En la etapa del sprint 3, se desarrolla las funcionalidades de modelado de la herramienta en base al DDD diseño táctico.

Definir Agregados – Registrar Entidades



Figura 49 **Formulario de registro de Entidades**

Fuente. Elaboración propia

Registrar Objetos de Valor, Eventos y Comandos



Figura 50 **Formulario de registro de Objetos de Valor, Eventos y Comandos**

Fuente. Elaboración propia

4.7. Desarrollo e incremento del sprint 4

En la etapa del sprint 4, se desarrolla las funcionalidades de configuración y definición de parámetros, generación de código de arquitectura y generación de agregados y entidades.

Configuración y Definición de Parámetros

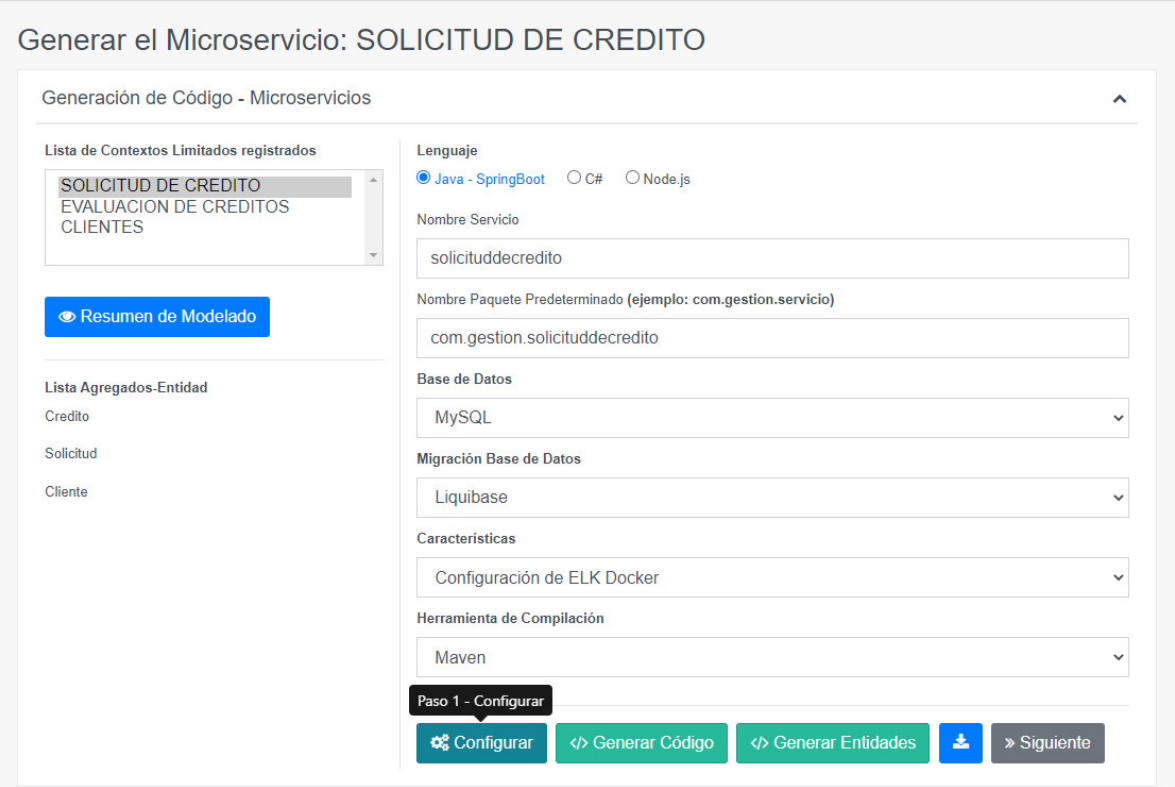


Figura 51 **Formulario para ingreso de parámetros**

Fuente. Elaboración propia

Generación de Código de Arquitectura

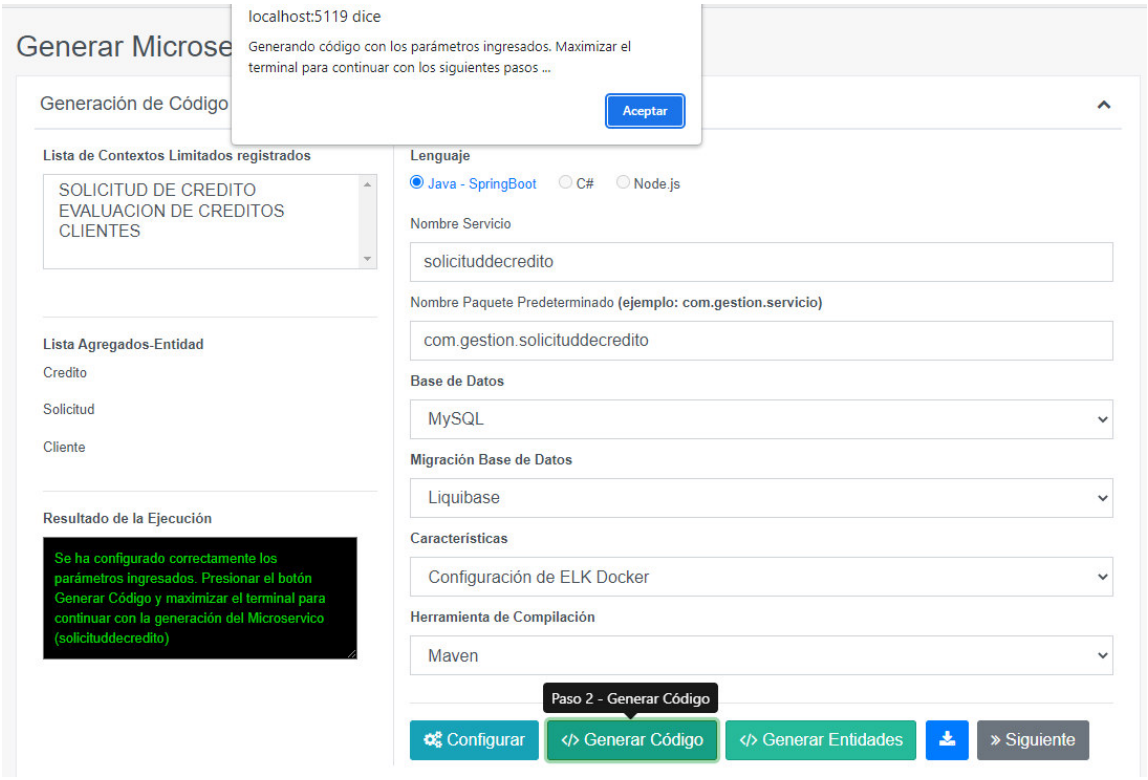


Figura 52 **Formulario para Generación de Código**

Fuente. Elaboración propia

Generación de Agregados - Entidades

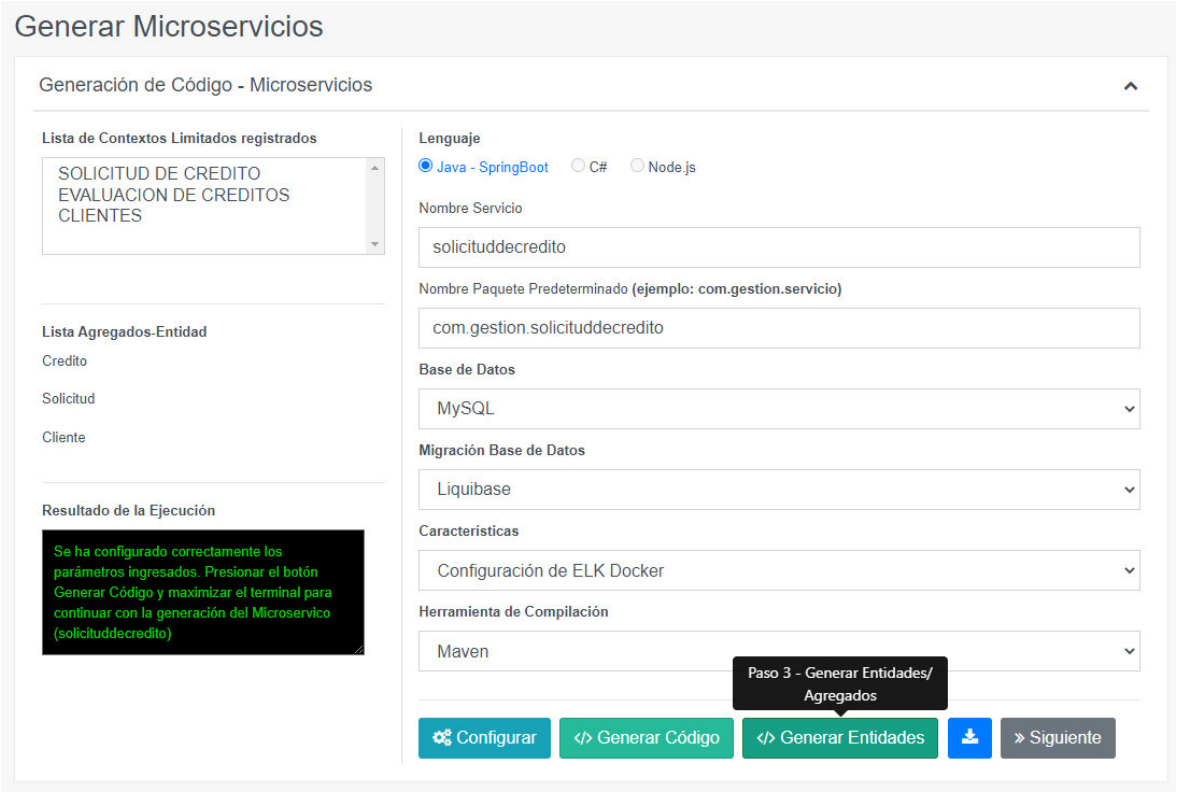


Figura 53 **Formulario para Generación de Agregados - Entidades**

Fuente. Elaboración propia

4.8. Desarrollo e incremento del sprint 5

En la etapa del sprint 5, se desarrolla la funcionalidad de descarga de código generado, importar y abrir el código de microservicio.

Descargar el código generado desde la herramienta

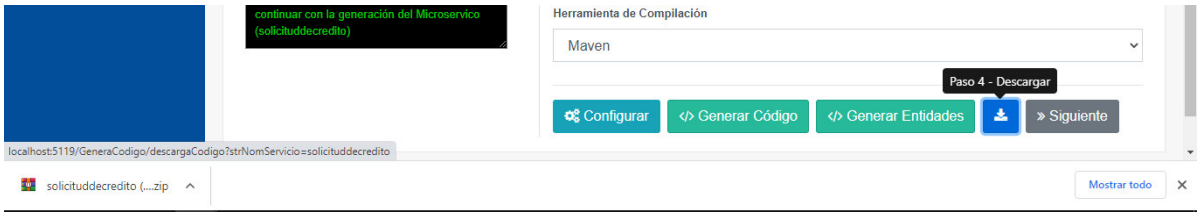


Figura 54 **Opción de descarga de código generado**

Fuente. Elaboración propia

Importar y Abrir el Código de Microservicio

Como resultado final, se importa y abre los microservicios generados con la herramienta, con ello, se podrá continuar los desarrollos adicionando nuevos componentes y funcionalidades.

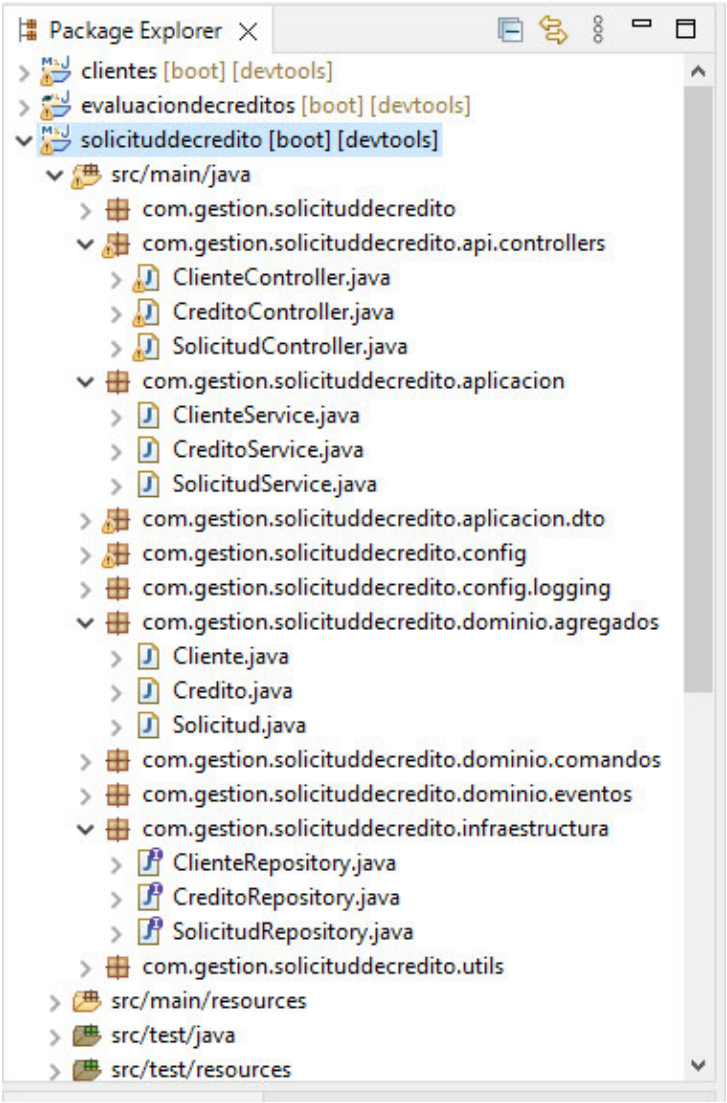


Figura 55 **Microservicios generados con la herramienta**

Fuente. Elaboración propia

Ejecución del microservicio abierto

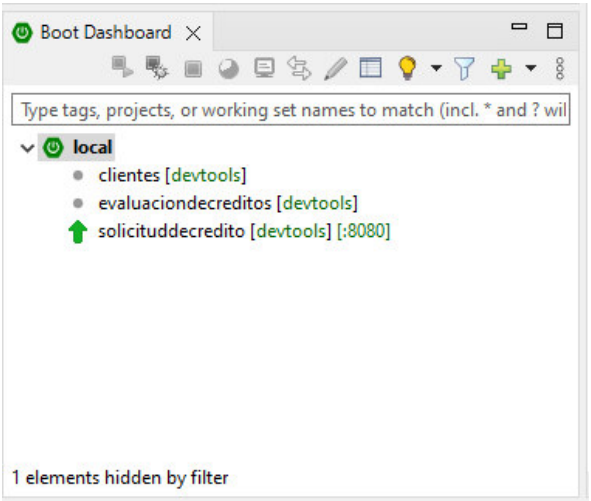


Figura 56 **Ejecución del microservicio solicituddecredito**

Fuente. Elaboración propia

Resultado de ejecución del microservicio

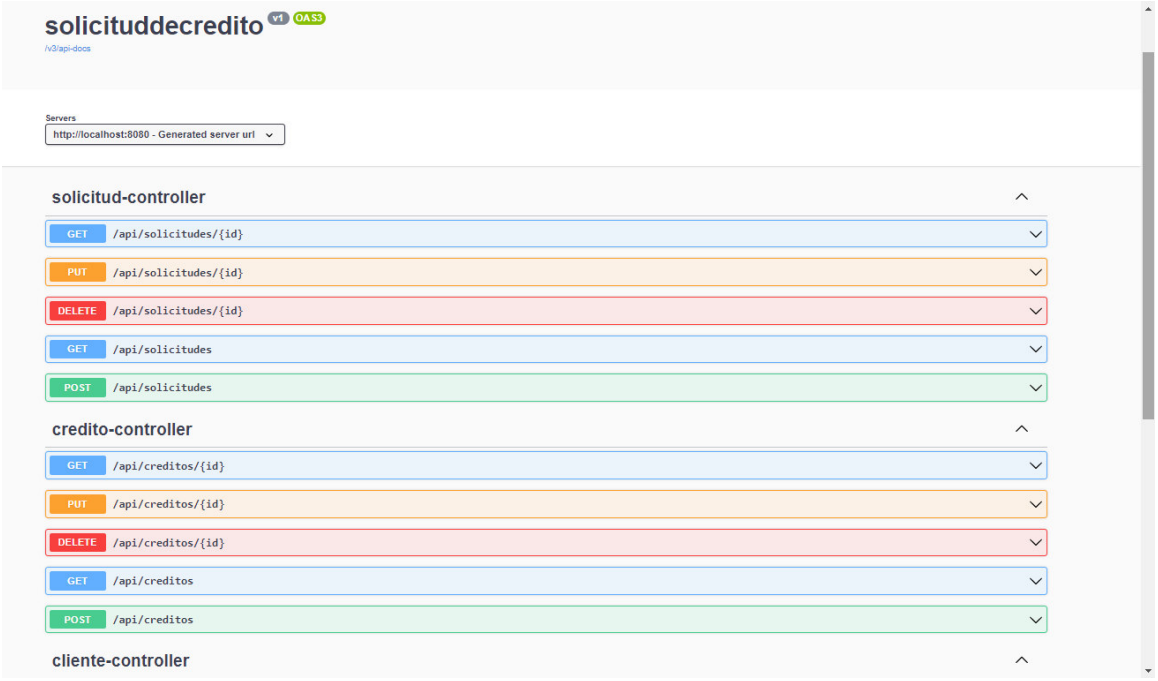


Figura 57 **Resultado de la ejecución usando Swagger**

Fuente. Elaboración propia

4.9. Resumen de Capítulo

En esta parte del capítulo se detalló el procedimiento de construcción de la herramienta web propuesta empleando el marco de trabajo Scrum divididos en 5 sprint para cada tarea a desarrollar.

En el sprint 1. Se desarrollo la aplicación web desde el análisis con la ilustración de los casos de uso, modelo de los datos, esquema de la arquitectura, seguido del desarrollo de la aplicación web en capas: Dominio, Persistencia, Aplicación, Utilitario e Interfaz de Usuario (Web), este último empleando el patrón MVC. Además, se implementó la integración del proyecto web con un proyecto de existente generator-springboot para la generación de código.

En el sprint 2. Se desarrollo los formularios con la funcionalidad de registrar y actualizar Dominio, Lenguaje Ubicuo, Subdominios y Contexto Limitado (Bounded Context).

En el sprint 3. Se desarrollo los formularios con funcionalidades de definir agregados, registrar y actualizar Entidades, Objetos de Valor, Eventos y Comandos.

En el sprint 4. Se desarrollo funcionalidades de configuración y definición de parámetros, generación de código de arquitectura y generación de agregados y entidades los cuales son los puntos importantes de la herramienta propuesta.

Finalmente, en el sprint 5. Se desarrollo la funcionalidad de descargar el código del microservicio generado, permitiendo importar y abrir el proyecto en un IDE de java para visualizar, compilar y ejecutar la solución.

CAPÍTULO 5: RESULTADOS Y DISCUSIÓN

A continuación, se detalla las pruebas de validación ejecutadas con el uso de la herramienta desarrollada para comprobar el cumplimiento del objetivo planteado como proyecto de investigación. En la primera parte, se define los objetivos a alcanzar con la validación de la herramienta, en la segunda parte se define el diseño de la validación a realizarse y en la tercera parte se detalla el proceso de ejecución de la validación. Finalmente se realiza a presentación de los resultados obtenidos.

5.1. Análisis, interpretación y discusión de resultados

En esta parte, se desarrollan los siguientes puntos que constan desde la definición de los objetivos de validación, diseño para la validación a realizar y la ejecución de la validación con el cual se busca recopilar información para un análisis de resultados y comprobar el aporte que genera la herramienta web propuesta.

5.1.1. Objetivos de la validación de la herramienta

- Conocer el tiempo invertido en el proceso de generación del código de arquitectura empleando el enfoque DDD con uso de una herramienta web desarrollada.
- Obtener porcentaje de la satisfacción de usuario (Arquitectos/ Desarrolladores) según la experiencia ganada al diseñar arquitecturas de software empleando el enfoque DDD con uso de una herramienta web desarrollada.

5.1.2. Diseño de la validación

- La validación se realizó con asistencia de 10 profesionales entre desarrolladores y arquitectos de software, cada uno con más de 5 años de experiencia.
- Se reviso el domino definido con sus respectivos subdominios, mapa de contexto y contextos limitados, todo ello elaborado mediante el uso del taller Event Storming.

- Cada participante fue capacitado con el uso de la herramienta a evaluar (ingreso de información y generación de código) desde el taller Event Storming mostrado.
- Cada participante usó con PC o Laptop con sistema operativo Windows 10 y navegadores web (Chrome, Edge y Firefox) para realizar la validación.

5.1.3. Ejecución de la validación

Para la ejecución de la validación, se partió desde un taller con Event Storming elaborado para definir el Dominio a analizar y descomponer para luego modelar y generar código con el uso de la herramienta desarrollada.

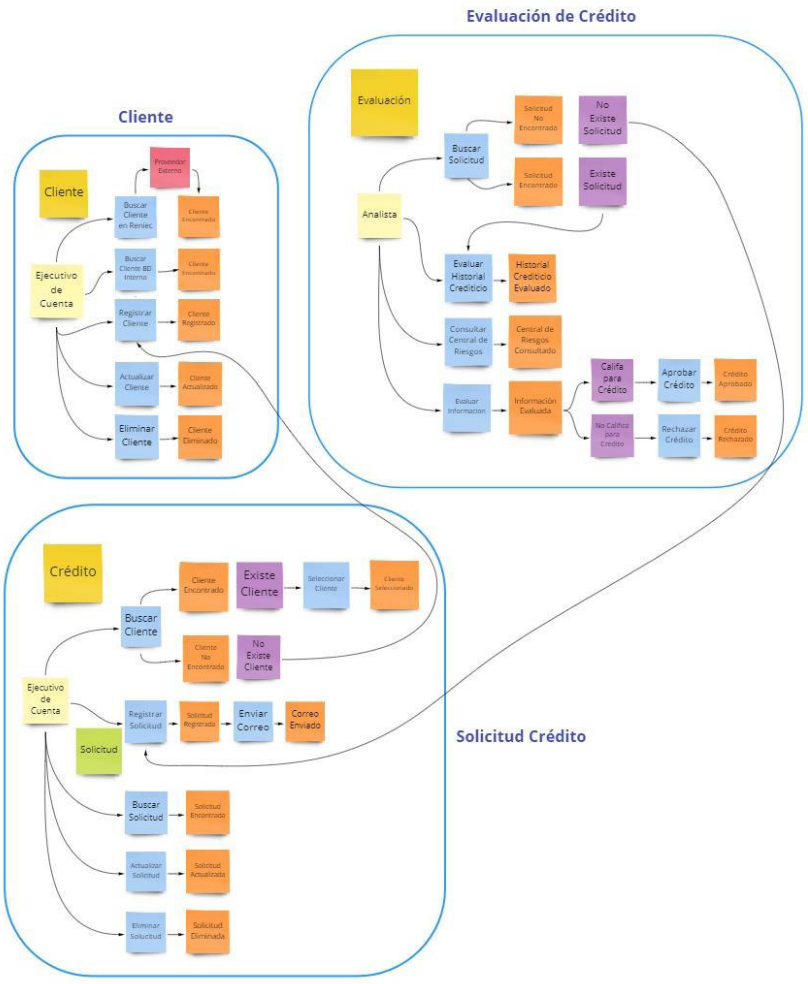


Figura 58 **Dominio analizado con el taller Event Storming**

Fuente. Elaboración propia

Cada participante accedió a la herramienta web y seleccionó la opción diseño estratégico donde realizaron el registro del dominio, descomposición del dominio, registraron cada subdominio identificado y seguido de la definición y registro de los bounded context (contextos limitados).

Luego cada participante selecciono la opción de diseño táctico donde definieron y registraron las entidades, objetos de valor y agregados.

Finalmente, cada participante selecciono la opción de generación de código de arquitectura siguiendo los procesos de la herramienta web propuesta, en base a los datos del dominio mostrado inicialmente en el taller Event Storming aplicando el enfoque del DDD.

5.2. Presentación de resultados

Los resultados obtenidos de cada participante se registraron en un cuadro para controlar la tarea realizada y el tiempo de esfuerzo tomado.

Cuadro 7 **Tiempo de esfuerzo tomado con el uso de la herramienta**

Participante	Uso de la Herramienta Web		
	Eficacia		Eficiencia
	¿Necesitó asistencia?	¿Completó tarea con éxito?	Tiempo de esfuerzo (minutos)
P1	No	Si	30
P2	No	Si	28
P3	No	Si	25
P4	No	Si	26
P5	Si	Si	30
P6	No	Si	24
P7	No	Si	27
P8	Si	Si	25
P9	No	Si	28
P10	No	Si	22
		Tiempo promedio	**26,5**

Fuente. Elaboración propia

En el cuadro 7, se expone el resultado de la validación realizada por los participantes, de los cuales solo 2 participantes requirieron asistencia para realizar la tarea de validación, todos los participantes completaron la tarea propuesta con éxito. También se observa que el tiempo promedio de esfuerza tomado para realizar la tarea fue de 26.5 minutos por participante.

En resumen, con el uso de la herramienta web propuesta se puede reducir tiempo de esfuerzo al diseñar de una arquitectura de software aplicando el enfoque del DDD, a diferencia que tomaría varias horas en deseñarlo de manera tradicional.

Finalmente, se realizó una encuesta de satisfacción de usuario Post-Uso a todos los participantes que realizaron la validación de la herramienta. Para la aplicación de la encuesta se elaboró el instrumento usando el “The Post-Study System Usability Questionnaire (PSSUQ)” traducido al español: El Cuestionario de Usabilidad del Sistema Posterior al Estudio (PSSUQ)

PSSUQ instrumento diseñado con objetivo de medir la satisfacción descubierta de los usuarios con sus sistemas informáticos, el cual fue iniciado en proyecto interno de IBS llamado SUMS (System Usability MetricS) dirigido por Suzanne Henry a finales de los años 80, el objetivo de SUMS era evidenciar y validar los pasos de medir la usabilidad del sistema teniendo en cuenta el rendimiento, la usabilidad y la satisfacción de los usuarios (Lewis, 2002).

El instrumento a lo largo del tiempo ha tenido 3 versiones, de los cuales la versión 1 conto con 18 ítems, luego la versión 2 conto con 19 ítems, y para la definir la versión 3, en un estudio realizado, se mostró que las altas confiabilidades de la escala PSSUQ no dependían de los ítems (3, 5 y 13) y su la eliminación de estos ítems no tuvieron un resultado fundamental

en la escala, con ello la versión 3 terminó con 16 ítems que se usa en la actualidad (Lewis, 2002).

El PSSUQ versión 3 cuenta con 16 preguntas, cada uno de ellos tiene 7 opciones (más una opción NA) para ser seleccionado por el encuestado. Siendo el 1 (totalmente de acuerdo) y 7 (totalmente en desacuerdo). En relación a los resultados, cuando sea menor, el rendimiento y la satisfacción se consideran como mejores (Sauro & Lewis, 2012).

Los elementos del PSSUQ generan cuatro puntajes: uno general y tres subescalas (Sauro & Lewis, 2012). Los cuales son:

- General: promedio de elementos 1 a 16 (todos).
- Calidad del sistema (SysQual): promedio de elementos 1 al 6.
- Calidad de la información (InfoQual): promedio de elementos 7 a 12.
- Calidad de interfaz (IntQual): promedio de elementos 13 a 15

	The Post-Study Usability Questionnaire Version 3		**Strongly agree** 1	2	3	4	5	6	**Strongly disagree** 7		**NA**
1	Overall, I am satisfied with how easy it is to use this system.		O	O	O	O	O	O	O		O
2	It was simple to use this system.		O	O	O	O	O	O	O		O
3	I was able to complete the tasks and scenarios quickly using this system.		O	O	O	O	O	O	O		O
4	I felt comfortable using this system.		O	O	O	O	O	O	O		O
5	It was easy to learn to use this system.		O	O	O	O	O	O	O		O
6	I believe I could become productive quickly using this system.		O	O	O	O	O	O	O		O
7	The system gave error messages that clearly told me how to fix problems.		O	O	O	O	O	O	O		O
8	Whenever I made a mistake using the system, I could recover easily and quickly.		O	O	O	O	O	O	O		O
9	The information (such as online help, on-screen messages and other documentation) provided with this system was clear.		O	O	O	O	O	O	O		O
10	It was easy to find the information I needed.		O	O	O	O	O	O	O		O
11	The information was effective in helping me complete the tasks and scenarios.		O	O	O	O	O	O	O		O
12	The organization of information on the system screens was clear.		O	O	O	O	O	O	O		O
13	The interface* of this system was pleasant.		O	O	O	O	O	O	O		O
14	I liked using the interface of this system.		O	O	O	O	O	O	O		O
15	This system has all the functions and capabilities I expect it to have.		O	O	O	O	O	O	O		O
16	Overall, I am satisfied with this system.		O	O	O	O	O	O	O		O

**The "interface" includes those items that you use to interact with the system. For example, some components of the interface are the keyboard, the mouse, the microphone, and the screens (including their graphics and language).*

Figura 59 **El PSSUQ versión 3.**

Fuente. Obtenido de (Sauro & Lewis, 2012).

Se realizo la encuesta con 10 participantes profesionales en desarrollo de software, a cada uno de ellos se le solicito completar la encuesta elaborada en base al PSSUQ, el resultado se puede observar en el cuadro 8:

Cuadro 8 **Resultados de la encuesta de satisfacción de usuario post-Uso**

Participante	Cuestionario																
	Año exp.	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
P1	8	2	1	2	2	2	1	2	3	3	2	3	2	2	2	3	2
P2	5	2	1	1	2	2	1	3	2	3	2	1	2	1	1	1	2
P3	8	1	2	2	1	1	2	2	3	2	1	2	2	1	1	2	1
P4	5	1	1	2	2	2	2	3	3	2	3	2	3	1	1	4	2
P5	12	2	1	1	1	1	1	3	1	2	1	1	2	1	1	2	2
P6	7	1	1	1	1	1	1	2	2	1	1	1	1	1	1	2	1
P7	10	1	1	2	2	1	2	3	3	2	2	2	2	2	1	2	1
P8	14	2	2	2	2	2	2	4	2	2	2	1	1	2	2	2	2
P9	9	2	2	1	2	2	1	3	3	3	2	2	1	2	2	1	2
P10	7	1	1	2	1	2	1	3	2	2	3	2	2	1	1	2	2

Fuente. Elaboración propia

Con el resultado general obtenido de las 16 preguntas, se le realizó la validación de confiabilidad empleando el Alfa de Cronbach, el resultado se expone a continuación:

Cuadro 9 **Validación de confiabilidad del instrumento.**

Alfa de Cronbach	N de elementos
,801	16

Fuente. Elaboración propia

El cuadro 9 muestra el resultado de la confiabilidad con .801 el cual es una fiabilidad buena que garantiza los resultados conseguidos mediante la encuesta.

Con ayuda del resultado obtenido, a continuación, se detallan cuadros y gráficos para un mejor estudio e interpretación de los resultados. En seguida, se expone los resultados alcanzados, teniendo la Puntuación general PSSUQ y cada subescala:

Cuadro 10 **Resultados de la encuesta por subescala.**

Puntuación general PSSUQ	Utilidad del Sistema (SYSUSE)	Calidad de la Información (INFOQUAL)	Calidad de la Interfaz (INTERQUAL)
2,13	1,67	2,50	2,33
1,69	1,50	2,17	1,00
1,63	1,50	2,00	1,33
2,13	1,67	2,67	2,00
1,44	1,17	1,67	1,33
1,19	1,00	1,33	1,33
1,81	1,50	2,33	1,67
2,00	2,00	2,00	2,00
1,94	1,67	2,33	1,67
1,75	1,33	2,33	1,33
1,77	**1,50**	**2,13**	**1,60**

Fuente. Elaboración propia

Dentro del cuadro 10 se puede apreciar los resultados alcanzados, para la Puntuación general PSSUQ el resultado es 1.77 siendo un resultado bastante bueno en cuanto a rendimiento y satisfacción de la herramienta web validada. Por otro lado, se tiene las sub escalas que son la Utilidad del Sistema (SYSUSE) con resultado 1.50, Calidad de la Información (INFOQUAL) con resultado 2.13 y Calidad de la Interfaz (INTERQUAL) con resultado 1.60, tomando como referencia lo que indica PSSUQ, cuando el resultado es menor y está más cerca al uno, se considera que la herramienta web ha tenido una de mayor satisfacción por los usuarios (arquitectos y desarrolladores) encuestado.

Exposición de los resultados haciendo comparación por cada subescala para tener una mejor visualización e interpretación:

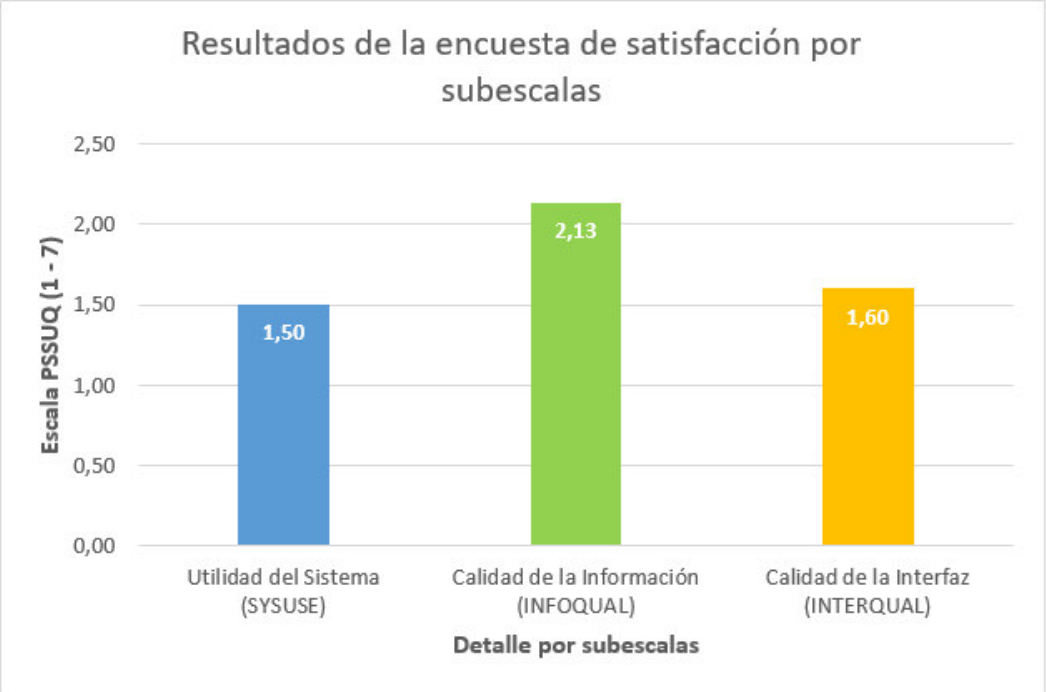


Figura 60 **Resultado de encuesta de satisfacción por subescalas**
Fuente. Elaboración propia

En la figura 60, se presenta los resultados en barras, donde la subescala Utilidad del Sistema causa mayor satisfacción para los encuestados con una puntuación de 1.50, seguido por la subescala Calidad de la Interfaz que causa también una alta satisfacción teniendo una puntuación de 1.60, mientras que la subescala Calidad de Información en comparación con las dos subescalas mencionadas, causa menos satisfacción con una puntuación 2.13.

CONCLUSIONES

En el presente trabajo, se propuso la implementación de una herramienta tecnológica capaz de modelar y generar código de arquitectura basada en microservicios con la finalidad de facilitar y agilizar la construcción de nuevos proyectos software y migración de sistemas legados, con ello ofrecer un aporte valioso a los profesionales en la ingeniería del software.

Se logró alcanzar el propósito principal con la culminación de la herramienta web para el modelado y generación de código de arquitectura de software basadas en microservicios empleando el enfoque del domain driven design (DDD), con el cual se pudo realizar el modelo mediante el diseño estratégico y táctico del DDD, para finalmente generar el código con las características y componentes que requiere un microservicio.

Se logró desarrollar un proyecto software y la migración de sistemas legados de manera Eficaz mediante la herramienta web para el modelado y generación de código de arquitecturas de software basadas en microservicios.

Se logró desarrollar un proyecto software y la migración de sistemas legados de manera Efectiva mediante la herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios.

Se logró modelar y generar código de arquitectura de software con facilidad de uso que aporta la herramienta.

Se validó la herramienta web desarrollada con la participación de arquitectos y desarrolladores con más de 5 años de experiencia en la industria del software. Del cual se obtuvo un tiempo de esfuerzo empleado de 26.5 minutos por participante siendo este resultado muy aceptable a diferencia de tiempo en horas que puede tomar el diseño de forma tradicional.

Por otro lado, se realizó la encuesta de satisfacción de usuario post-Uso, teniendo como resultado una puntuación General de 1.77 y por subescalas de Utilidad del Sistema (SYSUSE) con puntuación de 1.50, Calidad de la Información (INFOQUAL) con puntuación de 2.13 y Calidad de la Interfaz (INTERQUAL) con una puntuación de 1.60, los cuales se interpretan en

referencia al PSSUQ, cuando el resultado es menor o está cerca al uno, significa que la herramienta validada ha tenido una mayor aceptación de satisfacción de los usuarios.

Finalmente, se concluye que la herramienta desarrollada para el modelado y generación de código de arquitectura de software facilita y agiliza desarrollos de proyectos de software y la migración de sistemas legados desde la descomposición del dominio hasta la obtención de la estructura del proyecto, reduciendo el tiempo de desarrollo considerablemente.

RECOMENDACIONES

1. Para contar con una herramienta web más automatizada en la elaboración de modelado y diseño de arquitectura, se recomienda como trabajo futuro construir una interfaz web con la opción de realizar un taller de event storming para modelar junto a los expertos del dominio y de forma automática capturar los datos del modelo para procesarlos con la herramienta web hasta la generación del código.

2. Como complemento de la herramienta, se recomienda adicionar la opción de generación de documentación automática de la arquitectura diseñada, con ello ahorrar tiempo y esfuerzo en realizar la documentación necesaria para el proyecto.

3. En la generación del código, se recomienda adicionar funcionalidad de generar código para varios lenguajes de programación y definir las formas de comunicación entre microservicios para ahorrar mayor tiempo en el diseño de la arquitectura.

4. Se recomienda, realizar la validación de la herramienta con mayor número de expertos de desarrollo de software.

REFERENCIAS BIBLIOGRÁFICAS

Alshuqayran, N., Ali, N., & Evans, R. (2018). The question of radiation-induced phase changes*. *Journal of Physics and Chemistry of Solids*, *17*(2), 155–158. https://doi.org/10.1016/0022-3697(58)90179-3

Amazon Web Services. (2022). *Amazon API Gateway Developer Guide*. ews. https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html

Baas, K., Lindhard, K., Plöd, M., & Sanglan-Charlier, M. (s.f.). The Bounded Context Canvas. Obtenido de https://github.com/ddd-crew/bounded-context-canvas

Bachmann, F., Bass, L., Chastek, G., Donohoe, P., & Peruzzi, F. (2000). The Architecture Based Design Method. *Engineering*, 1–61. https://www.researchgate.net/publication/235088008_The_Architecture_Based_Design_Method

Balalaie, A., Heydarnoori, A., Jamshidi, P., Tamburri, D., & Lynn, T. (2018). Microservices migration patterns. *Software - Practice and Experience*, *48*(11), 2019–2042. https://doi.org/10.1002/spe.2608

Barrios, D. (2018). Arquitectura de microservicios Microservice architecture. *Tecnología Investigación y Academia*, *6*(1), 36–46. https://revistas.udistrital.edu.co/index.php/tia/article/view/9687/pdf

Best, J. (2018). Application Programming Interface (API). *Breaking Digital Gridlock*, 71–86. https://doi.org/10.1002/9781119421900.ch5

Betts, T. (2018). Doman driven design in practice. *Read Online on InfoQ*. https://www.infoq.com/minibooks/emag-domain-driven-design/

Bijlsma, A., Heeren, B., Roustsova, E., & Stuurman, S. (2011). *Software architecture*. Free technology academy. https://www.academia.edu/19620029/A_Bijlsma_B_J_Heeren_E_E_Roubtsova_S_Stuurman_Software_Architecture

Blancarte, O. (2020). *Introducción a la arquitectura de software – Un enfoque*

práctico (1st ed.). Oscar Blancarte Blog. https://pdfcoffee.com/introduccion-a-la-rquitectura-de-softwarepdf-2-pdf-free.html

Cañarte, T., & Morán, C. (2021). El Servicio Al Cliente Como Herramienta De Marketing Empresarial Customer Service As a Business Marketing Tool. *Journal Business Science*, *2*(2), 1–12. https://revistas.uleam.edu.ec/index.php/business_scienceLicenciadeCreativeCommons%0Ahttp://creativecommons.org/licenses/by-nc-sa/4.0

Carvalho, R., & Vendramel, W. (2019). Padrões de migração de sistemas legados para arquitetura baseada em microsserviços Migration patterns of legacy systems to microservices based architecture. *Revista de Sistemas e Computação*, *9*(1), 58–74. http://www.revistas.unifacs.br/index.php/rsc

Cervantes, H., Velasco-Elizondo, P., & Castro, L. (2016). ARQUITECTURA DE SOFTWARE. Conceptos y ciclo de desarrollo. México D.F.: Cengage Learning. https://issuu.com/cengagelatam/docs/arquitectura_de_software_cervantes

Chandrasekaran, P., & Krishnan, K. (2022). *Domain-Driven Design with Java – A Practitioner ' s Guide*. Packt Publishing.

Chen, C., Dong, C., Cai, J., & Cheng, X. (2019). Business Generator Model Based on Domain Driven Design. *Journal of Physics: Conference Series*, *1168*(5), 1–8. https://doi.org/10.1088/1742-6596/1168/5/052028

Clement, P., Bachmann, F., Bass, L., Garlan, D., Ivers, J., Little, R., Merson, P., Nord, R., & Stafford, J. (2010). *Documenting software architectures* (2nd ed.). Addison-Wesley Professional. https://www.oreilly.com/library/view/documenting-software-architectures/9780132488617/

Diepenbrock, A., Rademacher, F., & Sachweh, S. (2017). An Ontology-based Approach for Domain-driven Design of Microservice Architectures. *Lecture Notes in Informatics (LNI), Proceedings - Series of the Gesellschaft Fur Informatik (GI)*, *275*, 1777–1791. https://doi.org/10.18420/in2017_177

Eden, A. H. (2005). Strategic versus tactical design. *Proceedings of the Annual Hawaii International Conference on System Sciences*, 1–11. https://doi.org/10.1109/hicss.2005.563

Evans, E. (2003). *Domain-Driven design: Tackling complexity in the heart of software*. Addison-Wesley Professional. https://www.oreilly.com/library/view/domain-driven-design-tackling/0321125215/

Evans, E. (2015). *Domain-Driven Design Reference: Definitions and Pattern Summaries*. Google Libros. https://books.google.com.pe/books?hl=es&lr=&id=ccRsBgAAQBAJ&oi=fnd&pg=PA25&dq=Domain-Driven+Design+Reference:+Definitions+and+Pattern+Summaries&ots=gNd5VRkihF&sig=57SvvyMd2dWHX-gqqvLITcR5jkg#v=onepage&q=Domain-Driven Design Reference%3A Definitions and Pa

Fajar, A., Novianti, E., & Firmansyah, F. (2020). Design and Implementation of Microservices System Based on Domain-Driven Design. *International Journal of Emerging Trends in Engineering Research*, *8*(7), 3058–3062. https://doi.org/10.30534/ijeter/2020/30872020

Fowler, M. (2011). CQRS. Obtenido de https://martinfowler.com/bliki/CQRS.html

Fowler, M. (2014). BoundedContext. Obtenido de https://martinfowler.com/bliki/BoundedContext.html

Fowler, M. (2015). Architectures you Always Wondered About. *Infoq Microservices*, 1–37. https://vdoc.pub/download/architectures-you-always-wondered-about-2ep350gjgod0

Fowler, M. (2019). *Software Architecture Guide*. MartinFowler.Com. https://martinfowler.com/architecture/

Gamma, E., Helm, R., Johson, R., & Vlissides, J. (2003). *Patrones de Diseño: Elementos de software orientado a objetos Reutilizable*. https://profeuttec.yolasite.com/resources/Patrones de diseño - Erich Gamma.pdf

Gatell, A. (2019). Significado Del Proceso De Diseño En La Formación Del Arquitecto. Particularidades En Cuba. *Contexto*, *13*(19), 73–88. https://doi.org/10.29105/contexto13.19-7

Gillis, A. (2016). *¿Qué es la API REST (API RESTful)?* Search App Architecture. https://www.techtarget.com/searchapparchitecture/definition/RESTful-API

Hernández, R., Fernández, C., & Baptista, P. (2014). *Metodología de la investigación* (6th ed.). Mc Graw Hill. https://www.uca.ac.cr/wp-content/uploads/2017/10/Investigacion.pdf

Hodges, N. (2022). *Impacto del modelo SaaS en el desarrollo de software*. ComputerWorld. https://www.computerworld.es/tecnologia/impacto-del-modelo-saas-en-el-desarrollo-de-software

Jaiswal, M. (2019). Software Architecture and Software Design. *SSRN Electronic Journal*, 1–4. https://doi.org/10.2139/ssrn.3772387

Jimenez-Torres, V., Tello-Borja, W., & Rios-Patiño, J. (2014). Lenguajes de Patrones de Arquitectura de Software : Una Aproximación Al Estado del Arte Pattern Languages Software Architecture : An Approach To State of the Art. *Scientia et Technica Año XIX*, *19*(4), 371–376.

Jiménez, O. (2021). *La revolución del software que mueve al mundo*. Red Forbes y Forbes México. https://www.forbes.com.mx/red-forbes-la-revolucion-del-software-que-mueve-al-mundo/

K. Siva Prasad, R. (2020). generator-springboot. Obtenido de https://github.com/sivaprasadreddy/generator-springboot

Kapferer, S., & Zimmermann, O. (2020). Domain-specific language and tools for strategic domain-driven design, context mapping and bounded context modeling. *MODELSWARD 2020 - Proceedings of the 8th International Conference on Model-Driven Engineering and Software Development*, 299–306. https://doi.org/10.5220/0008910502990306

Kratzke, N., & Quint, P.-C. (2017). Understanding Cloud-native Applications after 10 Years of Cloud Computing - A Systematic Mapping Study The Journal of Systems and Software Understanding cloud-native applications after 10

years of cloud computing - A systematic mapping study. *The Journal of Systems and Software*, *126*, 1–16. https://doi.org/10.1016/j.jss.2017.01.001

Lähtevänoja, V. (2021). Communication Methods and Protocols Between Microservices on a Public Cloud Platform. *School of Science*, 1–57. https://aaltodoc.aalto.fi/bitstream/123456789/108243/1/master_Lähtevänoja_Vili_2021.pdf

Lewis, J. R. (2002). Psychometric Evaluation of the PSSUQ Using Data from Five Years of Usability Studies. International Journal of Human-Computer Interaction, 14(3&4), 463–488. DOI:10.1080/10447318.2002.9669130

Lewis, J., & Fowler, M. (2014). *Microservices*. MartinFowler.Com. https://martinfowler.com/articles/microservices.html

López, D., & Maya, E. (2017). Arquitectura de Software basada en Microservicios para Desarrollo de Aplicaciones Web. *Septima Conferencia de Directores de Tecnologia de Informacion 2017*, *7*, 1–12. http://dspace.redclara.net:8080/bitstream/10786/1277/1/93 Arquitectura de Software basada en Microservicios para Desarrollo de Aplicaciones Web.pdf

Meaurio, V., & Schmieder, E. (2013). La Arquitectura de Software en el Proceso de Desarrollo: Integrando MDA al Ciclo de Vida en Espiral. *Revista Latinoamericana de Ingenieria de Software*, *1*(4), 142–146. https://doi.org/10.18294/relais.2013.142-146

Microsoft (2022). Entity Framework Core. Recuperado el 21 de noviembre del 2022 de https://learn.microsoft.com/es-es/ef/core/

Microsoft (2022). Información general sobre ASP.NET MVC. Recuperado el 21 de noviembre del 2022 de https://learn.microsoft.com/es-es/aspnet/mvc/overview/older-versions-1/overview/asp-net-mvc-overview

Millett, S., & Tune, N. (2015). Patterns, Principles, and Practices of Domain-Driven Design. In *John Wiley & Sons, Inc.* Wrox. https://sd.blackball.lv/library/patterns_principles_and_practices_of_domain-driven_design_(2015).pdf

Ministerio de vivienda. (2021). Documento de arquitectura de software sistema de subsidio familiar de vivienda sisfv. *Minvivienda*. https://www.fiduoccidente.com/documents/116662/3200610/mvct_sad.pdf/73f7ef9b-64fa-ea76-9daa-1a6be72cd9bd?t=1628281110792

Pons, C., Giandini, R., & Pérez, G. (2010). *Desarrollo de Software dirigido por modelos, conceptos teóricos y su aplicación práctica*. Mc Graw Hill educación. https://ri.conicet.gov.ar/bitstream/handle/11336/127598/CONICET_Digital_Nro.958abf70-3f91-4d77-a306-e910c13cc4f4_B.pdf?sequence=5&isAllowed=y

Pressman, R. (2010). Ingeniería de Software un enfoque práctico. In *Journal of Chemical Information and Modeling* (7th ed.). Mc Graw Hill. http://cotana.informatica.edu.bo/downloads/ld-Ingenieria.de.software.enfoque.practico.7ed.Pressman.PDF

Rademacher, F., Sorgalla, J., & Sachweh, S. (2018). Challenges of Microservice Design. *IEEE Software*, *35*(3), 36–43. https://sci-hub.se/10.1109/MS.2018.2141028

Richards, M. (2015). Software Architecture Patterns. In *Home healthcare nurse* (1st ed.). O'Reilly Media, Inc. http://www.ncbi.nlm.nih.gov/pubmed/24798474

Richardson, C. (2015). *Introduction to Microservices*. NGINX. https://www.nginx.com/blog/introduction-to-microservices/

Richardson, C. (2018). *Microservices patterns with example in java*. Manning. https://thuvienpdf.com/microservices-patterns-with-examples-in-java

Rodríguez, A., Padilla, J., & Parra, H. (2018). Arquitectura basada en microservicios para aplicaciones web. *Revistas.Udistrital.Edu.Co*, *7*(2), 1–10. https://aaltodoc.aalto.fi/bitstream/123456789/108243/1/master_Lähteväinoja_Vili_2021.pdf

Roldán, D., Valderas, P., & Torres, V. (2018). *Microservicios: Un enfoque integrado*. Google Libros.

https://books.google.com.pe/books?id=2o6fDwAAQBAJ&printsec=frontcover&dq=Microservicios+Un+enfoque+integrado+pdf&hl=es&sa=X&ved=2ahUKEwi48ZXpiOH6AhV7s5UCHQYcApEQ6AF6BAgNEAI#v=onepage&q&f=false

Salesforce. (2022). REST API Developer Guide. *Salesforcedocs*. https://doi.org/10.1002/9781119421900.ch5

Santis, S. De, Florez, L., Nguyen, D. V., & Rosa, E. (2016). Evolve the Monolith to Microservices with Java and Node. *IBM Redbooks*, 125. http://www.redbooks.ibm.com/redbooks/pdfs/sg248358.pdf

Sauro, J., & Lewis, J. R. (2012). Quantifying the User Experience: Practical Statistics for User Research. Elsevier, USA. DOI:10.1016/B978-0-12-384968-7.00001-1

Sayago, J. (2018). Generador de Código Utilizando el Paradigma de Líneas de Producto Software Code Generator Using the Software Product Lines Paradigm. *Rev. Hallazgos21*, *3*(2), 190–212. https://revistas.pucese.edu.ec/hallazgos21/article/view/281

scrum.org (sf). ¿Qué es Scrum?. Recuperado el 18 de noviembre del 2022 de https://www.scrum.org/resources/what-is-scrum

Serna, E., & Hernán, J. (2016). Documentar la elicitación de requisitos: Una revisión sistemática. *Ingeniare*, *24*(4), 703–714. https://doi.org/10.4067/S0718-33052016000400014

Sharma, S. (2021). *Modern API Development with Spring and Spring Boot*. Packt Publishing. https://www.packtpub.com/product/modern-api-development-with-spring-and-spring-boot/9781800562479

Sommerville, I. (2005). Requerimientos Del Software. *Universidad Veracruzana*, 109–110. https://www.uv.mx/personal/fcastaneda/files/2015/08/F_Capitulo_5_Requerimientos_del_software.pdf

Sommerville, I. (2011). *Ingeniería de software 9*. PEARSON. https://gc.scalahed.com/recursos/files/r161r/w25469w/ingdelsoftwarelibro

9_compressed.pdf

Torre, C., Zorrilla, U., Calvarro, J., & Ramos, M. (2010). *Guía de Arquitectura N-Capas Orientada al Dominio con .NET 4.0*. Microsoft Architecture. https://resultados.qualab.com.pe/resources/guia.pdf

Velepucha, V., Flores, P., & Torres, J. (2019). MOMMIV: Modelo para descomposición de una arquitectura monolítica hacia una arquitectura de microservicios bajo el Principio de Ocultación de Información. *Revista Ibérica de Sistemas e …*, 1000–1010. https://www.proquest.com/openview/77d7d0f4e35e4177c485340f7c0ff283/1?pq-origsite=gscholar&cbl=1006393

Villadiego, R. (2020). *¿Por qué el software conquistó al mundo?* Forbes Colombia. https://forbes.co/2020/08/27/red-forbes/por-que-el-software-conquisto-al-mundo/

Vural, H., & Koyuncu, M. (2021). Does Domain-Driven Design Lead to Finding the Optimal Modularity of a Microservice? *IEEE Access*, *9*, 32721–32733. https://doi.org/10.1109/ACCESS.2021.3060895

ANEXOS

Anexo 1. Operacionalización de las variables

Variables	Tipo de variables	Definición de variables	Dimensiones o factores	Medición de indicador o cumplimiento	Definición de indicador
Herramienta tecnológica	Variable Independiente	Herramienta con interfaz web			
Modelado y generación de código de arquitecturas de software basadas en microservicios	Variable Dependiente	Generación de código de arquitecturas de software basadas en microservicios	Eficiencia	Tiempo en el modelado y generación de código.	Cantidad de tiempo que se demora durante la generación de código.
			Eficacia	Proceso de modelado y generación de código	Cumplimiento del proceso de modelado y generación de código
			Usabilidad	% de satisfacción post-Uso.	Alto porcentaje de satisfacción de facilidad de uso.

Anexo 2. Matriz de Consistencia

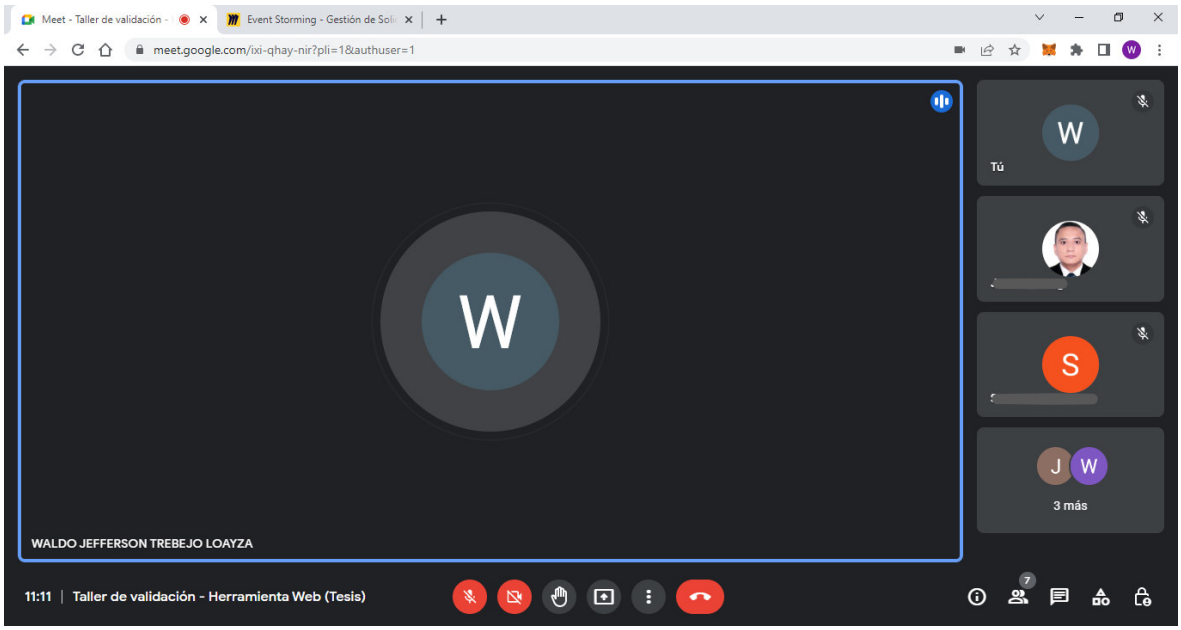
Problema	Objetivos	Hipótesis	Metodología
Problema general	**Objetivo general**	**Hipótesis general**	
¿Es posible implementar una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios para el proceso de desarrollo de proyectos de software y la migración de sistemas legados?	Implementar una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios para el desarrollo de proyectos de software y la migración de sistemas legados.	La implementación de herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios permitirá facilitar y agilizar el desarrollo de un proyecto de software y la migración de sistemas legados a los arquitectos y desarrolladores de software.	**Metodología de investigación** **Enfoque:** Cuantitativo. **Recolección de Datos:** La Encuesta. **Diseño:** No Experimental. **Alcances o estrategias:** Correlacional. **Población:** Profesionales de tecnologías de información dedicados al desarrollo de proyectos de software. **Muestra:** 10 profesionales conformados por arquitectos y desarrolladores de software. **Tipo de Muestreo:** No probabilístico.
Problemas específicos	**Objetivos específicos**	**Hipótesis especificas**	
¿Es posible desarrollar un proyecto software y la migración de sistemas legados de manera Eficaz mediante una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios?	Desarrollar un proyecto software y la migración de sistemas legados de manera Eficaz mediante una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios.	Se desarrollará un proyecto software y la migración de sistemas legados de manera Eficaz mediante el uso de la herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios.	
¿Es posible desarrollar un proyecto software y la migración de sistemas legados de manera Efectiva mediante una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios?	Desarrollar un proyecto software y la migración de sistemas legados de manera Efectiva mediante una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios.	Se desarrollará un proyecto software y la migración de sistemas legados de manera Efectiva mediante el uso de la herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios.	
¿Es posible desarrollar un proyecto software y la migración de sistemas legados mediante una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios con facilidad de uso?	Desarrollar un proyecto software y la migración de sistemas legados mediante una herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios con facilidad de uso.	Se desarrollará un proyecto software y la migración de sistemas legados mediante el uso de la herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios con facilidad de Uso.	
¿Es posible validar la herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios?	Validar la herramienta para el modelado y generación de código de arquitecturas de software basadas en microservicios.	Se logrará validar que la herramienta web es de mucha ayuda para facilitar y agilizar el desarrollo de un proyecto software y la migración de sistemas legados.	

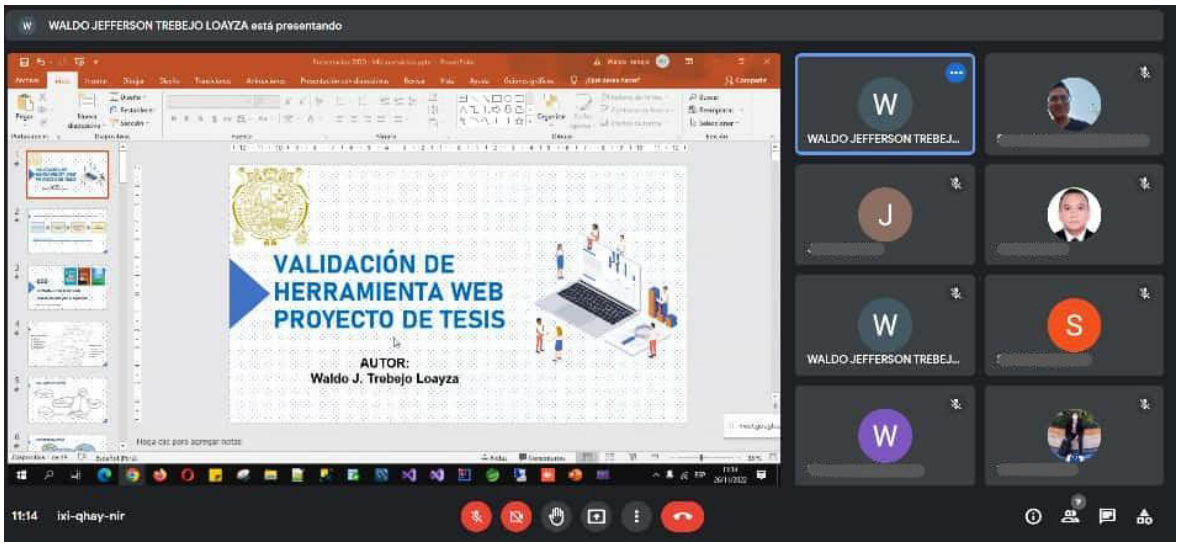
Anexo 3. Encuesta de Satisfacción de usuario Post-Uso

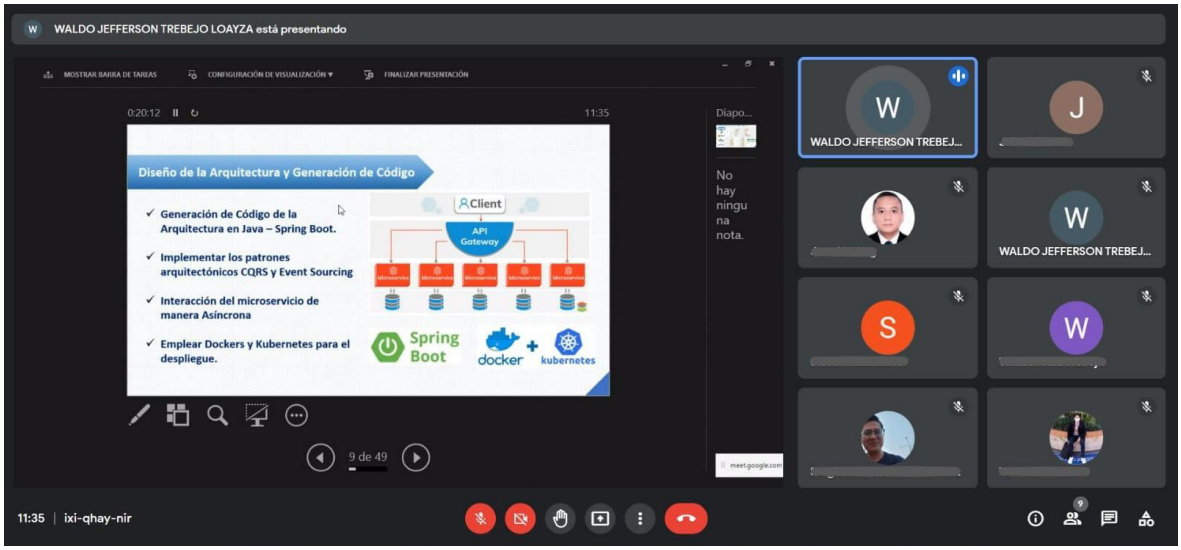
Nro.	PSSUQ (Post-Study System Usability Questionnaire) Versión 3 - Cuestionario de usabilidad del sistema posterior al estudio.	Totalmente de acuerdo						Totalmente en desacuerdo		Subescalas
		1	2	3	4	5	6	7	NA	
1	En general, estoy satisfecho con lo fácil que es usar la herramienta web.									Utilidad del sistema (SYSUSE)
2	Fue sencillo utilizar esta herramienta web.									
3	Pude completar las tareas y escenarios rápidamente usando esta herramienta web.									
4	Me sentí cómodo usando esta herramienta web.									
5	Fue fácil aprender a usar esta herramienta web.									
6	Creo que podría volverme productivo rápidamente usando esta herramienta web.									
7	La herramienta web dio mensajes de error que claramente me dijeron cómo solucionar problemas.									Calidad de la información (INFOQUAL)
8	Cada vez que cometía un error al usar la herramienta web, podía recuperarme fácil y rápidamente.									
9	La información (como ayuda en línea, mensajes en pantalla y otra documentación) proporcionada con esta herramienta web fue clara.									
10	Fue fácil encontrar la información que necesitaba.									
11	La información fue efectiva para ayudarme a completar las tareas y escenarios.									
12	La organización de la información en las pantallas de la herramienta web fue clara.									
13	La interfaz de esta herramienta web fue agradable.									Calidad de la interfaz (INTERQUAL)
14	Me gustó usar la interfaz de esta herramienta web.									
15	Esta herramienta web tiene todas las funciones y capacidades que espero que tenga.									
16	En general, estoy satisfecho con esta herramienta web.									

Anexo 4. Capturas del taller de validación de la herramienta

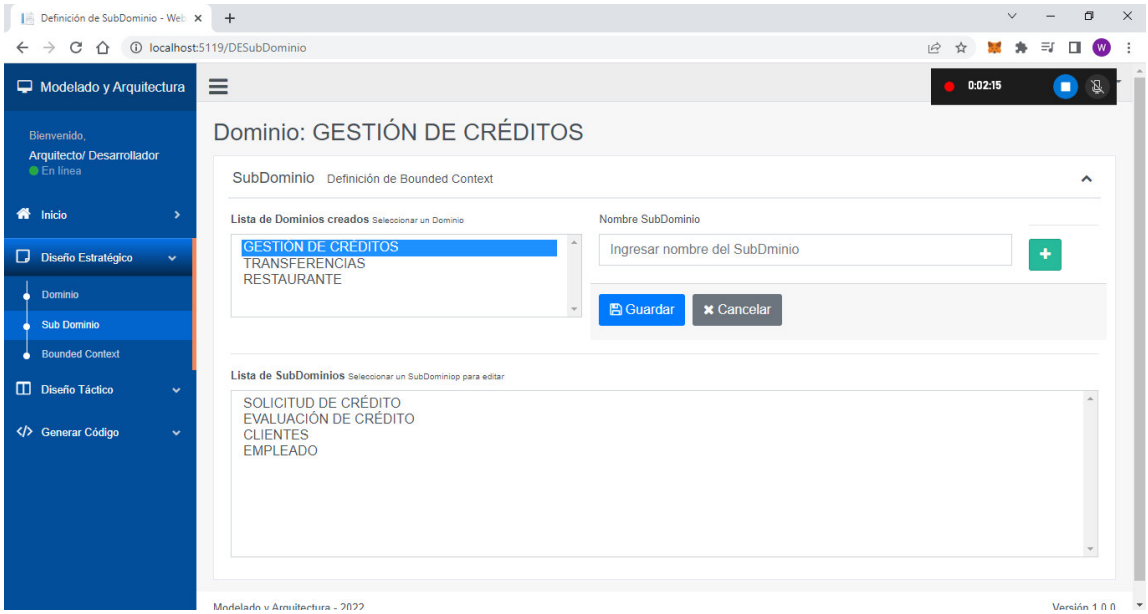
Primera reunión de capacitación para el uso de la herramienta web. Reunión realizado vía Google Meet.

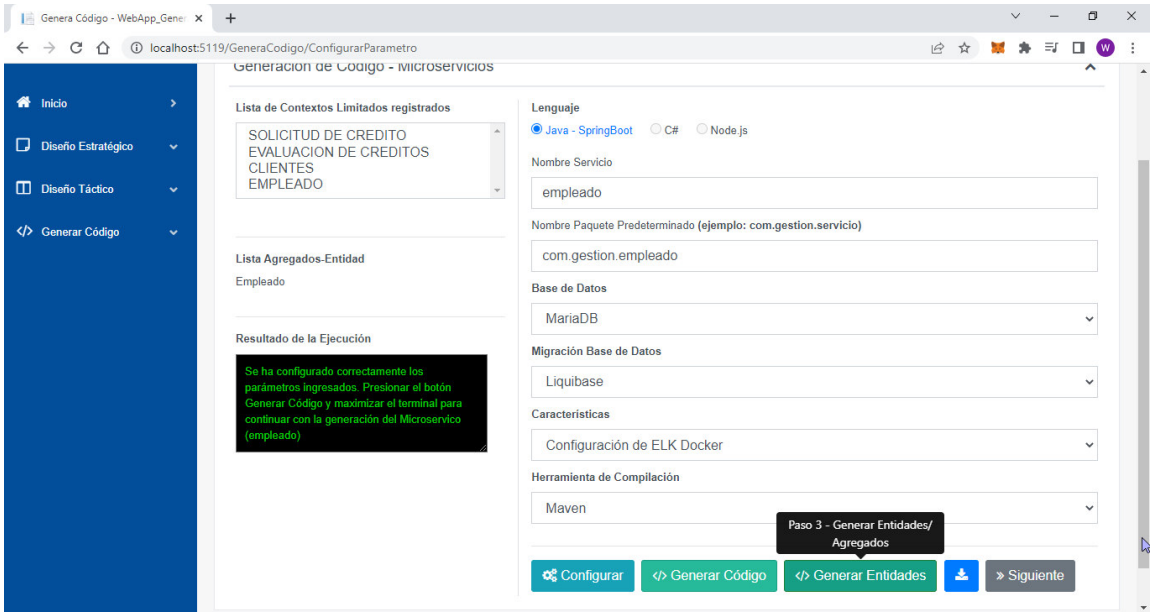


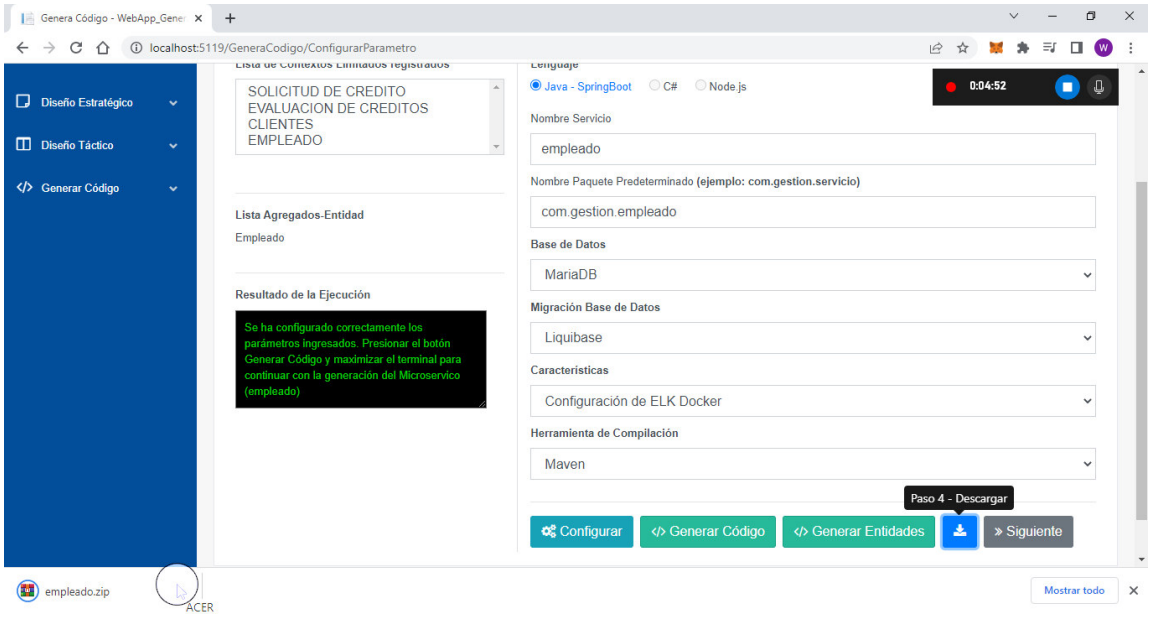




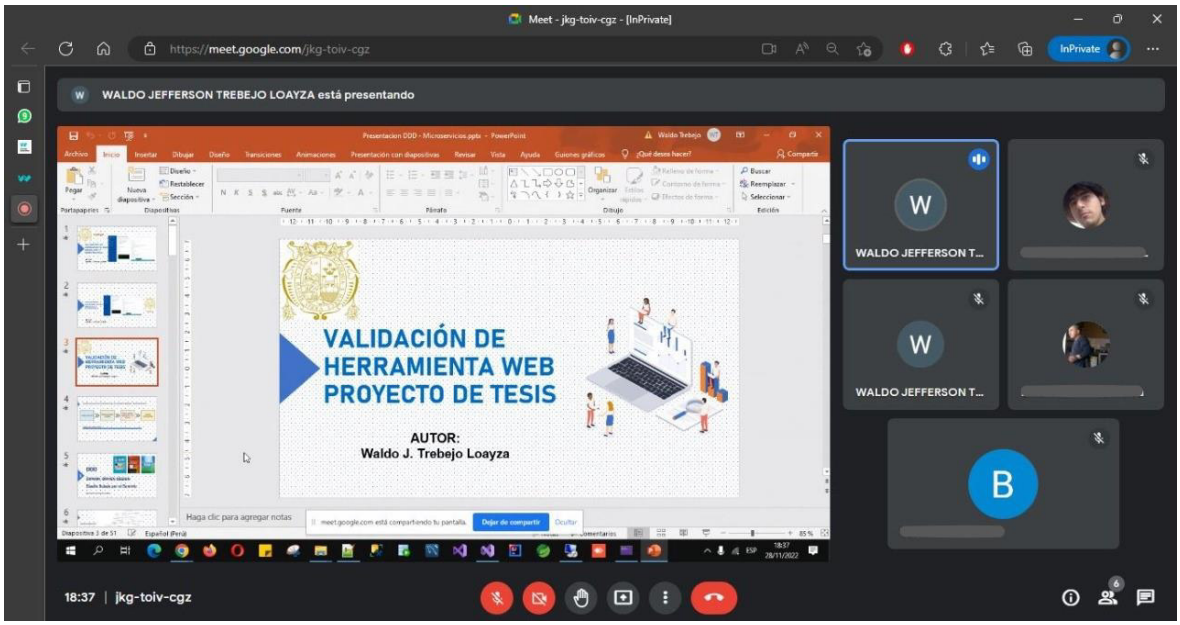
Presentación de la herramienta web a todos los participantes, donde se realizó ejercicios de modelado y generación del código propuesto en el proyecto de tesis.




Genera Código - WebApp_Gene
localhost:5119/GeneraCodigo/ConfigurarParametro
Generación de Código - Microservicios
Inicio
Diseño Estratégico
Diseño Táctico
Generar Código
Lista de Contextos Limitados registrados
SOLICITUD DE CREDITO
EVALUACION DE CREDITOS
CLIENTES
EMPLEADO
Lista Agregados-Entidad
Empleado
Resultado de la Ejecución
Se ha configurado correctamente los parámetros ingresados. Presionar el botón Generar Código y maximizar el terminal para continuar con la generación del Microservico (empleado)
Lenguaje
Java - SpringBoot
C#
Node.js
Nombre Servicio
empleado
Nombre Paquete Predeterminado (ejemplo: com.gestion.servicio)
com.gestion.empleado
Base de Datos
MariaDB
Migración Base de Datos
Liquibase
Características
Configuración de ELK Docker
Herramienta de Compilación
Maven
Paso 3 - Generar Entidades/ Agregados
Configurar
Generar Código
Generar Entidades
Siguiente

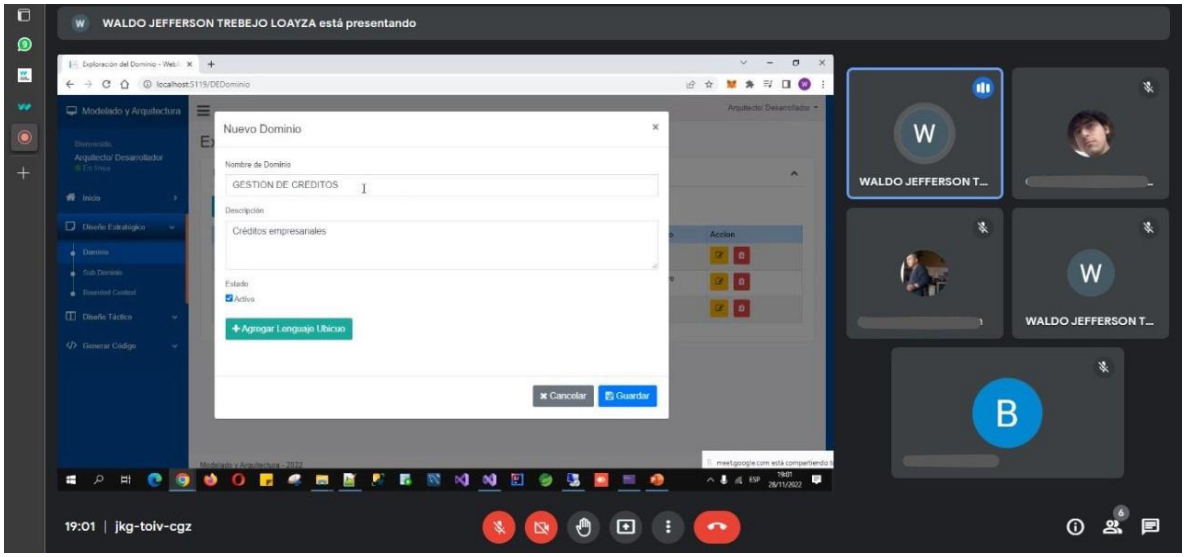

Genera Código - WebApp_Gene
localhost:5119/GeneraCodigo/ConfigurarParametro
Diseño Estratégico
Diseño Táctico
Generar Código
SOLICITUD DE CREDITO
EVALUACION DE CREDITOS
CLIENTES
EMPLEADO
Lista Agregados-Entidad
Empleado
Resultado de la Ejecución
Se ha configurado correctamente los parámetros ingresados. Presionar el botón Generar Código y maximizar el terminal para continuar con la generación del Microservico (empleado)
Java - SpringBoot
C#
Node.js
0:04:52
Nombre Servicio
empleado
Nombre Paquete Predeterminado (ejemplo: com.gestion.servicio)
com.gestion.empleado
Base de Datos
MariaDB
Migración Base de Datos
Liquibase
Características
Configuración de ELK Docker
Herramienta de Compilación
Maven
Paso 4 - Descargar
Configurar
Generar Código
Generar Entidades
Siguiente
empleado.zip
Mostrar todo

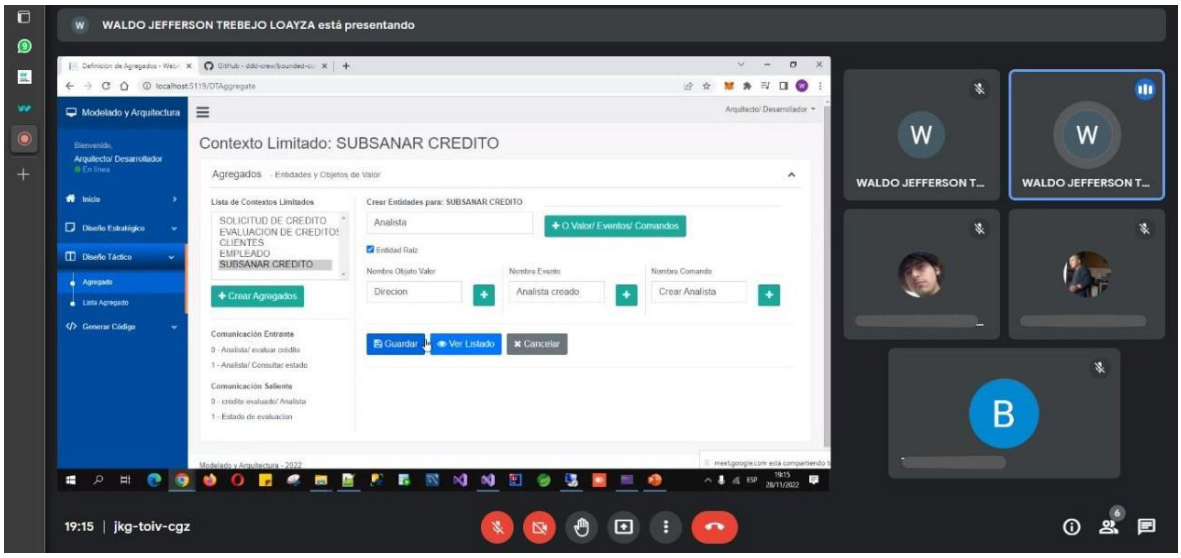
Segunda reunión de capacitación para el uso de la herramienta web. Reunión realizado vía Google Meet.

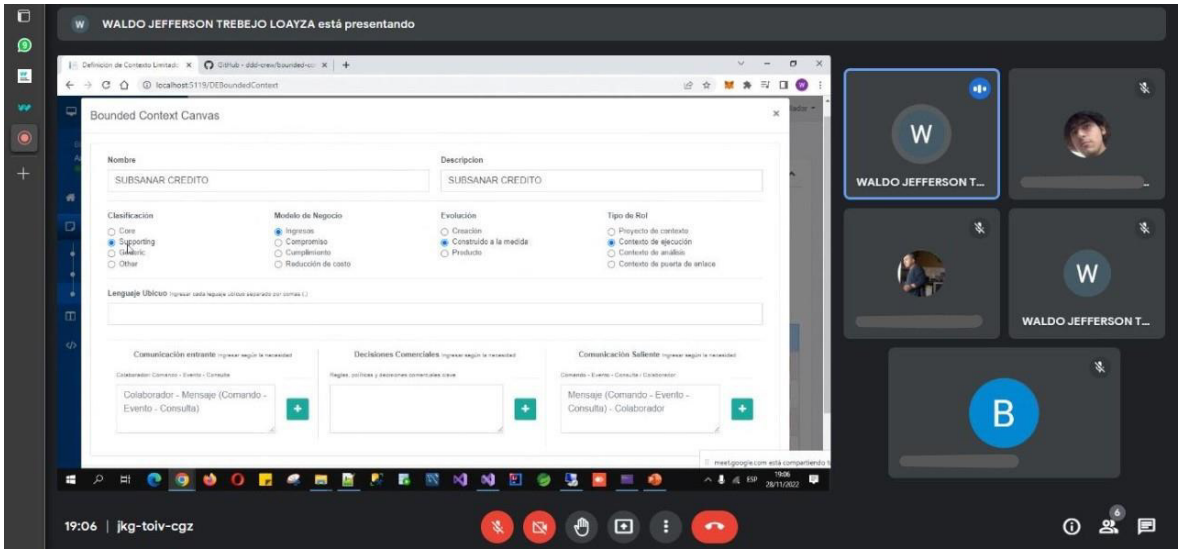


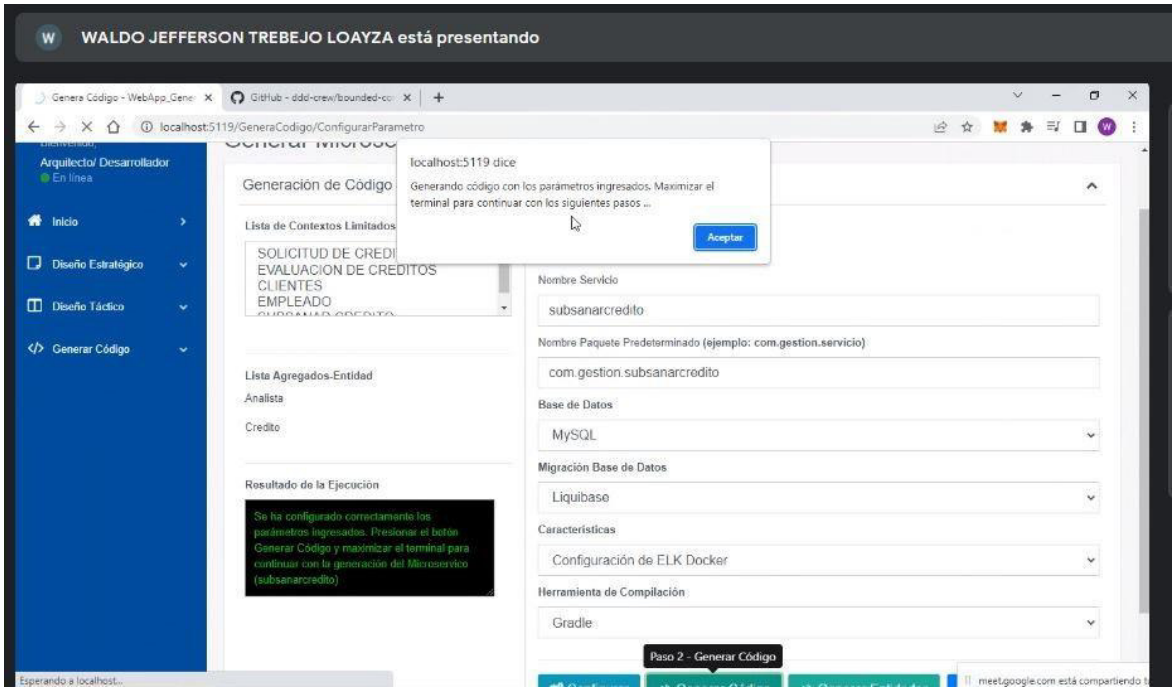


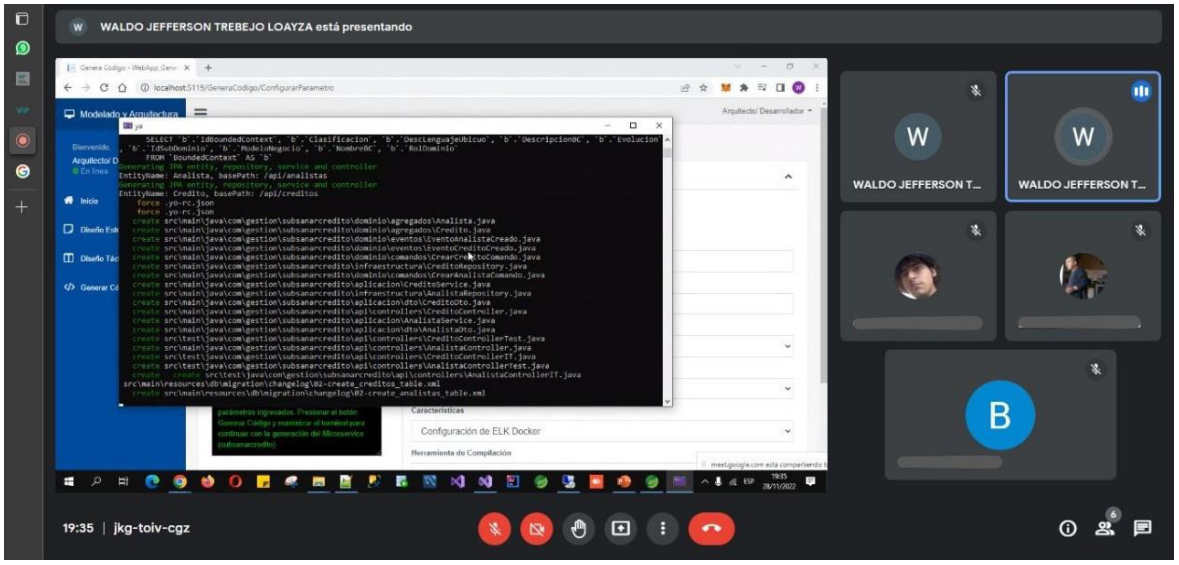
Presentación de la herramienta web a todos los participantes de la segunda reunión, donde se realizó el ejercicio de modelado y generación del código propuesto en el proyecto de tesis.

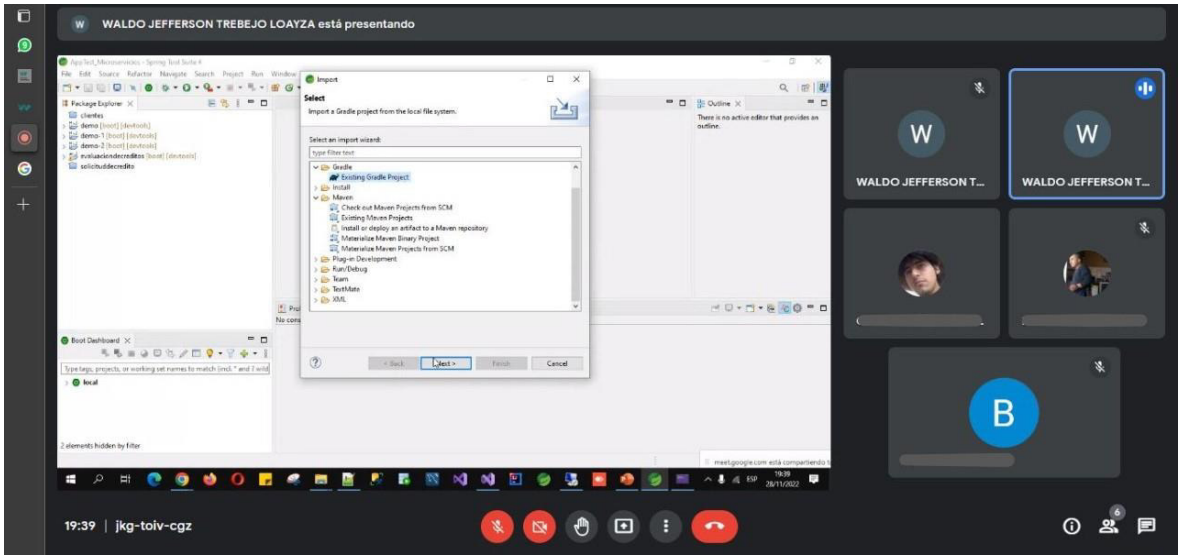

WALDO JEFFERSON TREBEJO LOAYZA está presentando
Nuevo Dominio
Nombre de Dominio
GESTION DE CREDITOS
Descripción
Créditos empresariales
Estado
Activo
Agregar Lenguaje Ubicuo
Cancelar
Guardar
WALDO JEFFERSON T...
19:01 | jkg-toiv-cgz


WALDO JEFFERSON TREBEJO LOAYZA está presentando
Contexto Limitado: SUBSANAR CREDITO
Agregados
Lista de Contextos Limitados
SOLICITUD DE CREDITO
EVALUACION DE CREDITOS
CLIENTES
EMPLEADO
SUBSANAR CREDITO
Crear Agregados
Crear Entidades para: SUBSANAR CREDITO
Analista
Direcion
Analista creado
Crear Analista
Guardar
Ver Listado
Cancelar
WALDO JEFFERSON T...
19:15 | jkg-toiv-cgz

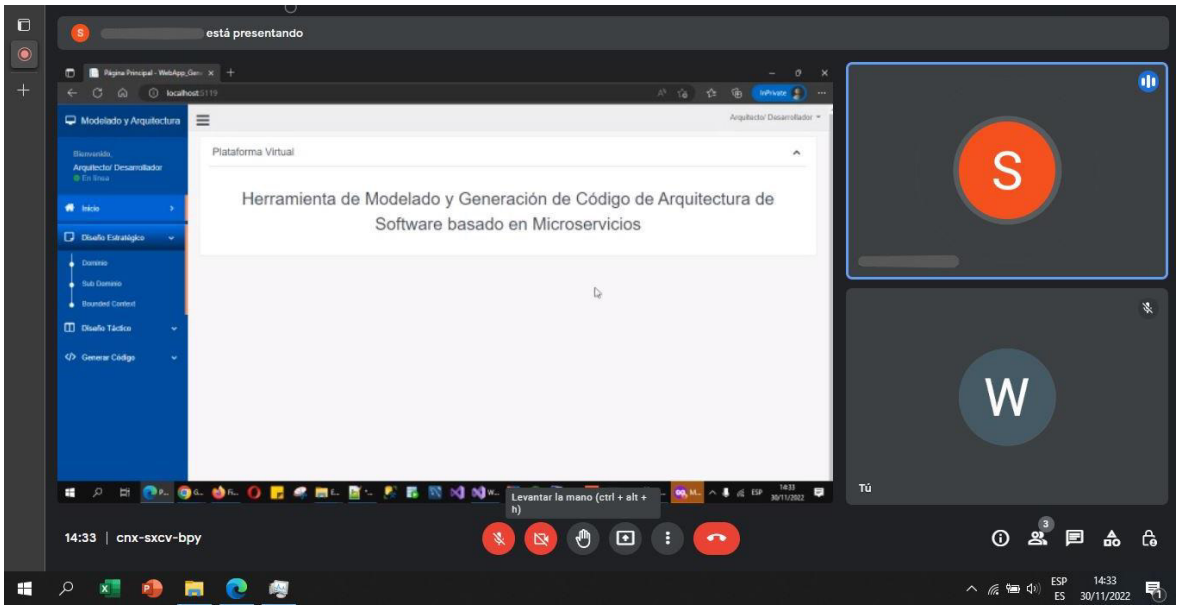

WALDO JEFFERSON TREBEJO LOAYZA está presentando
Bounded Context Canvas
Nombre
SUBSANAR CREDITO
Descripcion
SUBSANAR CREDITO
Clasificación
Modelo de Negocio
Evolución
Tipo de Rol
Lenguaje Ubicuo
Comunicación entrante
Decisiones Comerciales
Comunicación Saliente
Colaborador - Mensaje (Comando - Evento - Consulta)
Mensaje (Comando - Evento - Consulta) - Colaborador
WALDO JEFFERSON T...
19:06 | jkg-toiv-cgz

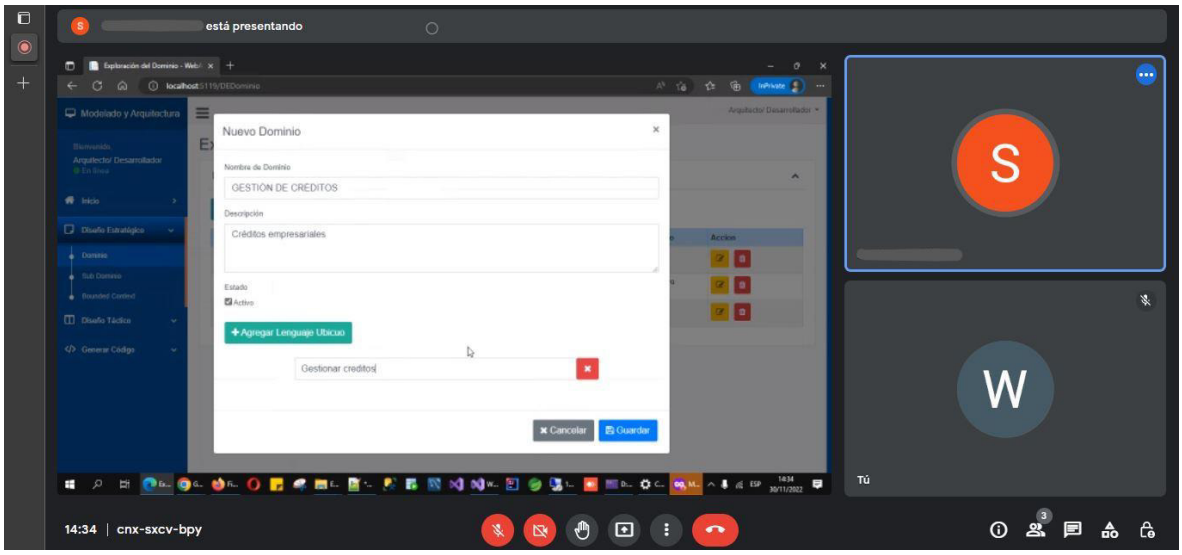

WALDO JEFFERSON TREBEJO LOAYZA está presentando
localhost:5119/GeneraCodigo/ConfigurarParametro
Arquitecto/ Desarrollador
En línea
Inicio
Diseño Estratégico
Diseño Táctico
Generar Código
Generación de Código
localhost:5119 dice
Generando código con los parámetros ingresados. Maximizar el terminal para continuar con los siguientes pasos ...
Aceptar
Lista de Contextos Limitados
SOLICITUD DE CREDI
EVALUACION DE CREDITOS
CLIENTES
EMPLEADO
Nombre Servicio
subsanarcredito
Nombre Paquete Predeterminado (ejemplo: com.gestion.servicio)
com.gestion.subsanarcredito
Lista Agregados-Entidad
Analista
Credito
Base de Datos
MySQL
Migración Base de Datos
Liquibase
Características
Configuración de ELK Docker
Herramienta de Compilación
Gradle
Resultado de la Ejecución
Se ha configurado correctamente los parámetros ingresados. Presionar el botón Generar Código y maximizar el terminal para continuar con la generación del Microservico (subsanarcredito)
Paso 2 - Generar Código
Esperando a localhost...
meet.google.com está compartiendo t

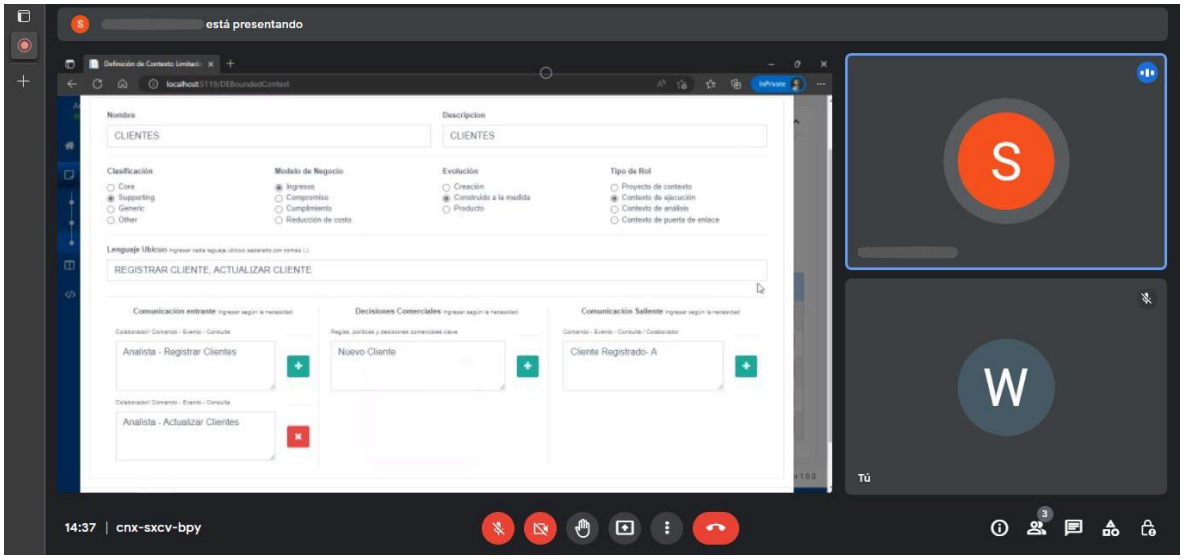

WALDO JEFFERSON TREBEJO LOAYZA está presentando
WALDO JEFFERSON T...
WALDO JEFFERSON T...
B
19:35 | jkg-toiv-cgz

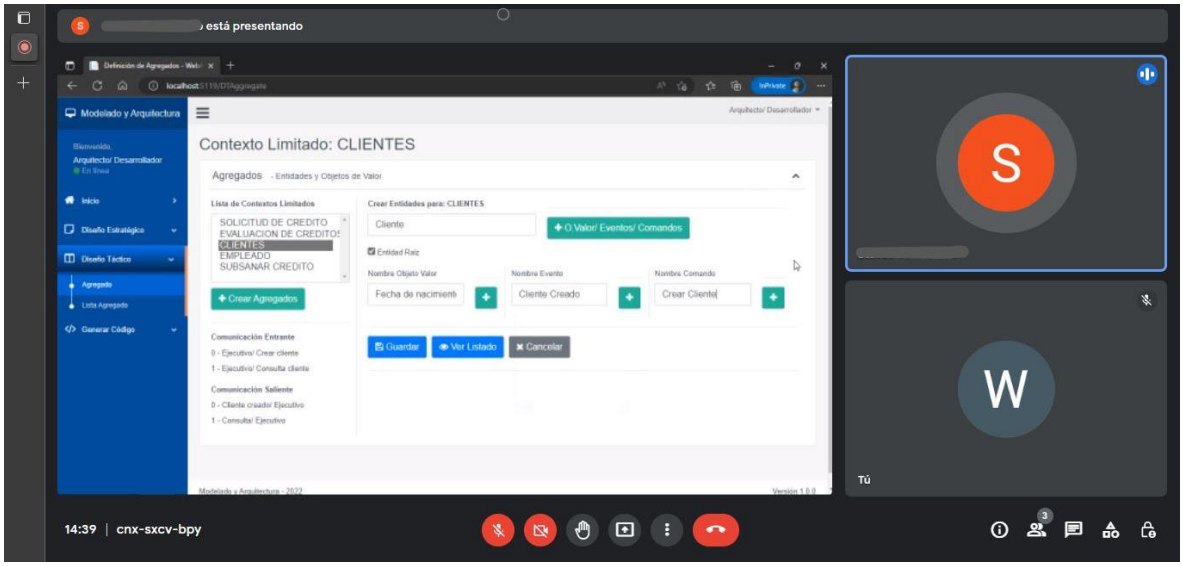

WALDO JEFFERSON TREBEJO LOAYZA está presentando
WALDO JEFFERSON T...
WALDO JEFFERSON T...
B
19:39 | jkg-toiv-cgz

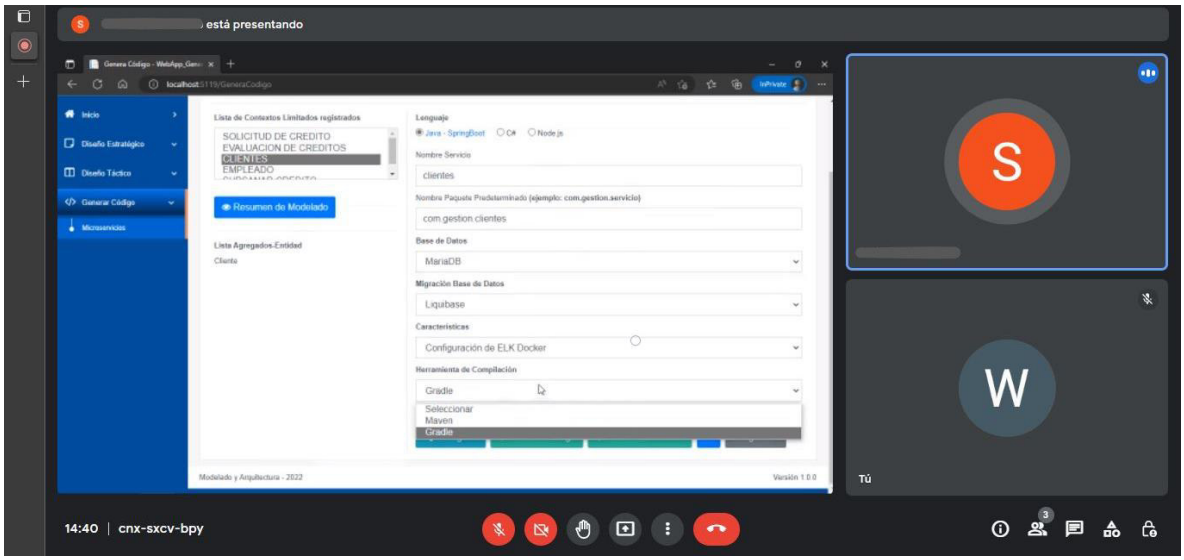
Captura de pantallas de las validaciones realizadas con la herramienta. Reunión realizado vía Google Meet.

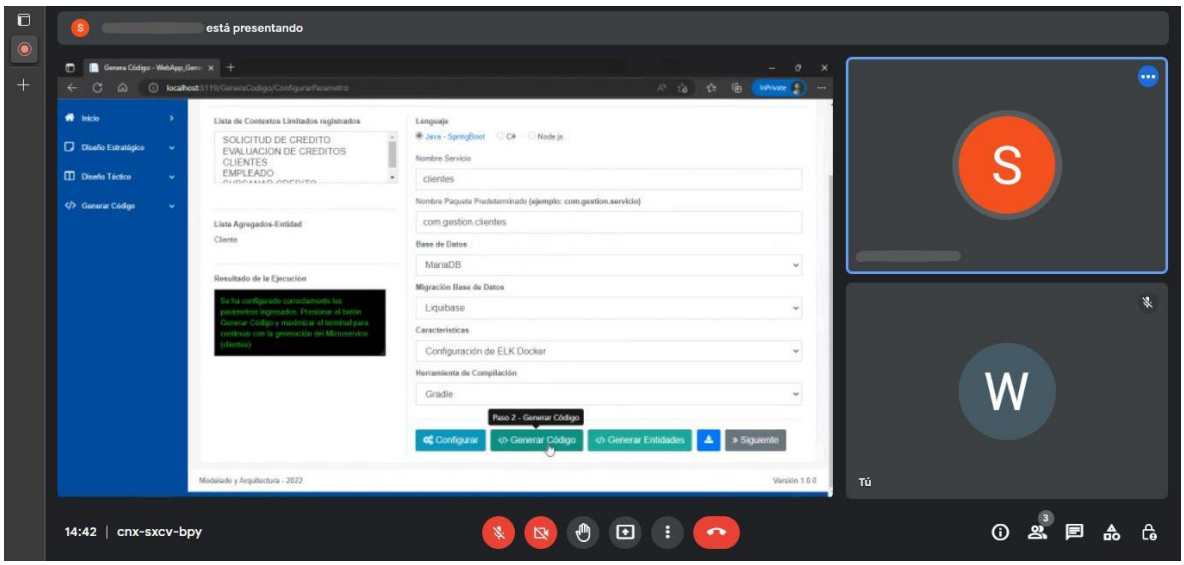


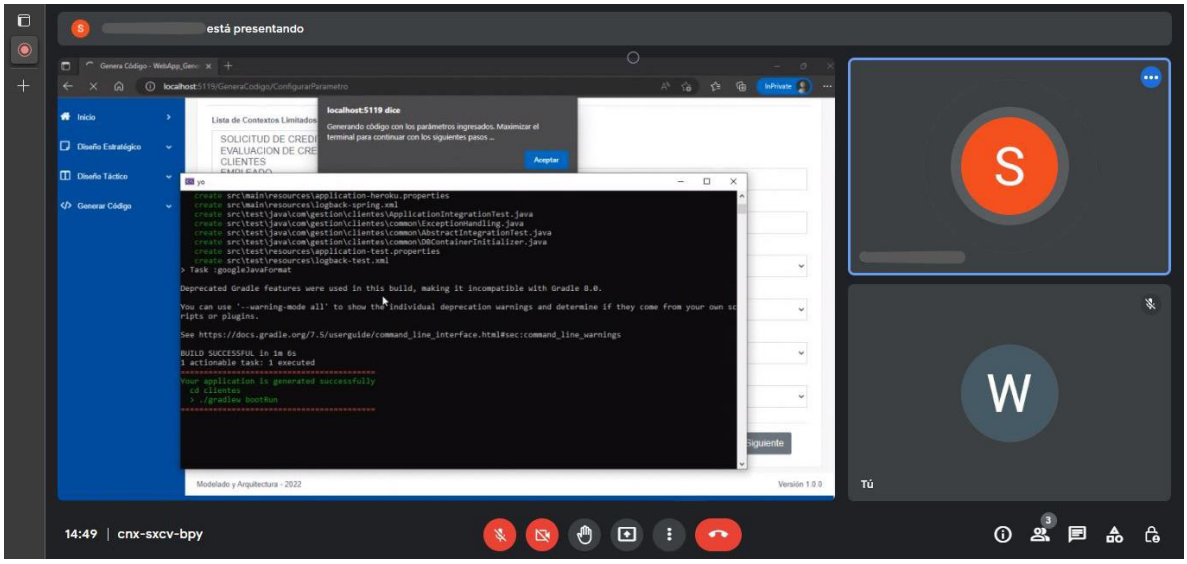


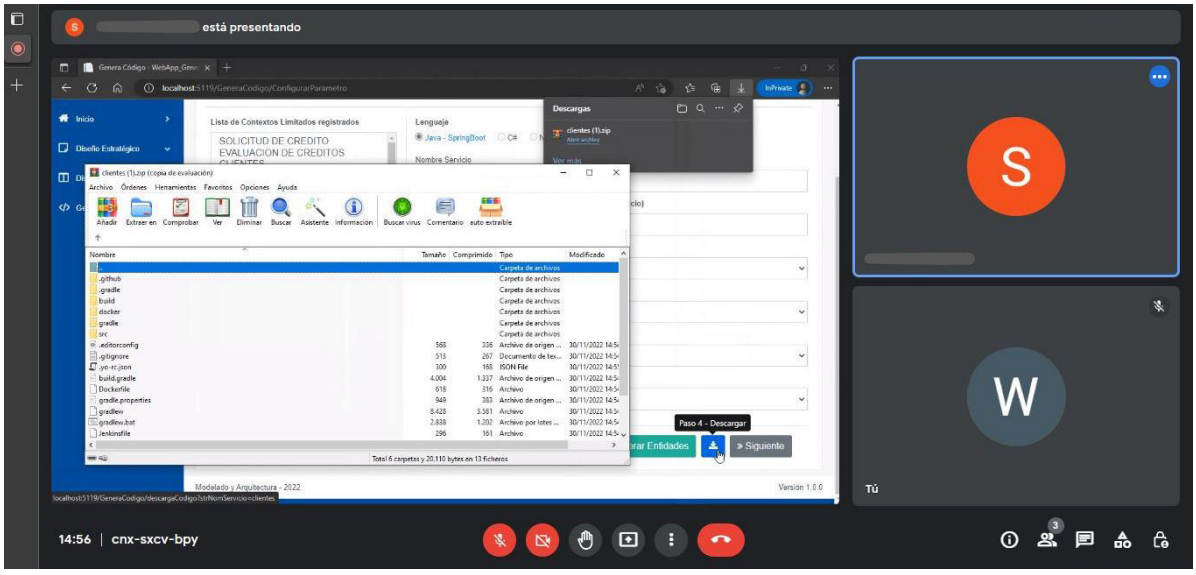



está presentando
Modelado y Arquitectura
Contexto Limitado: CLIENTES
Agregados
Lista de Contextos Limitados
SOLICITUD DE CREDITO
EVALUACION DE CREDITOS
CLIENTES
EMPLEADO
SUBSANAR CREDITO
Crear Agregados
Crear Entidades para: CLIENTES
Cliente
+ O.Valor/ Eventos/ Comandos
Entidad Raiz
Nombre Objeto Valor
Fecha de nacimient
Nombre Evento
Cliente Creado
Nombre Comando
Crear Cliente
Guardar
Ver Listado
Cancelar
Comunicación Entrante
0 - Ejecutivo/ Crear cliente
1 - Ejecutivo/ Consulta cliente
Comunicación Saliente
0 - Cliente creado/ Ejecutivo
1 - Consulta/ Ejecutivo
S
W
Tú
14:39 | cnx-sxcv-bpy


está presentando
Lista de Contextos Limitados registrados
SOLICITUD DE CREDITO
EVALUACION DE CREDITOS
CLIENTES
EMPLEADO
Resumen de Modelado
Lista Agregados-Entidad
Cliente
Lenguaje
Java - SpringBoot
C#
Node.js
Nombre Servicio
clientes
Nombre Paquete Predeterminado (ejemplo: com.gestion.servicio)
com.gestion.clientes
Base de Datos
MariaDB
Migración Base de Datos
Liquibase
Características
Configuración de ELK Docker
Herramienta de Compilación
Gradle
Seleccionar
Maven
Gradle
S
W
Tú
14:40 | cnx-sxcv-bpy


está presentando
Lista de Contextos Limitados registrados
SOLICITUD DE CREDITO
EVALUACION DE CREDITOS
CLIENTES
EMPLEADO
Lista Agregados-Entidad
Cliente
Resultado de la Ejecución
Lenguaje
Java - SpringBoot
C#
Node.js
Nombre Servicio
clientes
Nombre Paquete Predeterminado (ejemplo: com.gestion.servicio)
com.gestion.clientes
Base de Datos
MariaDB
Migración Base de Datos
Liquibase
Características
Configuración de ELK Docker
Herramienta de Compilación
Gradle
Paso 2 - Generar Código
Configurar
Generar Código
Generar Entidades
Siguiente
S
W
Tú
14:42 | cnx-sxcv-bpy


está presentando
localhost:5119 dice
Generando código con los parámetros ingresados. Maximizar el terminal para continuar con los siguientes pasos ...
Aceptar
Tú
14:49 | cnx-sxcv-bpy


está presentando
Paso 4 - Descargar
Tú
14:56 | cnx-sxcv-bpy